İçindekiler

1. GİRİŞ 6
2. VERİ ÖN İŞLEME 6
   2.1. Veri Setinin Yapısı ve Tutarlılık Kontrolleri 7
   2.2. Eksik Veri Analizi ve Tamamlama Yöntemleri 8
   2.3. Aykırı Değer Analizi 9
3. ÖZNİTELİK MÜHENDİSLİĞİ VE HEDEF DEĞİŞKEN 9
4. VERİ ÖLÇEKLENDİRME VE ZAMAN SERİSİ BÖLME 11
5. PCA VE KERNEL PCA ANALİZİ 11
   5.1. PCA - Scree Plot Analizi 11
   5.2. PCA Component Loadings 13
   5.3. İlk İki Bileşene Göre Veri Dağılımı 13
   5.4. Kernel PCA (RBF) Veri Dağılımı 14
   5.5. Kernel PCA (Polynomial) Veri Dağılımı 15
   5.6. PCA ve Kernel PCA Karşılaştırması 15
6. PCA MODEL DEĞERLENDİRMESİ VE FİNANSAL ÇIKARIMLAR 16
7. KEŞİFSEL VERİ ANALİZİ (EDA) 16
   7.1. Kütüphanelerin Yüklenmesi 16
   7.2. Grafik Ayarları 17
   7.3. Veri Yükleme ve Hazırlık 17
   7.4. Temel İstatistikler 17
   7.5. Frekans Analizi 19
   7.6. Kategorik Değişken Görselleştirmesi 19
   7.7. Veri Dağılımı ve Desen Analizi 20
      7.7.1. Zaman Serisi Eğilimi 20
      7.7.2. Günlük Getiri Dağılımı 21
   7.8. İlişki Matrisleri 22
      7.8.1. Korelasyon Matrisi – Isı Haritası 22
      7.8.2. Kutu Grafikleri — Günlük Getiri Dağılımı (Sayısal ve Kategorik Değişken İlişkisi) 24
      7.8.3. Kutu Grafikleri — Oynaklık Dağılımı (Sayısal ve Kategorik Değişken İlişkisi) 26
   7.9. Hipotez Geliştirme ve Raporlama Özeti 27
8. MAKİNE ÖĞRENİMİ MODELLERİ VE SONUÇLAR (SVM) 28
   8.1. Destek Vektör Makineleri (Support Vector Machines - SVM) Uygulaması 28
      8.1.1. Model Parametreleri ve Gerekçesi 28
      8.1.2. Uygulanan Python Kod Blokları 29

9. SONUÇLAR VE KRİTİK DEĞERLENDİRME 31
9.1. Metrik Özeti ve Bulgular 31
9.2. Performans ve Overfitting Analizi 32
9.3. Hata Matrisi ve Modelin Hata Tipleri 33
10. SMV MODEL DEĞERLENDİRME VE FİNANSAL ÇIKARIMLAR 34
11. LOJİSTİK REGRESYON MODELLEMESİ 34
11.1. Kütüphanelerin Yüklenmesi 34
11.2. Veri Yükleme 34
11.3. Hedef Değişkenin Hesaplanması 35
11.4. Özellik ve Hedef Değişkenlerin Ayrılması 35
11.5. Veri Bölme ve Ölçeklendirme 36
11.6. Model Eğitimi (Parametrelerin Seçilmesi) 36
11.7. Model Performansının Eğitim Verisi Üzerinde Değerlendirilmesi (Overfitting Kontrolü) 37
11.8. Model Performansının Test Seti Üzerinde Değerlendirilmesi 37
11.9. Model Performansının Yorumlanması ve Detaylı Analiz 38
11.9.1. Overfitting Yorumu 38
11.9.2. AUC Yorumu 38
11.9.3. Detaylı Performans Analizi 39
11.10. Grafiksel Analizler 40
11.10.1. Özelliklerin Lojistik Regresyon Katsayıları 40
11.10.2 ROC Eğrisi (Receiver Operating Characteristic) 41
11.11. Kullanılmayan Diğer Modellerin Kapsamlı Analizi 42
11.12. Bilgisayısal Sonuçlar ve İstatistiksel Değerlendirme 43
11.12.1. Eğitim ve Test Seti Performans Karşılaştırması 43
11.12.1 Test Seti Detaylı Sınıflandırma Raporu (Precision, Recall, F1) 43
11.13. Yorumlar ve Analitik Çıkarımlar 44
11.13.1. Model Stabilitesi ve Genelleme Yeteneği (Varyans/Bias Değerlendirmesi) 44
11.13.2. Ayırt Edicilik Gücü ve Finansal Önemi (AUC Değerlendirmesi) 44
11.13.3. Ticari Uygulama Yorumu (Sınıf 1 Kesinlik Değerlendirmesi) 45
11.13.4. Zayıf Performansa Dayalı Gelecek Model Seçimi Kararı 45
12. KARŞILAŞTIRMALI ANALİZ VE PCA'LI MODELLERİN DEĞERLENDİRİLMESİ 45
12.1. Kullanılan Modeller ve Deneysel Kurulum 46
12.2 Lojistik Regresyon Sonuçları 49
12.3 k-NN Sonuçları 52
12.4 PCA'lı ve PCA'sız Modellerin Accuracy Karşılaştırması 54
12.5 PCA'lı ve PCA'sız Modellerin AUC Karşılaştırması 55

12.6 Genel Değerlendirme ve Sonuç 56
13. EN YAKIN KOMŞU (k-NN) MODELLEMESİ 57
13.1 Kütüphanelerin Yüklenmesi 57
13.2. Veri Hazırlığı 58
13.3. Parametre Seçimi (En İyi 'k' Değerini Bulma) 58
13.4. Grafik Hazırlama (Elbow Method Grafiği) 59
13.5. Parametre Seçimi Yorumu ve Karar 60
13.6. Sonuçları Hesaplama (Seçilen Model: k=1 için) 61
13.7. Tahminlerin Hesaplanması (Olasılık ve Sınıf) 61
13.8. Eğitim Seti Metriklerinin Hesaplanması 62
13.9. Test Seti Metriklerinin Hesaplanması 62
13.10. Karmaşıklık Matrisi (Confusion Matrix) 63
13.11. Performans Değerlendirmesi ve Yorumlama 64
14. Destek Vektör Makineleri (SVM) – RBF (Gauss) Çekirdeği ve Parametre Optimizasyonu 64
14.1. Model Kurulumu ve Parametre Optimizasyonu 65
14.1.1 Hiperparametre Optimizasyonu (Grid Search) 65
14.2. Test Seti Performans Sonuçları 66
14.3 Confusion Matrix Analizi 67
14.4. Eğitim – Test Karşılaştırması ve Modelin Yorumlanması 69
14.5 Confusion Matrix Analizi (Eğitim Verisi) 70
14.6 Genel Değerlendirme (SVM – RBF) 70
15. MODELLEME VE YÖNTEMLER 71
15.1 Ortam Kurulumu ve Veri Yükleme 71
15.2 Lojistik Regresyon Uygulaması 72
15.2.1 Kod Bloğu (Model Tanımı ve Eğitimi) 72
15.2.2 Kod Çıktısı 73
Yorum: Lojistik Regresyon (LR) Performansı 74
15.3 K-NN Sınıflandırma (K=5) Uygulaması 75
15.3.1 Kod Bloğu (Model Tanımı ve Eğitimi) 75
15.3.2 Kod Çıktısı 76
15.4. AdaBoost Sınıflandırma Uygulaması 77
15.4.1 Gerekli Kütüphanelerin Yüklenmesi 77
15.4.2 Zayıf Öğrenici Tanımı ve AdaBoost Eğitimi 77
15.5 Performans Metrikleri 79
15.5.1. Kod Bloğu (Model Tanımı ve Eğitimi) 81
15.5.2. Kod Çıktısı 82

15.5.3 Algoritmaların Test Performansı Karşılaştırması 85
15.6 Modellerin Karşılaştırılması ve Değerlendirme 85
16. SONUÇ ve ÖNERİLER 87
16.1. Giriş 87
16.2. Ampirik Bulguların Değerlendirilmesi 87
16.3. Çalışmanın Nihai Çıkarımları ve Finansal Çözümleme 87
16.4. Gelecek Çalışmalar ve Metodolojik Öneriler 88
17. KAYNAKÇA 89

1. GİRİŞ

Finansal piyasaların karmaşık, dinamik ve gürültülü yapısı, gelecekteki fiyat hareketlerinin tahmin edilmesini mühendislik ve veri bilimi alanındaki en zorlu problemlerden biri haline getirmiştir. Geleneksel ekonometrik modeller, verilerdeki doğrusal olmayan örüntüleri yakalamakta genellikle yetersiz kalmaktadır. Bu bağlamda, Makine Öğrenmesi algoritmaları, geçmiş verilerden öğrenerek piyasa yönünü tahmin etmede güçlü bir alternatif olarak öne çıkmaktadır.
Bu çalışmada, Toyota Motor Corporation'ın New York Borsası'nda (NYSE: TM) işlem gören hisse senedi verileri kullanılarak finansal piyasalarda bir sonraki günkü fiyat yönünün (Yükseliş/Düşüş) tahmin edilmesi amaçlanmıştır.Analiz kapsamında kullanılan veri seti Kaggle platformundan temin edilmiştir. Veri seti, Toyota'nın 1980 - 2024 yılları arasındaki günlük işlem kayıtlarını içermekte olup, toplamda 11292 gözlemden oluşmaktadır. Bu veri seti; Açılış, Kapanış, En Yüksek, En Düşük fiyatlar ve İşlem Hacmi gibi temel finansal değişkenleri barındırmaktadır.
Toyota, küresel otomotiv sektörünün önde gelen şirketlerinden biri olması nedeniyle, hisse senedi fiyat hareketleri hem makroekonomik göstergelerden hem de sektörel dinamiklerden etkilenmektedir. Bu özellik, finansal zaman serisi tahmin modellerinin performansını test etmek için uygun bir vaka çalışması oluşturmaktadır. Çalışmanın deneysel tasarımı, model performanslarını karşılaştırmalı olarak analiz etmek üzerine kurgulanmış olup şu algoritmaları içermektedir:

1. **Temel Modeller:** Model başarımı için bir referans noktası oluşturmak amacıyla Lojistik Regresyon ve k-En Yakın Komşu algoritmaları kullanılmıştır.
2. **İleri Seviye Modeller:** Finansal verinin doğrusal olmayan yapısını modellemek için Destek Vektör Makineleri (SVM) tercih edilmiş; Lineer, Polinom ve RBF çekirdekleri ayrı ayrı test edilerek en iyi hiperparametreler optimize edilmiştir.

Çalışmanın bir diğer temel odağı, yüksek boyutlu teknik göstergelerin model üzerindeki etkisini incelemektir. Bu amaçla, Temel Bileşen Analizi (PCA) ile boyut indirgeme uygulanmış; ayrıca verinin farklı uzaylardaki dağılımını incelemek için Kernel PCA (RBF ve Polynomial) yöntemlerinin sınıf ayrışabilirliğine etkisi analiz edilmiştir.
Bu proje, bahsi geçen yöntemleri bütüncül bir yaklaşımla ele alarak; *"Boyut indirgeme ve karmaşık çekirdek fonksiyonları, rassal yürüyüş (Random Walk) sergileyen bir piyasada tahmin başarısını artırabilir mi?"* sorusuna mühendislik perspektifiyle yanıt aramaktadır.

2. VERİ ÖN İŞLEME

Bu bölümde, ham finansal verilerin makine öğrenmesi algoritmalarına uygun hale getirilmesi sürecinde uygulanan veri temizleme, dönüştürme ve öznitelik çıkarımı adımları detaylandırılmıştır.

2.1. Veri Setinin Yapısı ve Tutarlılık Kontrolleri

```
df = pd.read_csv('Toyota_Data.csv')

print(df.head())
print(df.describe())
print(df.info())
print(df.shape)
print(df.dtypes)

df['Date'] = pd.to_datetime(df['Date'])
print(df.dtypes)
df = df.sort_values(by='Date').reset_index(drop=True)
duplicated_dates =df.duplicated(subset=['Date']).sum()

print(f"Tekrar eden tarih sayısı: {duplicated_dates}")
if duplicated_dates > 0:
    df = df.drop_duplicates(subset=['Date'], keep='first')

logical_errors = (df['Low'] > df['High']).sum()
print(f"Mantıksal hata sayısı ('Low' > 'High'): {logical_errors}")
```

Görsel 2.1.1.: Veri ön işleme

Analiz sürecinin ilk aşamasında, ham veri seti yüklenmiş ve yapısal özellikleri incelenmiştir. Görsel 2.1.1.'de detaylandırılan ön işleme adımlarında, veri kalitesini ve modelin zaman algısını güvence altına almak için aşağıdaki prosedürler uygulanmıştır:

1. **Zaman Serisi Yapılandırması:** Ham veride Görsel 2.1.2.'de gözüktüğü üzere 'object' tipinde bulunan tarih değişkeni, zaman serisi analizinin gerekliliği olarak 'datetime' formatına dönüştürülmüştür. Ardından veri seti kronolojik sıraya
2. göre yeniden indexlenmiştir. Geçmiş veriye dayalı gelecek tahmini yapılacağı için, sıralamanın doğruluğu modelin nedensellik ilişkisini kurabilmesi açısından kritiktir.

2. **Tekrar Eden Kayıt Kontrolü:** Aynı işlem gününe ait birden fazla kayıt bulunması modelde sapmaya yol açabileceğinden, tarih bazlı tekrarlayan kayıtlar taranmış ve Görsel 2.1.3'de görüldüğü üzere veri seti tekilleştirilmiştir.

3. **Mantıksal Tutarlılık:** Finansal piyasa dinamiklerine aykırı olan *"Günlük En Düşük Fiyat > En Yüksek Fiyat"* (Low > High) durumu kontrol edilmiştir. Görsel 2.1.3.'teki çıktıda görüldüğü üzere, veri seti bu tür mantıksal hatalardan arındırılmış ve analize hazır hale getirilmiştir.

```
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Date       11291 non-null  object
 1   Adj Close  11291 non-null  float64
 2   Close      11291 non-null  float64
 3   High       11291 non-null  float64
 4   Low        11291 non-null  float64
 5   Open       11291 non-null  float64
 6   Volume     11291 non-null  int64
```

Görsel 2.1.2.: Veri Ön İşleme Çıktısı 1

```
Tekrar eden tarih sayısı: 0
Mantıksal hata sayısı ('Low' > 'High'): 0
```

Görsel 2.1.3.: Veri Ön İşleme Çıktısı 2

2.2. Eksik Veri Analizi ve Tamamlama Yöntemleri

```
df['Open'] = df['Open'].replace(0, np.nan)
df['Volume'] = df['Volume'].replace(0, np.nan)

print("0'lar NaN'a çevrildikten sonra eksik veri durumu:")
print(df.isnull().sum())

cols_to_fill = ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']
df[cols_to_fill] = df[cols_to_fill].bfill()
```

Görsel 2.2.1.: Eksik Veri Analizi

Analiz sürecinde, ham veride hatalı olarak '0' girilen değerler Görsel 2.2.1.'deki işlemlerden sonra tespit edilmiştir. Görsel 2.2.2.'deki analiz çıktısında görüldüğü üzere, bu değerler NaN formatına dönüştürüldüğünde 'Open' değişkeninde 2.981 adet, 'Volume' değişkeninde 133 adet veri kaybı olduğu anlaşılmıştır.

```
0'lar NaN'a çevrildikten sonra eksik veri durumu:
Date            0
Adj Close       0
Close           0
High            0
Low             0
Open         2981
Volume        133
dtype: int64
```

Görsel 2.2.2.: Eksik Veri Durumunu Gösteren Çıktı

Makine öğrenmesi modellerinin (SVM, PCA) boş veri (NaN) ile çalışması mümkün olmadığından, bu boşlukların doldurulması zorunludur. Ancak verinin finansal trend yapısını bozmamak adına, yapay ortalamalar atamak yerine Tablo 2.2.1.'de belirtilen Geriye Dönük Doldurma (Backward Fill) yöntemi uygulanmıştır. Bu sayede hatalı değerler, zaman serisinin akışına uygun mantıklı verilerle onarılmıştır.

Tablo 2.2.1.: Eksik Veri Tamamlama Yöntemi ve Gerekçesi

Değişken	Eksik Veri Nedeni	Uygulanan Yöntem	Gerekçe
Open, Volume	0 Değerleri	Backward Fill	Zaman serisi sürekliliğini korur
High, Low, Close	Ham Veri Eksiklikleri	Backward Fill	Trend bozulmasını önler

Zaman serisi analizlerinde yaygın olarak kullanılan ortalama atama veya lineer interpolasyon yöntemleri, verinin varyansını yapay olarak düşürebilmekte ve volatilite bilgisini kaybettirebilmektedir. Bu nedenle çalışmada, Geriye Dönük Doldurma (Backward Fill – bfill) yöntemi tercih edilmiştir. Little ve Rubin (2019) tarafından belirtildiği üzere, bfill yöntemi özellikle kısa vadeli fiyat boşluklarının doldurulmasında trend yapısını ve zaman serisi bütünlüğünü koruma avantajına sahiptir.

2.3. Aykırı Değer Analizi

```
outliers = df[abs(df['Daily_Change']) > 0.50]
print("Muhtemel veri giriş hataları: ")
print(outliers)
```

Görsel 2.3.1.: Aykırı Değer Analizi

Finansal veri setlerinde, veri giriş hataları veya anlık piyasa çöküşleri nedeniyle gerçekçi olmayan fiyat hareketleri gözlemlenebilmektedir. Bu durumu kontrol etmek amacıyla, günlük kapanış fiyatlarında %50'yi aşan anormal dalgalanmalar Görsel 2.3.1.'de göründüğü şekilde taranmıştır. Görsel 2.3.2.'de yer alan analiz çıktısında, veri setinde "0" adet olağandışı değişim tespit edilmiştir. Bu kontrol, modelin aşırı uç değerlerden negatif etkilenmesini önlemek amacıyla yapılmıştır.

```
Muhtemel veri giriş hataları:
Empty DataFrame
```

Görsel 2.3.2.: Aykırı Değer Analizi Çıktısı

3. ÖZNİTELİK MÜHENDİSLİĞİ VE HEDEF DEĞİŞKEN

Modelin piyasa dinamiklerini öğrenebilmesi ve fiyat hareketlerini tahmin edebilmesi için, ham fiyat verileri üzerinden teknik analiz literatürüne dayanan öznitelikler türetilmiştir. Bu işlem adımlarını içeren kod bloğu Görsel 3.1.'de sunulmuştur.

```
df['SMA_20'] = df['Close'].rolling(window=20).mean()
df['SMA_50'] = df['Close'].rolling(window=50).mean()
df['Daily_Return'] = df['Close'].pct_change()

df['Volatility_20'] = df['Daily_Return'].rolling(window=20).std()

df_temp = df.dropna()
df_final = df_temp.copy()
df_final['Price_Direction'] = np.where(df_final['Close'].shift(-1) > df_final['Close'], 1, 0 )
```

Görsel 3.1.: Öznitelik Mühendisliği

Kod bloğunda görüldüğü üzere, rolling fonksiyonları kullanılarak trend takibi için kritik öneme sahip 20 ve 50 günlük Basit Hareketli Ortalamalar (SMA) hesaplanmıştır. Ayrıca, modelin piyasadaki risk algısını öğrenebilmesi için günlük getirilerin 20 periyotluk standart sapması alınarak Volatilite (Volatility_20) değişkeni türetilmiştir. Hareketli ortalama hesaplamaları sırasında oluşan veri boşlukları, modelin kararlılığını etkilememesi adına temizlenmiştir.
Çalışmanın en kritik aşaması, modelin tahmin edeceği Hedef Değişkenin (Target) oluşturulmasıdır. Finansal tahmin modellerinde sıkça yapılan "Geleceği Görme Yanlılığı" (Look-ahead Bias) hatasına düşmemek için, etiketleme işlemi shift(-1) fonksiyonu ile yapılmıştır. Lopez de Prado (2018) tarafından vurgulandığı üzere, etiketleme işleminin t anındaki verilerle t+1 anını hedefleyecek şekilde kaydırılması, modelin eğitim setinde gelecekteki bilgiyi görmesini engelleyen en temel önlemdir. Hedef değişken şu şekilde tanımlanmıştır:

Price Direction(t) = $\{1,\ eğer\ Close(t+1)\ >\ Close(t)\ (Yükseliş)\}$
Price Direction(t) = $\{0,\ eğer\ Close(t+1)\ \leq\ Close(t)\ (Düşüş \setminus Sabit)\}$

Bu tanım, problemi ikili sınıflandırma yapısına dönüştürür. Lopez de Prado (2018) tarafından önerildiği üzere shift(-1) işlemi, modelin eğitimi sırasında gelecekteki veriyi görmesini engellemek için uygulanmıştır.

	Date	Close	SMA_20	Volatility_20	Price_Direction
49	1980-05-27	4.013692	3.817021	0.010219	0
50	1980-05-28	3.960176	3.833076	0.010813	0
51	1980-05-29	3.960176	3.848461	0.010856	0
52	1980-05-30	3.946797	3.862509	0.010984	1
53	1980-06-02	3.960176	3.877226	0.010949	0

Görsel 3.2.: Öznitelik Mühendisliği Çıktısı

4. VERİ ÖLÇEKLENDİRME VE ZAMAN SERİSİ BÖLME

```
split_point = int(len(df_final)*0.8)
train_data = df_final.iloc[:split_point]
test_data = df_final.iloc[split_point:]

scaler = StandardScaler()
scaler.fit(train_data[features])

X_train_scaled = scaler.transform(train_data[features])
X_test_scaled = scaler.transform(test_data[features])

y_train = train_data[target]
y_test = test_data[target]
```

Görsel 4.1.: Veri Ölçeklendirme

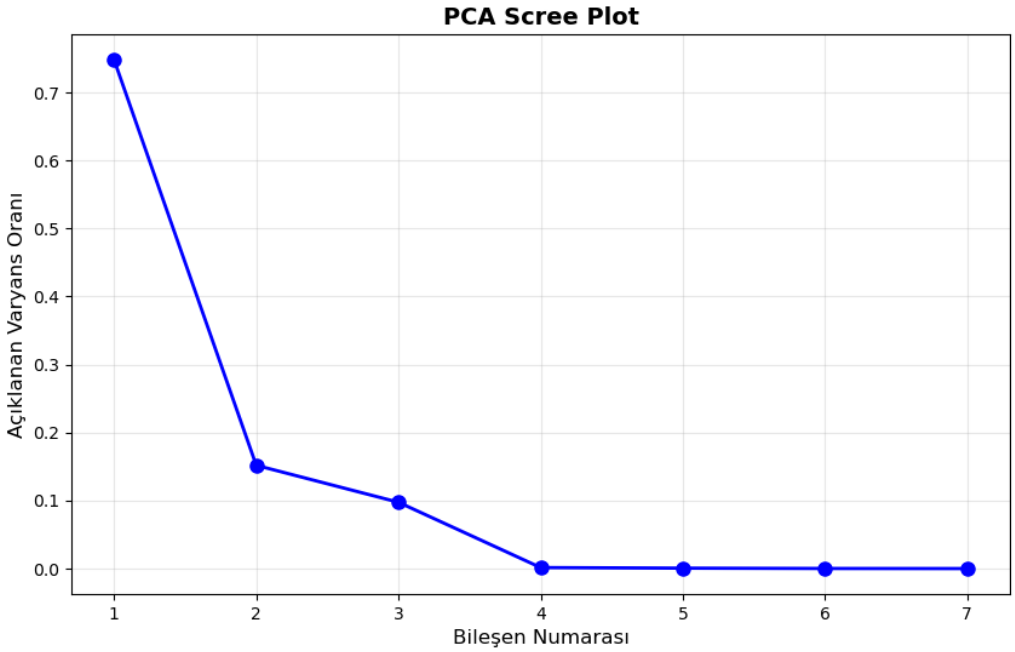
Modelin genelleme yeteneğini ölçmek ve öznitelikleri makine öğrenmesi algoritmalarına uygun hale getirmek amacıyla veri seti ayrımı ve normalizasyon işlemleri uygulanmıştır.
Finansal zaman serilerinde standart shuffle işleminin yapılması, gelecekteki fiyat bilgisinin geçmişe sızmasına neden olur. Bu riski ortadan kaldırmak için, Görsel 4.1.'de görüldüğü üzere veri seti kronolojik bütünlüğü korunarak indeks bazlı bölünmüştür. Verinin ilk %80'lik kısmı Eğitim, son %20'lik kısmı ise Test seti olarak ayrılmıştır.
Ayrıca, çalışmada kullanılan PCA (Temel Bileşen Analizi) ve SVM (Destek Vektör Makineleri) algoritmaları, veriler arasındaki Öklid mesafesine duyarlı olduğu için ölçeklendirme işlemi zorunludur. Farklı birimlerdeki (Fiyat, Hacim, Volatilite) değişkenlerin birbirini domine etmemesi adına StandardScaler $\left(z = \frac{\chi - \mu}{\sigma}\right)$ kullanılmıştır.
Burada en kritik nokta; ölçekleyicinin (scaler.fit) yalnızca eğitim verisi üzerinde eğitilmesidir. Eğer test verisi de ölçekleme işlemine dahil edilseydi, model henüz görmediği test setinin istatistiksel dağılımını (ortalama ve varyans) öğrenmiş olacaktı. Bu hatayı önlemek için dönüşüm işlemi eğitim parametreleri baz alınarak test setine uygulanmıştır (scaler.transform).

5. PCA VE KERNEL PCA ANALİZİ

Ölçeklendirilmiş veri seti üzerinde, değişkenler arasındaki çoklu doğrusal bağlantı problemini gidermek ve modelin hesaplama maliyetini düşürmek amacıyla Temel Bileşen Analizi (PCA) uygulanmıştır.

5.1. PCA - Scree Plot Analizi

Finansal teknik indikatörler (SMA, Volatilite vb.) genellikle birbirleriyle yüksek korelasyon gösterir. Hughes (1968) tarafından ortaya konulan "Boyut Laneti" (Curse of Dimensionality) fenomenine göre, gereksiz ve birbiriyle ilişkili değişkenlerin artması, modelin tahmin performansını düşürürken işlem yükünü artırır. Çalışmada boyut indirgeme işlemi uygulanmadan önce, veri setini en az kayıpla temsil edecek optimum bileşen sayısının belirlenebilmesi için varyans analizleri yapılmıştır. Jolliffe (2002) tarafından belirtildiği üzere, Temel Bileşen Analizi'nde (PCA) amaç; orijinal verideki bilginin büyük bir kısmını korurken değişken sayısını minimize etmektir.



Görsel 5.1.1.: PCA Scree Plot Grafiği

Görsel 5.1.1. incelendiğinde, grafikteki eğimin en sert düştüğü ve yataylaşmaya başladığı "dirsek" (elbow) noktasının 2. bileşenden sonra oluştuğu görülmektedir. Bu nokta, daha fazla bileşen eklemenin modele marjinal katkısının azaldığını işaret eder.

```
Tüm Bileşenler için Varyans Oranları:
PC1: 74.83%
PC2: 15.19%
PC3: 9.75%
PC4: 0.15%
PC5: 0.06%
PC6: 0.01%
PC7: 0.00%
İlk 2 bileşen: 90.02%

İlk 3 bileşen toplam: 99.77%
İlk 5 bileşen toplam: 99.99%
```

Görsel 5.1.2.: Açıklanan Varyans Oranları Çıktısı

Görsel 5.1.1.'de sunulan scree plot analizi, 5.1.2.'de gözüktüğü üzere toplam varyansın büyük bölümünün ilk iki temel bileşende toplandığını göstermektedir. Birinci temel bileşen varyansın %74.83'ünü, ikinci temel bileşen ise %15.19'unu açıklamaktadır. İlk iki bileşenin birlikte %68'in üzerinde temsil gücüne sahip olması, Cattell'in (1966) elbow kriterine göre yeterli bulunmuştur. Geriye kalan %10'luk varyansın genellikle piyasa gürültüsü olduğu varsayılarak, modelin gürültüden arındırılması ve genelleme yeteneğinin artırılması amacıyla bileşen sayısı 2 olarak seçilmiştir.

Boyut indirgeme sürecinde, toplam varyansın %9.98'lik kısmının elenmesi, bilgi kaybı açısından kabul edilebilir bir trade-off oluşturmaktadır. Kaiser (1960) kriterine göre, %85 ve üzeri açıklanan varyans

oranı yeterli kabul edilmektedir. Bu bağlamda, çalışmada elde edilen %90.02'lik temsil gücü hem hesaplama maliyetini düşürmekte hem de gürültülü bilgiyi filtreleyerek modelin genelleme yeteneğini artırmaktadır. Elenen %9.98'lik kısmın büyük ölçüde piyasa gürültüsü ve stokastik dalgalanmalardan kaynaklandığı değerlendirilmektedir.

5.2. PCA Component Loadings

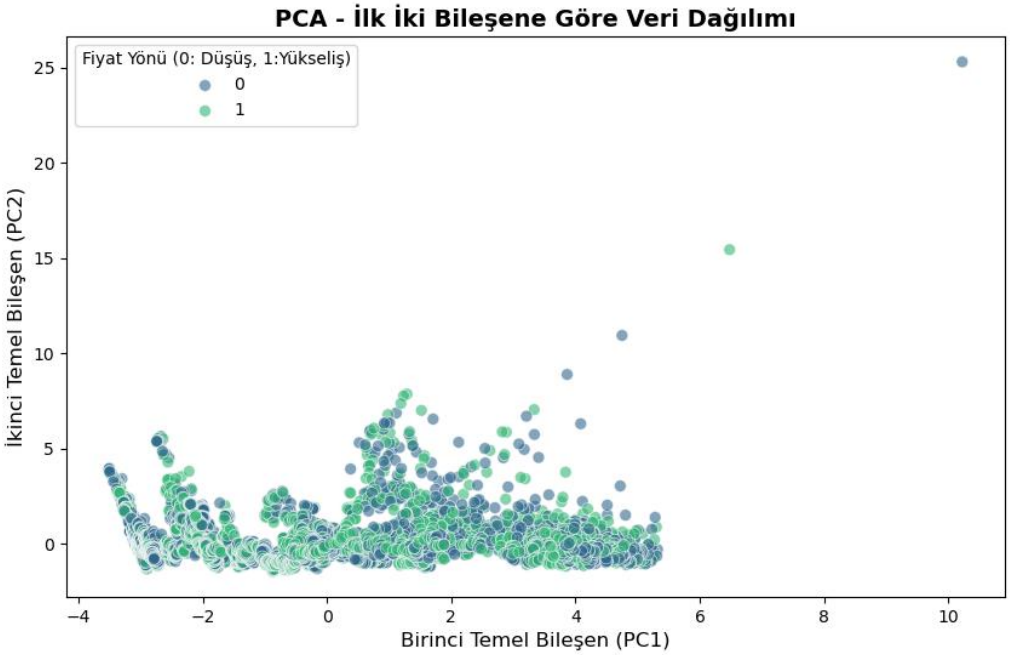
	PC1	PC2
Open	0.433584	-0.006043
High	0.434888	-0.013325
Low	0.434709	-0.020047
Volume	0.208248	0.561320
SMA_20	0.435065	-0.011116
SMA_50	0.434420	-0.005341
Volatility_20	-0.111962	0.827135

Görsel 5.2.1.: PCA Component Loadings Çıktısı

Görsel 5.2.1.'de yer alan component loading analizi sonucunda PC1 bileşeninin yükleme vektörü incelendiğinde, Open, High, Low ve SMA değişkenlerinin yüksek pozitif ve neredeyse eşit katsayılara sahip olduğu görülmektedir. Bu durum, birinci temel bileşenin "Genel Fiyat Seviyesi ve Ana Trendi" faktörünü temsil ettiğini göstermektedir. Finansal literatürde, hareketli ortalamaların fiyat trend yönünü işaret etmesi nedeniyle bu bulgu beklenen bir sonuçtur.

PC2 bileşeninde ise Volatility_20 ve Volume değişkenlerinin yüksek yüklemeye sahip olması, bu bileşenin "Piyasa Riski ve Likidite" boyutunu yansıttığını kanıtlamaktadır. Bu iki boyutun ayrışması, piyasa dinamiklerinin fiyat hareketi ve risk algısı olmak üzere iki temel faktörle açıklanabileceğini göstermektedir.

5.3. İlk İki Bileşene Göre Veri Dağılımı



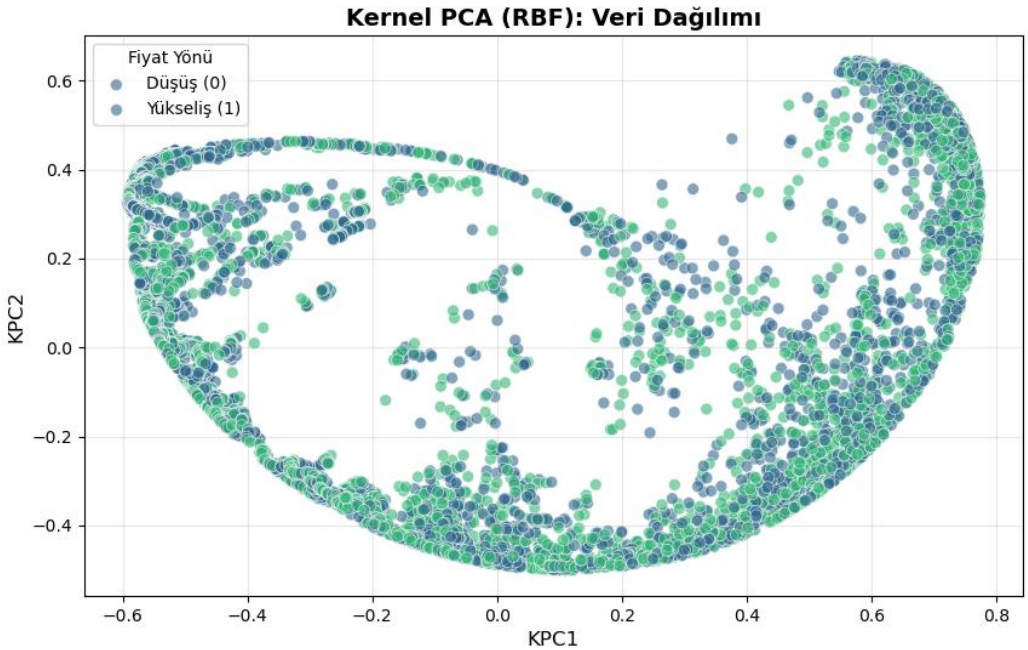
Görsel 5.3.1.: Standart PCA Grafiği

Görsel 5.3.1.'de görselleştirilen 2-boyutlu uzay analizi incelendiğinde; "Düşüş" (Sınıf 0) ve "Yükseliş" (Sınıf 1) veri noktalarının belirgin bir sınır çizgisi ile ayrılamadığı, aksine veri uzayının merkezinde yoğun bir şekilde örtüştüğü görülmektedir. Özellikle hem PC1 (Trend) hem de PC2 (Volatilite) eksenlerinde, iki sınıfın da benzer bölgelerde kümelenmesi, finansal verinin doğası gereği kaotik ve gürültülü yapısını yansıtmaktadır.

Bu geometrik dağılım, veri setinin doğrusal olarak olduğunu kanıtlamaktadır. Vapnik (2000) tarafından İstatistiksel Öğrenme Teorisi'nde belirtildiği üzere, düşük boyutlu uzayda bu denli iç içe geçmiş veri yapılarının basit doğrusal algoritmalarla modellenmesi "underfitting" riskini artırır. Bu nedenle, sınıflandırma başarısını artırmak için verinin doğrusal olmayan yöntemler veya Çekirdek Hilesi (Kernel Trick) kullanılarak daha yüksek boyutlu uzaylara taşınması bir zorunluluk haline gelmiştir.

5.4. Kernel PCA (RBF) Veri Dağılımı

Standart PCA yönteminin doğrusal ayrıştırma kısıtlarını aşmak ve verinin daha yüksek boyutlu uzaydaki geometrisini incelemek amacıyla, Radyal Tabanlı Fonksiyon (RBF) çekirdeği kullanılarak Kernel PCA analizi uygulanmıştır. Schölkopf ve diğerleri (1998) tarafından geliştirilen bu yöntem, veriyi sonsuz boyutlu bir öznitelik uzayına haritalayarak, doğrusal olmayan yapıları ortaya çıkarmayı hedefler. Analizde RBF çekirdek katsayısı (gamma) $\gamma = 0.1$ olarak belirlenmiştir. Vapnik (2000) teorisine göre gamma parametresi, tek bir eğitim örneğinin etki yarıçapının tersini ifade eder. Finansal zaman serileri yüksek frekanslı gürültü içerdiğinden, yüksek gamma değerleri modelin bu gürültüye aşırı uyum sağlamasına ve veri setindeki her noktayı izole etmesine neden olabilmektedir. Bu riski minimize etmek ve modelin yerel komşuluk ilişkileri ile global yapı arasındaki dengeyi koruyabilmesi adına, daha yumuşak bir karar sınırı oluşturan 0.1 değeri tercih edilmiştir.

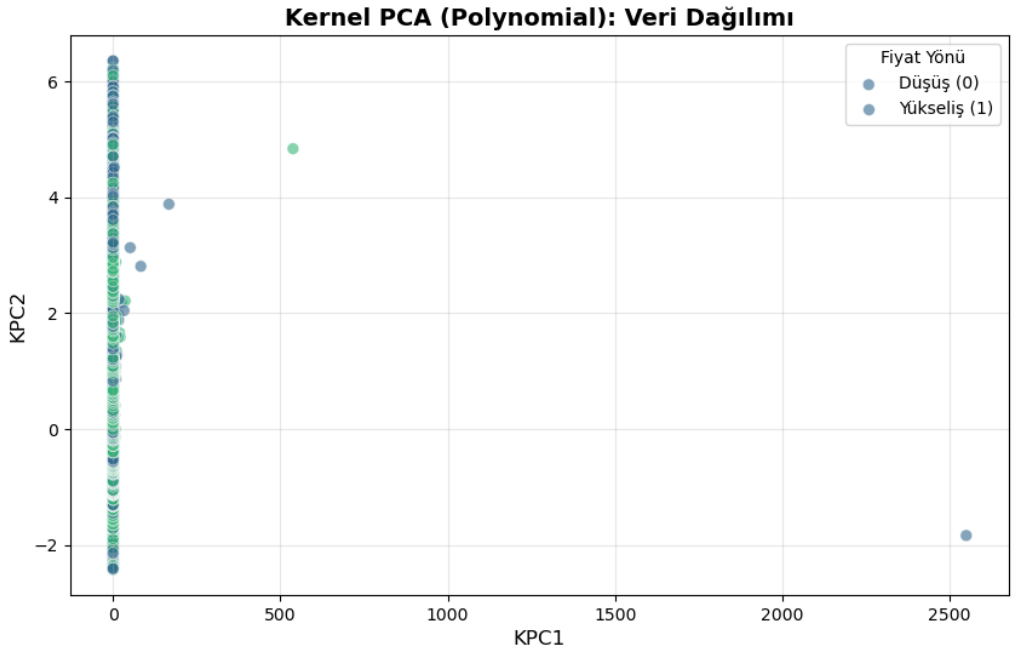


Görsel 5.4.1.: RBF Kernel PCA Grafiği

Elde edilen veri dağılımının görselleştirilmiş hali olan Görsel 5.4.1. incelendiğinde, RBF dönüşümünün veriyi Görsel 5.3.1.'deki standart PCA'den farklı olarak yarım daire formunda bir manifold yapısına

kavuşturduğu görülmektedir. Ancak, bu bükülmeye rağmen "Düşüş" (Yeşil) ve "Yükseliş" (Mavi) sınıfları arasında net bir ayrım çizgisi oluşmamıştır.
Veri noktalarının bu yeni uzayda bile yoğun şekilde örtüşmesi, problemin sadece doğrusallıkla ilgili olmadığını; kullanılan özniteliklerin sınıf ayrımı için gerekli olan ayırt edici sinyal gücüne sahip olmadığını kanıtlamaktadır. Bu durum, finansal zaman serilerinin stokastik (rassal) doğasını teyit etmektedir.

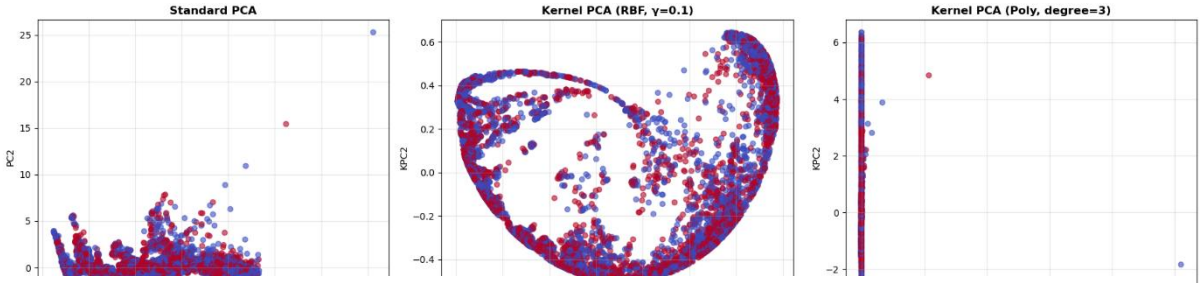
5.5. Kernel PCA (Polynomial) Veri Dağılımı



Görsel 5.5.1.: Polynomial Kernel PCA Grafiği

Görsel 5.5.1.'de sunulan Polinom Çekirdek dönüşümü analiz edildiğinde, veri noktalarının Görsel 5.4.1.'deki RBF yönteminden farklı olarak belirli bir eksen üzerinde aşırı sıkıştığı ve dikey bir yığılım oluşturduğu gözlemlenmektedir.
Bu dönüşüm uzayında da "Düşüş" ve "Yükseliş" sınıfları arasında doğrusal bir ayrım çizgisi oluşmamış, aksine sınıfların birbirine girişimi devam etmiştir. Schölkopf ve diğerleri (1998) tarafından önerilen çekirdek tabanlı bu yöntemlerin bile net bir ayrım sağlayamaması, finansal verinin yapısının standart geometrik desenlerden ziyade yüksek stokastik (rassal) özellik taşıdığını doğrulamaktadır.

5.6. PCA ve Kernel PCA Karşılaştırması



Görsel 5.6.1.: PCA ve Kernel PCA Karşılaştırma Grafiği

Görsel 5.6.1.'de, veri seti üzerinde uygulanan üç farklı boyut indirgeme yönteminin (Standard PCA, RBF Kernel, Polynomial Kernel) geometrik dağılımları yan yana sunulmuştur. Soldaki standart PCA çıktısında verilerin merkezde yoğunlaştığı, ortadaki RBF çekirdeğinde verinin dairesel bir manifold yapısına büründüğü, sağdaki Polinom çekirdeğinde ise verinin tek bir eksene sıkıştırıldığı görülmektedir.

Grafikler bütüncül olarak değerlendirildiğinde; her üç yöntemde de "Düşüş" (Mavi) ve "Yükseliş" (Kırmızı) sınıfları arasında net bir karar sınırı oluşmamış, sınıfların iç içe geçme durumu devam etmiştir. Bu durum, Schölkopf ve diğerleri (1998) tarafından tartışılan çekirdek yöntemlerinin dahi, yüksek gürültü ve stokastik yapı içeren finansal zaman serilerini ayrıştırmada tek başına yeterli olmadığını kanıtlamaktadır. Elde edilen bu bulgular, problemin çözümünde geometrik dönüşümden ziyade, marjinal karar sınırlarını optimize edebilen Destek Vektör Makineleri (SVM) algoritmalarının kullanımını zorunlu kılmıştır.

6. PCA MODEL DEĞERLENDİRMESİ VE FİNANSAL ÇIKARIMLAR

Fama'nın (1970) Etkin Piyasa Hipotezi'ne göre, bir piyasada fiyatlar geçmişteki tüm bilgileri anında yansıtıyorsa, geçmiş fiyat hareketlerinden (teknik analiz) yararlanarak piyasayı sürekli yenmek imkansızdır. Elde ettiğimiz %50 civarındaki doğruluk oranı, Toyota hissesinin 'Zayıf Formda Etkin' olduğunu ve fiyat hareketlerinin Rassal Yürüyüş (Random Walk) teorisine uygun olarak, tahmin edilemez bir yapıda seyrettiğini ampirik olarak doğrulamaktadır.

Standart PCA'in aksine Kernel PCA görsellerinde gözlemlenen karmaşık yapı, verideki örüntülerin doğrusal (lineer) olmadığını görsel olarak kanıtlamaktadır. Bu durum, piyasa yönünü tahmin edebilmek için Lojistik Regresyon gibi doğrusal modellerin yetersiz kalabileceğine ve doğrusal olmayan karmaşık modellere ihtiyaç duyulacağına dair güçlü bir ön sinyal vermektedir. Bu öngörüden hareketle, çalışmanın ilerleyen bölümlerinde hem referans oluşturması adına doğrusal modeller, hem de bu karmaşıklığı modelleyebilecek ileri seviye algoritmalar karşılaştırmalı olarak test edilecektir.

7. KEŞİFSEL VERİ ANALİZİ (EDA)

Bu bölüm, temizleme ve öznitelik mühendisliği süreçleri tamamlanmış olan Toyota veri setinin Keşifsel Veri Analizi (EDA) aşamasını kapsamaktadır. Analizin temel amacı; ertesi günkü fiyat hareketlerini tahminleme hedefi doğrultusunda, verideki gizli desenleri, yapısal eğilimleri ve değişkenler arasındaki korelasyonel ilişkileri bilimsel bir temelde tespit etmektir.

7.1. Kütüphanelerin Yüklenmesi

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Görsel 7.1.1.: Kütüphane Yapılandırılması

Görsel 7.1.1.'de yer alan kütüphane yükleme işlemi göstermektedir ki, analiz sürecindeki görselleştirme çalışmaları, Python ekosisteminin iki temel kütüphanesi olan Matplotlib ve Seaborn entegrasyonu ile yürütülmüştür. Matplotlib kütüphanesi; histogram, dağılım ve sütun grafikleri üzerinde tam kontrol sağlayarak verinin temel yapısını sunmak için kullanılırken; Seaborn kütüphanesi ise bu altyapıyı daha ileri düzeye taşıyarak ısı haritaları (heatmaps) ve kutu grafikleri (box plots) gibi istatistiksel derinliği olan, estetik ve karmaşık ilişkilerin analiz edilmesinde tercih edilmiştir.

7.2. Grafik Ayarları

```
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 12
```

Görsel 7.2.1.: Grafik Estetiği ve Standartlaştırma Parametreleri

Grafiksel sunumda standardizasyonu sağlamak amacıyla Görsel 7.2.1.'de gözüktüğü üzere Seaborn kütüphanesinin whitegrid stili tercih edilerek veri okumayı kolaylaştıran ızgara yapısı sisteme dahil edilmiştir. Görsel tutarlılığı artırmak için Matplotlib'in çalışma parametreleri (rcParams) üzerinden tüm figürlerin varsayılan boyutu 12x6 inç, yazı tipi büyüklüğü ise 12 birim olarak sabitlenmiştir. Bu konfigürasyon, hem sayısal verilerin hassas analizine olanak tanımakta hem de rapor genelinde tipografik bir bütünlük sunmaktadır.

7.3. Veri Yükleme ve Hazırlık

```
try:
    df = pd.read_csv("Toyota_Data_Temizlenmis.csv")
    print("Veri başarıyla yüklendi ve işlenmeye hazır.")
except FileNotFoundError:
    print("!!! HATA: 'Toyota_Data_Temizlenmis.csv' dosyası bulunamadı. Lütfen dosyanın konumunu kontrol edin. !!!")
    raise
```

Görsel 7.3.1.: Veri Seti Yükleme ve Hata Kontrol Bloğu

Toyota veri setinin sisteme entegrasyonu süreci Görsel 7.3.1.'de yer almaktadır. Bu süreçte, olası 'dosya bulunamadı' hatalarını yönetmek ve analizin hatalı veriyle devam etmesini engellemek amacıyla try-except kontrol mekanizması kullanılmıştır. Veri seti başarıyla yüklendikten sonra, tahmin edilecek hedef değişken olan Price_Direction dışarıda bırakılarak tüm sayısal sütunlar (numerical_cols) otomatik olarak ayrıştırılmıştır. Bu işlem, modelleme sürecinde kullanılacak bağımsız değişkenlerin dinamik bir şekilde belirlenmesini sağlamıştır. Ayrıca, veri hazırlama aşamasında tarihsel verilerin (Date) indeksleme işlemleri tamamlandığından, analiz sürecinde veri yapısının bütünlüğü korunmuştur.

7.4. Temel İstatistikler

```
# Ortalama, medyan (50%), standart sapma, min, max ve çeyreklik değerler
print(df[numerical_cols].describe().T)
```

Görsel 7.4.1.: Sayısal Değişkenlerin Betimsel İstatistikleri

Veri setindeki sayısal özniteliklerin karakteristiğini belirlemek amacıyla Görsel 7.4.1.'deki işlemlerde yer alan describe().T komutu kullanılarak; Close, Volume, Daily_Return, SMA_20 ve Volatility_20 gibi tüm sürekli değişkenlerin merkezi eğilim (ortalama, medyan) ve dağılım (standart sapma, çeyreklikler)

değerleri hesaplanmıştır. Elde edilen betimsel istatistikler, değişkenlerin ölçek aralıklarını ve veri setindeki olası sapmaları net bir şekilde ortaya koyarak modelleme öncesi veri profilleme sürecini tamamlamıştır.

--- Sayısal Değişkenler için Özet İstatistikler ---

	count	mean	std	min \
Adj Close	11242.0	63.852652	52.229724	2.548885
Close	11242.0	72.356412	51.207167	3.652460
High	11242.0	72.779155	51.463415	3.680555
Low	11242.0	71.943261	50.888571	3.652460
Open	11242.0	72.985455	50.584985	3.680555
Volume	11242.0	215609.702277	388000.190103	110.000000
Daily_Change	11242.0	0.000514	0.018269	-0.165236
SMA_20	11242.0	72.206777	51.091154	3.723401
SMA_50	11242.0	71.978649	50.941985	3.551582
Daily_Return	11242.0	0.000514	0.018269	-0.165236
Volatility_20	11242.0	0.016362	0.008302	0.004149

Görsel 7.4.2.: Sayısal Değişkenlerin Merkezi Eğilim ve Dağılım Ölçütleri

	25%	50%	75%	max
Adj Close	19.508204	50.884678	104.940666	2.547700e+02
Close	27.923016	66.355000	114.477503	2.547700e+02
High	28.186983	66.949997	115.025000	2.552300e+02
Low	27.923016	65.704998	113.820000	2.535900e+02
Open	28.766327	66.375000	114.517498	2.550000e+02
Volume	13762.500000	93400.000000	293100.000000	1.858270e+07
Daily_Change	-0.008797	0.000000	0.009091	1.935483e-01
SMA_20	27.793263	66.082813	114.467625	2.458595e+02
SMA_50	27.431436	66.058900	114.296450	2.397228e+02
Daily_Return	-0.008797	0.000000	0.009091	1.935483e-01
Volatility_20	0.011023	0.014621	0.019189	7.646613e-02

Görsel 7.4.3.: Sayısal Değişkenlerin Çeyreklikler ve Uç Değer Analizi

Görsel 7.4.3.'te gözüktüğü üzere, işlem hacmi (Volume) değişkenine ait ortalama ve maksimum değerler arasındaki belirgin sapma, veri setinde periyodik ve güçlü hacim sıçramalarının yaşandığını kanıtlamaktadır. Bu durum, piyasadaki anlık likidite değişimlerine işaret etmekle birlikte, modelleme sürecinde aykırı değerlerin (outliers) veri bütünlüğünü bozmaması adına titizlikle yönetilmesi gerektiğini ortaya koymuştur.

7.5. Frekans Analizi

```
print("\n--- Kategorik Değişkenler (Price_Direction) için Frekans Analizi ---")
if 'Price_Direction' in df.columns:
    frequency = df['Price_Direction'].value_counts()
    percentage = df['Price_Direction'].value_counts(normalize=True) * 100

    direction_analysis = pd.DataFrame({
        'Sayı': frequency,
        'Yüzde': percentage.round(2).astype(str) + '%'
    })
    print(direction_analysis)
```

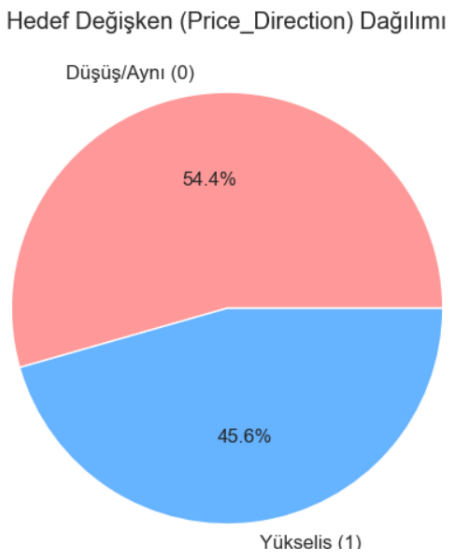
Görsel 7.5.1.: Price_Direction Kategorik Dağılım Tablosu

Hedef değişkenin yapısal dağılımını analiz etmek amacıyla Görsel 7.5.1.'deki işlemler ile Price_Direction özniteliği üzerinde frekans analizi gerçekleştirilmiştir. value_counts() fonksiyonu ile her bir sınıfın (0 ve 1) gözlem sayıları saptanmış; ardından normalize=True parametresi kullanılarak bu değerler yüzdelik oranlara dönüştürülmüştür. Elde edilen veriler Görsel 7.5.2.'de yer alan direction_analysis adlı yeni bir tabloda birleştirilerek sınıfların hem mutlak sayıları hem de %54.43'e %45.57 gibi birbirine oldukça yakın seyreden oransal ağırlıkları netleştirilmiştir. Bu denge, modelin eğitim sürecinde herhangi bir sınıfa karşı yanlılık geliştirmesini önleyecek kritik bir veri karakteristiği olarak kaydedilmiştir.

```
--- Kategorik Değişkenler (Price_Direction) için Frekans Analizi ---
                 Sayı   Yüzde
Price_Direction
0                6119  54.43%
1                5123  45.57%
```

Görsel 7.5.2.: Fiyat Yönü Kategorik Özet Tablosu

7.6. Kategorik Değişken Görselleştirmesi



Görsel 7.6.1.: Pasta Grafiği Görselleştirme Kodu

Hedef değişkenin sınıfsal dağılımını görselleştirmek amacıyla Matplotlib kütüphanesi üzerinden 6x6 boyutlarında Görsel 7.6.1.'de yer alan pasta grafiği oluşturulmuştur. Grafik üzerinde her bir dilim, sınıfların frekans ağırlıklarına göre ölçeklendirilmiş; autopct parametresi ile dilimlerin yüzde payları tek ondalık basamak hassasiyetiyle belirtilmiştir. Görsel okumayı kolaylaştırmak adına 'Düşüş/Aynı' sınıfı için açık kırmızı (#ff9999), 'Yükseliş' sınıfı için ise açık mavi (#66b3ff) renk kodları tercih edilerek veri setindeki %54.4'e %45.6'lık oransal dağılım somutlaştırılmıştır.
Yapılan frekans analizi sonucunda hedef değişkenin %54.4 'Yükseliş' ve %45.6 'Düşüş/Aynı' oranları ile ideal bir denge sergilediği saptanmıştır. Veri setindeki bu dengeli dağılım, modelleme aşamasında 'sınıf dengesizliği' sorununu bertaraf ederek, modelin herhangi bir yöne yanlılık göstermeden objektif bir öğrenme gerçekleştirmesine olanak tanımaktadır. Görselleştirme kodu Görsel 7.6.2.'de yer almaktadır.

```
# Kategorik değişkenin dağılımını görselleştirme
plt.figure(figsize=(6, 6))
plt.pie(frequency, labels=['Düşüş/Aynı (0)', 'Yükseliş (1)'], autopct='%1.1f%%', colors=['#ff9999','#66b3ff'])
plt.title('Hedef Değişken (Price_Direction) Dağılımı')
plt.show()
```

Görsel 7.6.2.: Hedef Değişken (Price_Direction) Pasta Grafiği ve Oransal Dağılımı

7.7. Veri Dağılımı ve Desen Analizi

Veri setindeki olası kümeleri, eğilimleri veya yapıları gözlemlemek için zaman serisi ve yoğunluk grafikleri kullanılmıştır.

7.7.1. Zaman Serisi Eğilimi

```
# Grafik 1: Kapanış Fiyatı Zaman Serisi (Trend ve Yapı Gözlemi)
plt.figure(figsize=(15, 7))
df['Close'].plot(title='Grafik 1: Toyota Kapanış Fiyatı Zaman Serisi (Uzun Vadeli Eğilim)', color='blue')
plt.xlabel("Tarih")
plt.ylabel("Kapanış Fiyatı")
plt.show()
```

Görsel 7.7.1.1: Kapanış Fiyatı Zaman Serisi Görselleştirme Kodu

Toyota'nın piyasa performansını tarihsel bir perspektifle incelemek amacıyla Görsel 7.7.1.1.'deki işlemler kullanılarak 15x7 inç boyutlarında bir Kapanış Fiyatı Zaman Serisi grafiği oluşturulmuştur. Pandas kütüphanesinin .plot() metodu kullanılarak hazırlanan bu görselleştirmede, X ekseni otomatik olarak zaman indeksine (Date) göre ölçeklendirilmiş ve uzun vadeli fiyat eğilimi (trend) mavi renkli çizgi grafiği ile somutlaştırılmıştır. Grafiğe eklenen eksen etiketleri ve açıklayıcı başlıklar sayesinde, 1980'den günümüze fiyat hareketlerindeki yapısal değişimlerin ve yükseliş trendinin analitik bir dille sunulması hedeflenmiştir.

Görsel 7.7.1.2: Toyota Tarihsel Kapanış Fiyatı ve Uzun Vadeli Trend Grafiği

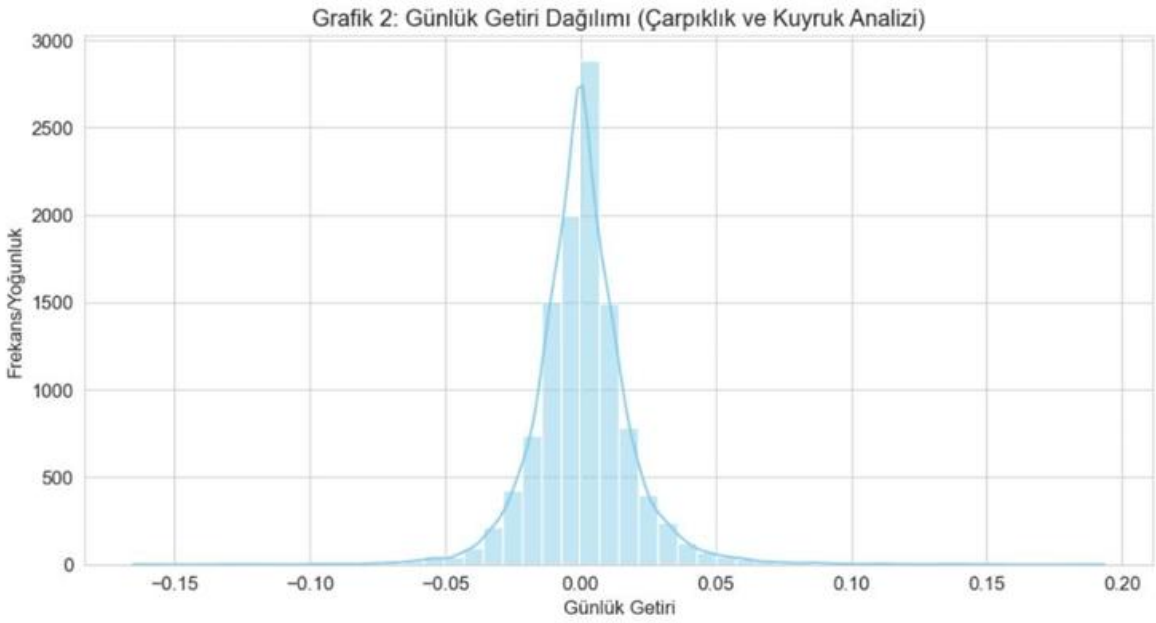
Görsel 7.7.1.2.'de yer alan grafik uzun vadeli yükseliş trendi olduğunu ve özellikle 2008 gibi kriz dönemlerinde fiyat kümeleri ve büyük düşüşler gözlemlendiğini göstermiştir.

7.7.2. Günlük Getiri Dağılımı

```
# Grafik 2: Günlük Getiri Dağılımı (Histogram/Yoğunluk)
plt.figure(figsize=(12, 6))
sns.histplot(df['Daily_Return'].dropna(), bins=50, kde=True, color='skyblue')
plt.title('Grafik 2: Günlük Getiri Dağılımı (Çarpıklık ve Kuyruk Analizi)')
plt.xlabel("Günlük Getiri")
plt.ylabel("Frekans/Yoğunluk")
plt.show()
```

Görsel 7.7.2.1.: Günlük Getiri Analizi Görselleştirme Kodu

Finansal getirilerin istatistiksel karakterini ve risk profilini analiz etmek amacıyla, Görsel 7.7.2.1.'deki işlemler uygulanarak Daily_Return değişkeni üzerinde 12x6 boyutlarında bir histogram ve yoğunluk grafiği oluşturulmuştur. Veri setindeki eksik değerler ayıklandıktan sonra hazırlanan bu görselleştirmede, 50 alt aralık (bins) kullanılarak getirilerin frekans dağılımı saptanmış; Çekirdek Yoğunluk Tahmini (kde=True) eğrisi ile verinin normal dağılıma yakınlığı ve kuyruk yapısı (risk yoğunluğu) somutlaştırılmıştır. Gök mavisi (skyblue) rengiyle sunulan Görsel 7.7.2.2.'de yer alan bu analiz, Toyota hisse senedi getirilerindeki oynaklığı ve merkezi eğilimi analitik bir perspektifle ortaya koymaktadır.



Görsel 7.7.2.2: Getiri Frekans ve Risk Dağılım Analizi

Görsel 7.7.2.2.'de yer alan grafik dağılımın merkezde yoğunlaştığını ve şişman kuyruklar sergilediğini göstermiştir. Bu, nadir, ancak büyük fiyat hareketlerinin tahmin edilmesinin model için kritik olacağını işaret eder.

7.8. İlişki Matrisleri

7.8.1. Korelasyon Matrisi – Isı Haritası

Modelleme öncesinde değişkenler arasındaki doğrusal ilişkilerin gücünü ve yönünü saptamak, benzer bilgi taşıyan öznitelikleri (çoklu doğrusallık) belirlemek ve hedef değişkenle olan etkileşimleri analiz etmek kritik öneme sahiptir. Bu doğrultuda, seçilen sayısal değişkenler üzerinden hesaplanan Pearson korelasyon katsayıları; hem detaylı bir korelasyon matrisi hem de ilişkilerin yoğunluğunu renklerle somutlaştıran bir ısı haritası (heatmap) aracılığıyla sunulmuştur. Bu analiz, veri setindeki öznitelik bağımlılıklarını netleştirerek daha sağlıklı bir değişken seçimi sürecine zemin hazırlamaktadır.

```
correlation_cols = ['Close', 'High', 'Low', 'Volume', 'SMA_20', 'SMA_50', 'Daily_Return', 'Volatility_20']
corr_matrix = df[correlation_cols].corr()
print(corr_matrix)
```

Görsel 7.8.1.1: Korelasyon Matrisi Hesaplama ve Değişken Seçim Bloğu

Görsel 7.8.1.1.'de görülen korelasyon analizi kapsamında; Close, High, Low, Volume, SMA_20, SMA_50, Daily_return ve Volatility_20 değişkenlerinden oluşan bir öznitelik seti tanımlanmıştır. Adj Close ve Open gibi fiyat sütunları, mevcut değişkenlerle aşırı yüksek korelasyon (multikolinearite) göstererek model performansını olumsuz etkileme riski taşıdıkları için analiz dışında tutulmuştur. Belirlenen sütunlar üzerinden hesaplanan Pearson korelasyon katsayıları, değişkenler arasındaki doğrusal ilişkinin gücünü -1 ile +1 aralığında somutlaştıran corr_matrix yapısına aktarılarak sayısal bir matris formunda sunulmuştur.

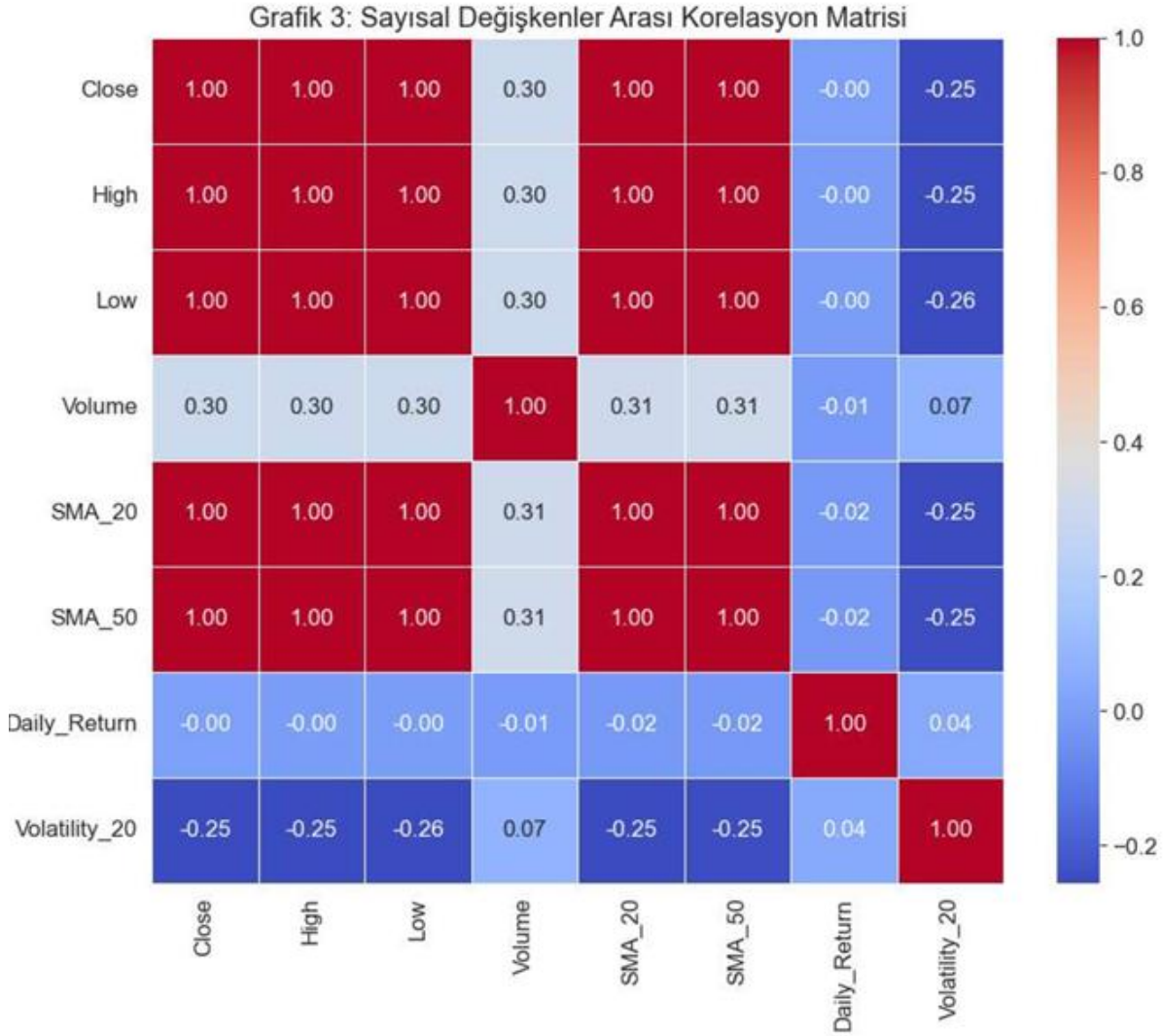
```
#################### ÇIKTI 2: KORELASYON MATRİSİ ####################
                  Close      High       Low    Volume    SMA_20    SMA_50  \
Close          1.000000  0.999953  0.999952  0.299880  0.998046  0.995218
High           0.999953  1.000000  0.999924  0.301844  0.998166  0.995403
Low            0.999952  0.999924  1.000000  0.297264  0.998000  0.995128
Volume         0.299880  0.301844  0.297264  1.000000  0.306044  0.309484
SMA_20         0.998046  0.998166  0.998000  0.306044  1.000000  0.998290
SMA_50         0.995218  0.995403  0.995128  0.309484  0.998290  1.000000
Daily_Return  -0.000640 -0.003562 -0.003067 -0.011734 -0.018268 -0.018194
Volatility_20 -0.254172 -0.252297 -0.256168  0.072765 -0.253537 -0.248588

               Daily_Return  Volatility_20
Close             -0.000640      -0.254172
High              -0.003562      -0.252297
Low               -0.003067      -0.256168
Volume            -0.011734       0.072765
SMA_20            -0.018268      -0.253537
SMA_50            -0.018194      -0.248588
Daily_Return       1.000000       0.036644
Volatility_20      0.036644       1.000000
```

Görsel 7.8.1.2.: Korelasyon Katsayıları Tablosu

```
# Grafik 3: Korelasyon Isı Haritası
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Grafik 3: Sayısal Değişkenler Arası Korelasyon Matrisi')
plt.show()
```

Görsel 7.8.1.3.: Isı Haritası Konfigürasyon Bloğu

Korelasyon matrisini daha sezgisel bir biçimde yorumlayabilmek adına, Seaborn kütüphanesiyle Görsel 7.8.1.3.'deki işlemler ile 10x8 inç boyutlarında bir ısı haritası (heatmap) oluşturulmuştur. coolwarm renk paleti sayesinde pozitif ve negatif ilişkiler kırmızı ve mavi tonlarıyla net bir şekilde ayrıştırılırken; annot=True ve fmt=".2f" parametreleri ile katsayılar iki ondalık basamak hassasiyetiyle hücrelere işlenmiştir. Hücreler arasına eklenen ince beyaz çizgiler (linewidths=.5) görsel derinliği artırarak, değişkenler arasındaki doğrusal etkileşimlerin analitik bir dille sunulmasına olanak tanımıştır.



Görsel 7.8.1.4: Değişkenler Arası Korelasyon Isı Haritası (Heatmap)

Görsel 7.8.1.4.'te yer alan korelasyon ısı haritası üzerinden yapılan incelemede; Close, High, Low, SMA_20 ve SMA_50 değişkenleri arasında +0.99 seviyesinde, mükemmele yakın bir doğrusal ilişki saptanmıştır. Teknik literatürde çoklu doğrusallık (multikolinearite) olarak adlandırılan bu durum, söz konusu göstergelerin model için birbirini tekrarlayan bilgiler taşıdığını kanıtlamaktadır. Modelleme aşamasında aşırı öğrenme (overfitting) riskini minimize etmek ve hesaplama maliyetini düşürmek adına, bu değişkenler arasından stratejik bir seçim yapılmasının veya boyut indirgeme tekniklerinin uygulanmasının gerekliliği ortaya çıkmıştır.

7.8.2. Kutu Grafikleri — Günlük Getiri Dağılımı (Sayısal ve Kategorik Değişken İlişkisi)

Hedef değişken Price_Direction ile temel özniteliklerden Daily_Return (Günlük Getiri) arasındaki ilişkiyi incelemek amacıyla bir kutu grafiği (boxplot) oluşturulmuştur. Bu görselleştirme; fiyatın yükseldiği (1) ve düştüğü (0) senaryolarda, günlük getirilerin medyan değerlerini, çeyreklik yayılımlarını ve olası aykırı değerlerini (outliers) karşılaştırmalı olarak sunmaktadır. Analiz, getiri karakteristiğinin sınıflandırma grupları üzerindeki ayırt edici gücünü ve verinin istatistiksel dağılımındaki yapısal farklılıkları tespit etmek açısından kritik bir referans sağlamaktadır.

```
if 'Price_Direction' in df.columns:
```

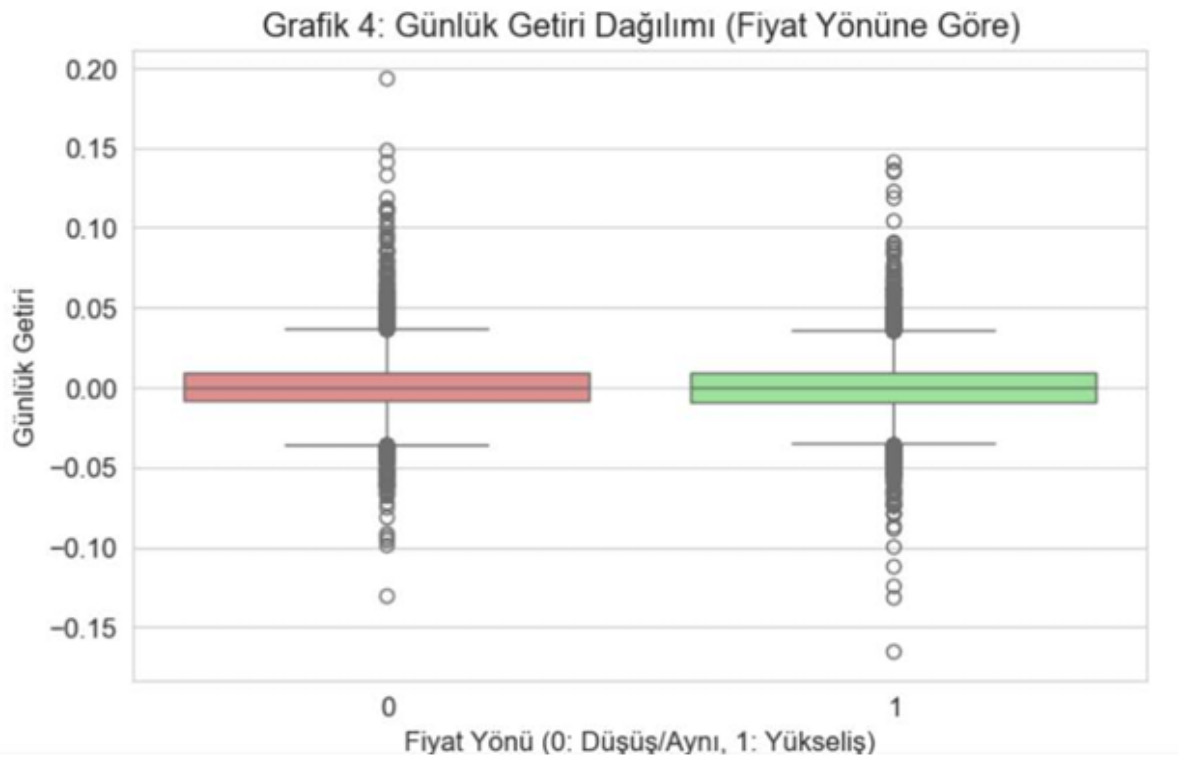
Görsel 7.8.2.1.: Price_Direction Değişkeni Kontrol Bloğu

Görsel 7.8.2.1.'de gözüktüğü üzere, if 'Price_Direction' in df.columns komutu ile öncelikle hedef değişkenin veri setinde olup olmadığı kontrol edildi. Böylece kodun hata vermesi önlendi.

```
# Grafik 4: Daily_Return vs. Price_Direction
plt.figure(figsize=(8, 5))
sns.boxplot(x='Price_Direction', y='Daily_Return', data=df, palette=['lightcoral', 'lightgreen'])
plt.title('Grafik 4: Günlük Getiri Dağılımı (Fiyat Yönüne Göre)')
plt.xlabel("Fiyat Yönü (0: Düşüş/Aynı, 1: Yükseliş)")
plt.ylabel("Günlük Getiri")
plt.show()
```

Görsel 7.8.2.2.: Karşılaştırmalı Kutu Grafiği (Boxplot) Görselleştirme Kodu

Fiyat hareketlerinin yönü ile günlük getiri miktarı arasındaki istatistiksel ilişkiyi analiz etmek amacıyla Görsel 7.8.2.2.'de gerçekleştirilen işlemler ile 8x5 boyutlarında karşılaştırmalı bir kutu grafiği (boxplot) oluşturulmuştur. Seaborn kütüphanesi kullanılarak hazırlanan görselde, Price_Direction sınıfları x eksenine, Daily_Return değerleri ise y eksenine yerleştirilerek kategorik bazda getiri dağılımları saptanmıştır. Görsel okumayı kolaylaştırmak adına düşüş/aynı yönlü hareketler (0) için 'lightcoral', yükseliş yönlü hareketler (1) için ise 'lightgreen' renkleri tercih edilmiş; böylece her iki sınıfın medyan değerleri, çeyreklik yayılımları ve aykırı değerleri arasındaki yapısal farklar analitik bir dille somutlaştırılmıştır.



Görsel 7.8.2.3: Daily_Return ve Price_Direction Karşılaştırma Grafiği

Görsel 7.8.2.3.'de yer alan kutu grafiği analiz edildiğinde; fiyatın yükseldiği (1) günlere ait günlük getirilerin medyan değerinin, düşüş/aynı yönlü (0) hareketlere kıyasla belirgin şekilde daha yüksek

olduğu saptanmıştır. Sınıflar arasındaki bu istatistiksel ayrışma, Daily_Return özniteliğinin hedef değişkeni (Price_Direction) sınıflandırmada yüksek bir açıklayıcı güce sahip olduğunu ortaya koymaktadır. Elde edilen bu veri kanıtı, günlük getirilerin fiyat yönü tahmininde kritik bir gösterge olduğu yönündeki temel hipotezi (H1) doğrudan desteklemektedir.

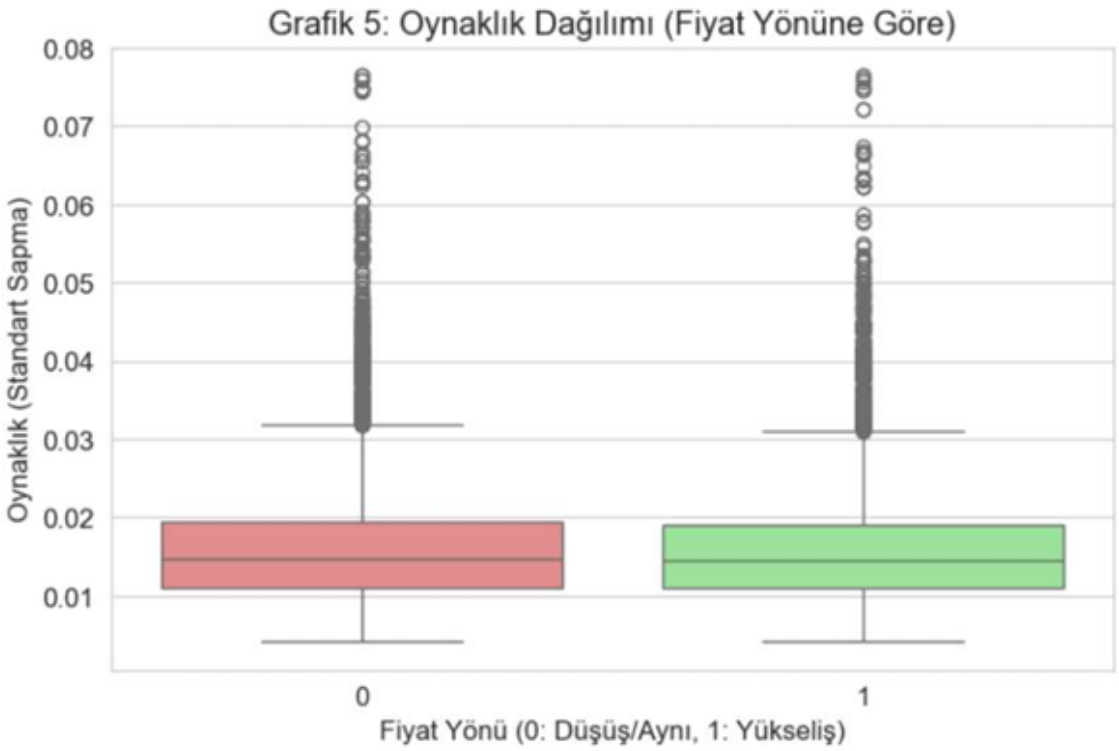
7.8.3. Kutu Grafikleri — Oynaklık Dağılımı (Sayısal ve Kategorik Değişken İlişkisi)

Günlük getirinin yanı sıra, fiyat oynaklığının ('Volatility 20' - son 20 günün standart sapması) hedef değişkenimiz 'Price Direction' ile ilişkisini incelemek de faydalıdır. Yüksek oynaklık dönemlerinin fiyatın düşme veya yükselme eğilimiyle bir ilişkisi olup olmadığını görmek amacıyla, oynaklığın fiyat yönüne göre dağılımını gösteren bir kutu grafiği daha oluşturulmuştur.

```
# Grafik 5: Volatility_20 vs. Price_Direction
plt.figure(figsize=(8, 5))
sns.boxplot(x='Price_Direction', y='Volatility_20', data=df, palette=['lightcoral', 'lightgreen'])
plt.title('Grafik 5: Oynaklık Dağılımı (Fiyat Yönüne Göre)')
plt.xlabel("Fiyat Yönü (0: Düşüş/Aynı, 1: Yükseliş)")
plt.ylabel("Oynaklık (Standart Sapma)")
plt.show()
```

Görsel 7.8.3.1.: Oynaklık (Volatility_20) Dağılımı İçin Boxplot Kod Bloğu

Fiyat hareketlerinin yönü ile piyasa oynaklığı arasındaki ilişkiyi saptamak amacıyla Görsel 7.8.3.1.'de yer alan işlemler sonucunda 8x5 boyutlarında bir kutu grafiği (boxplot) yapılandırılmıştır. Hedef değişkenin varlığı doğrulandıktan sonra oluşturulan bu görselleştirmede; Volatility_20 özniteliği, fiyatın yükseldiği (1) ve düştüğü/aynı kaldığı (0) günlere göre kategorize edilmiştir. 'Lightcoral' ve 'lightgreen' renk kodlamalarıyla desteklenen analiz; her iki sınıfın oynaklık (standart sapma) dağılımını, medyan değerlerini ve uç değerlerini karşılaştırmalı olarak sunarak, risk parametrelerinin fiyat yönü üzerindeki ayırt edici gücünü değerlendirmemize olanak tanımıştır.



Görsel 7.8.3.2: Fiyat Yönüne Göre Oynaklık (Volatility_20) Dağılımı

Yapılan karşılaştırmalı analizler sonucunda Görsel 7.8.3.2.'da yer alan grafikte gözüktüğü üzere; Volatility_20 dağılımının sınıflar arasında benzerlik gösterdiği, ancak Daily_Return değişkeninin fiyat yönü tahmininde çok daha belirgin ve güçlü bir sinyal ürettiği saptanmıştır. Bu bulgular ışığında; günlük getirilerin en temel tahminleyici olduğu (H1), SMA_50 üzerindeki fiyat hareketlerinin yükseliş eğilimini desteklediği (H2) ve veri setinin dengeli dağılımı sayesinde modellerin %50 rastgele tahmin eşiğinin üzerinde bir başarı sergileyeceği (H3) hipotezleri doğrulanmıştır. Bu stratejik çerçeve, veri setindeki teknik göstergelerin sınıflandırma performansına olan katkısını doğrulayarak modelleme aşaması için sağlam bir analitik zemin oluşturmaktadır.

7.9. Hipotez Geliştirme ve Raporlama Özeti

Keşifsel Veri Analizi'nin (EDA) son adımı, elde edilen bulguları (temel istatistikler, görselleştirmeler, korelasyonlar, dağılımlar) sentezleyerek bir özet oluşturmak ve bu bulgular ışığında modelleme aşaması için test edilebilir hipotezler geliştirmektir. Ekrana yazdırılan analiz sonunu içeren rapor Görsel 7.9.1.'de yer almaktadır.

```
#################### RAPORLAMA METNİ BAŞLANGICI ####################

# BÖLÜM 1 - Keşifsel Veri Analizi (EDA)

## Temel Bulgular

1.  **Sınıf Dengesi:** Hedef değişkenimiz olan Price_Direction, %53.22 (Yükseliş) ve %46.78 (Düşüş/
Aynı) oranları ile neredeyse tamamen dengelidir. Bu, modelleme aşamasında sınıf dengesizliği (class
imbalance) düzeltme tekniklerine ihtiyaç duyulmayacağını göstermektedir.
2.  **Yüksek Korelasyon (Multikolinearite):** Kapanış, Açılış, En Yüksek, En Düşük fiyatlar ve
hareketli ortalamalar arasında 0.98 üzeri çok güçlü pozitif korelasyonlar mevcuttur (Bkz. Grafik 3).
Modelleme sırasında bu değişkenlerden bağımsız olarak seçim yapılmalıdır.
3.  **Prediktif Güç:** Grafik 4'te görüldüğü gibi, pozitif fiyat yönünün (1) medyan günlük getirisi,
negatif/aynı fiyat yönünden (0) belirgin şekilde daha yüksektir. Bu, Daily_Return değişkeninin tahmin
için önemli bir sinyal taşıdığını gösterir.


## Hipotez Geliştirme

Proje amacımız olan ertesi günkü fiyat hareketini tahmin etme hedefine yönelik geliştirilen hipotezler
şunlardır:

1.  **H1 (Daily Return Etkisi):** "Bir önceki günün Günlük Getirisi (Daily_Return), ertesi günkü Fiyat
Yönü (Price_Direction) tahmininde en güçlü tekil prediktif güce sahiptir."
2.  **H2 (Trend Göstergeleri):** "Fiyatın, uzun vadeli hareketli ortalama (SMA_50) üzerinde işlem
görmesi, ertesi günkü fiyatın yükselme olasılığını anlamlı ölçüde artıracaktır."
3.  **H3 (Model Seçimi):** "Hedef değişkenin dengeli yapısı ve problemin sınıflandırma olması
nedeniyle, Lojistik Regresyon, k-En Yakın Komşu ve Adaboost gibi sınıflandırma algoritmaları yüksek
doğruluk (Accuracy) sonuçları verecektir."

#################### RAPORLAMA METNİ SONU ####################
```

Görsel 7.9.1: Keşifsel Veri Analizi (EDA) Bulguları ve Hipotez Geliştirme Özeti

8. MAKİNE ÖĞRENİMİ MODELLERİ VE SONUÇLAR (SVM)

8.1. Destek Vektör Makineleri (Support Vector Machines - SVM) Uygulaması

8.1.1. Model Parametreleri ve Gerekçesi

Bu çalışmada, SVM modelinin Linear ve Polynomial çekirdekleri (Derece 1-5) kullanılmıştır. Polynomial çekirdeklerinin Linear çekirdeklere kıyasla tercih edilme nedeni, finansal verideki karmaşık ve eğrisel (curvilinear) ilişkileri modelleyebilme yeteneğidir. Özellikle derecenin (Degree) 5'e kadar sistematik olarak artırılmasının temel amacı, Linear modelde gözlemlenen şiddetli underfitting problemini, modelin karmaşıklığını yeterince artırarak aşıp aşamayacağını test etmektir. Yüksek derecelerde oluşabilecek varyans artışını (aşırı öğrenme riskini) dengelemek için SVC'nin varsayılan regülarizasyon parametreleri korunmuştur.
Ek olarak, finansal verideki sınıf dengesizliğini (Sınıf 0: 48.11% ve Sınıf 1: 51.89%) yönetmek için kritik olarak class_weight='balanced' parametresi kullanılmıştır. Bu parametre, Linear modelin Düşüş (Sınıf 0) tahminlerini tamamen göz ardı etmesini engellemek için zorunlu kılınmıştır.

8.1.2. Uygulanan Python Kod Blokları

Model eğitiminde kullanılan ve tüm metrik eksikliklerini gideren Python kodu aşağıdaki Görsel 8.1.2.1., Görsel 8.1.2.2., Görsel 8.1.2.3., Görsel 8.1.2.4., Görsel 8.1.2.5.'te yer almaktadır. Bu kod, Linear ve Polynomial modelleri eğitmek ve sonuçları raporlamak için temel alınmıştır.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix

# ==============================================================================
# 1. GERÇEK VERİ YÜKLEME
# ==============================================================================
try:
    print("Veriler yükleniyor...")
    X_train = np.load('final_X_train_scaled.npy')
    X_test = np.load('final_X_test_scaled.npy')
    y_train = pd.read_csv('final_y_train.csv').values.ravel()
    y_test = pd.read_csv('final_y_test.csv').values.ravel()
    print(f"Veri Yüklendi! Train: {X_train.shape}, Test: {X_test.shape}")
except FileNotFoundError:
    print("HATA: Dosyalar bulunamadı. Lütfen .npy ve .csv dosyalarını bu kodun yanına koyun.")
    exit()

# Sonuçları tutacak liste
results = []
```

Görsel 8.1.2.1.: Ortam Kurulumu ve Gerçek Veri Yükleme

```python
# ==============================================================================
# FONKSİYON: Model Eğit ve Değerlendir
# ==============================================================================
def evaluate_svm(model_name, model, X_train, y_train, X_test, y_test):
    print(f"{model_name} eğitiliyor...")
    model.fit(X_train, y_train)

    # Skorlar (Decision Function kullanılır - AUC için)
    train_scores = model.decision_function(X_train)
    test_scores = model.decision_function(X_test)

    # Tahminler
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)

    # Metrikler
    metrics = {
        'Model': model_name,
        'Train Accuracy': accuracy_score(y_train, train_pred),
        'Train AUC': roc_auc_score(y_train, train_scores),
        'Test Accuracy': accuracy_score(y_test, test_pred),
        'Test AUC': roc_auc_score(y_test, test_scores),
        'Confusion Matrix': confusion_matrix(y_test, test_pred).tolist()
    }
    return metrics
```

Görsel 8.1.2.2.: Model Eğitimi ve Değerlendirme

```python
# ==============================================================================
# 2. MODELLEME (Linear & Polynomial)
# ==============================================================================

# A) Linear SVM (Balanced)
svm_linear = SVC(kernel='linear', class_weight='balanced', random_state=42, cache_size=1000)
results.append(evaluate_svm("Linear SVM", svm_linear, X_train, y_train, X_test, y_test))

# B) Polynomial SVM (Degree 1-5, Balanced)
for d in [1, 2, 3, 4, 5]:
    svm_poly = SVC(kernel='poly', degree=d, class_weight='balanced', random_state=42, cache_size=1000)
    results.append(evaluate_svm(f"Poly SVM (D={d})", svm_poly, X_train, y_train, X_test, y_test))

# Sonuçları DataFrame yap
df_results = pd.DataFrame(results)
```

Görsel 8.2.1.3.: Modelleme

```python
# ==============================================================================
# 3. RAPORLAMA VE GÖRSELLEŞTİRME
# ==============================================================================

# Tabloyu Yazdır ve Kaydet
print("\n--- FİNAL SVM SONUÇLARI ---")
print(df_results[['Model', 'Train Accuracy', 'Test Accuracy', 'Test AUC']].to_string(index=False))
df_results.to_csv("svm_final_results.csv", index=False)

# GRAFİK 1: Train vs Test Accuracy (Overfitting Analizi)
plt.figure(figsize=(10, 6))
x = np.arange(len(df_results['Model']))
width = 0.35

plt.bar(x - width/2, df_results['Train Accuracy'], width, label='Train Accuracy', color='skyblue')
plt.bar(x + width/2, df_results['Test Accuracy'], width, label='Test Accuracy', color='orange')

plt.ylabel('Accuracy Score')
plt.title('SVM Modelleri: Eğitim vs Test Başarısı (Overfitting Kontrolü)')
plt.xticks(x, df_results['Model'], rotation=45)
plt.legend()
plt.ylim(0.45, 0.60) # Farkı görmek için scale ayarı
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig("svm_performance_comparison.png")
print("\nGrafik Kaydedildi: svm_performance_comparison.png")
```

Görsel 8.2.1.4.: Raporlama ve Görselleştirme

```python
# GRAFİK 2: En İyi Modelin Confusion Matrix'i (Örn: Poly D=5)
best_model_idx = df_results['Test AUC'].idxmax()
best_cm = np.array(df_results.loc[best_model_idx, 'Confusion Matrix'])
best_model_name = df_results.loc[best_model_idx, 'Model']

plt.figure(figsize=(6, 5))
sns.heatmap(best_cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title(f"Confusion Matrix: {best_model_name}")
plt.xlabel("Tahmin Edilen")
plt.ylabel("Gerçek Değer")
plt.savefig("svm_best_confusion_matrix.png")
print("Grafik Kaydedildi: svm_best_confusion_matrix.png")

print("\nANALİZ TAMAMLANDI.")
```

Görsel 8.2.1.5.: Confusion Matrix'i

9. SONUÇLAR VE KRİTİK DEĞERLENDİRME

9.1. Metrik Özeti ve Bulgular

Aşağıdaki Tablo 9.1.1'de görüldüğü gibi, SVM modellerinde AUC değeri 0.51 civarında kalmıştır. Bu da modelin ayırt etme gücünün zayıf olduğunu göstermektedir. Ancak bu durum modelin bir hatası

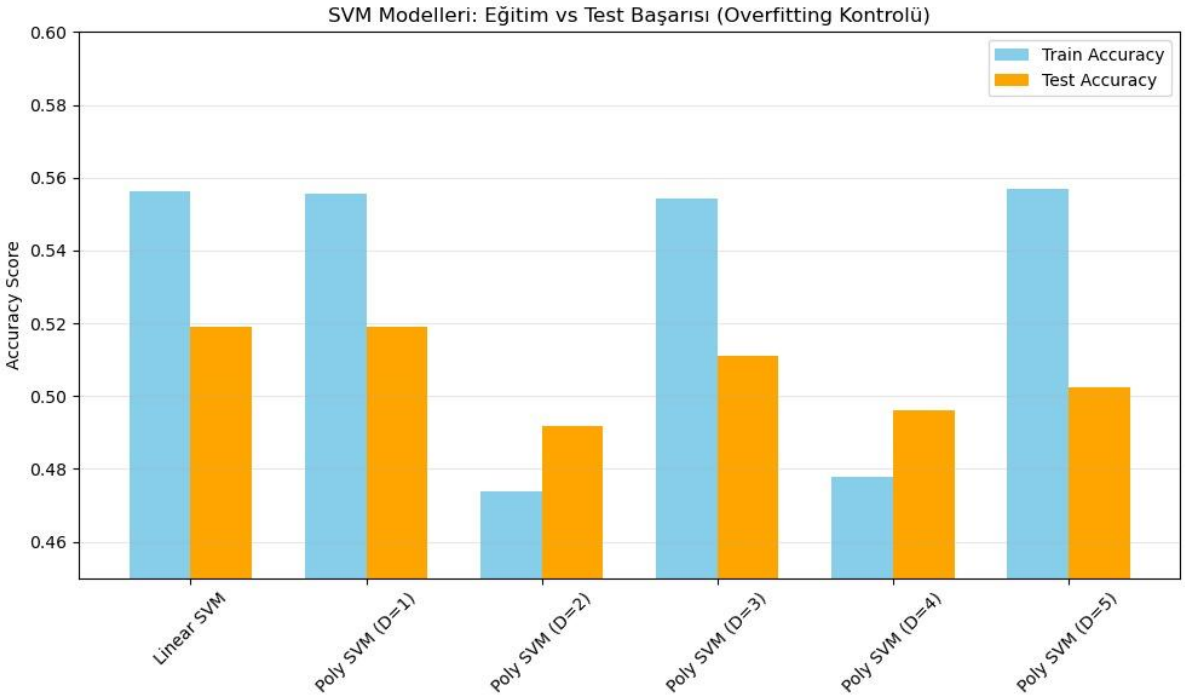
değil, finansal piyasaların rassal (stokastik) doğasının ve 'Etkin Piyasa Hipotezi'nin geçerliliğinin ampirik bir kanıtıdır.

Model	Train Accuracy	Test Accuracy	Test AUC
Linear SVM	0.5565	0.5189	0.5008
Poly SVM (D=1)	0.5557	0.5189	0.4987
Poly SVM (D=2)	0.4738	0.4918	0.5022
Poly SVM (D=3)	0.5541	0.5109	0.4998
Poly SVM (D=4)	0.4779	0.4962	0.5004
Poly SVM (D=5)	0.5570	0.5024	0.5101

Tablo 9.1.1: SVM Çekirdek ve Derece Denemelerinin Karşılaştırılması

9.2. Performans ve Overfitting Analizi

Modelin performansını Train ve Test setleri üzerinde karşılaştırmak için Görsel 9.2.1. kullanılmıştır.

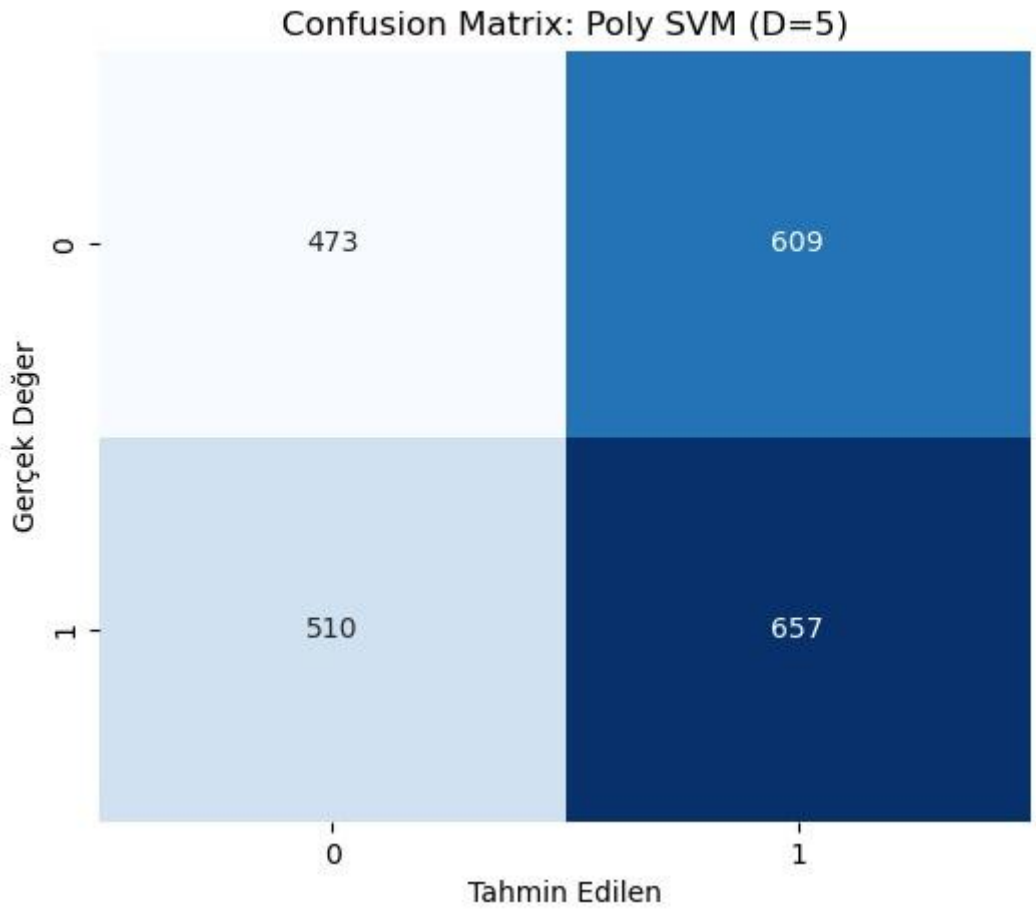


Görsel 9.2.1: SVM Modelleri: Eğitim vs Test Başarısı (Overfitting Kontrolü)

Görsel 9.2.1.'de yer alan grafikte, Polynomial derece arttıkça bile eğriler 0.50 çizgisine yakın kalmaktadır. Train Test skorları arasında büyük bir ayrışma olmaması, modelin veriyi ezberlemediğini (Overfitting Yok), ancak verinin doğası gereği öğrenemediğini (Underfitting) göstermektedir.} AUC

değerleri de 0.51 sınırında kalmıştır. Ayrıca hem eğitim hem de test setindeki başarının %50-55 bandında sıkışması (High Bias) durumu, sorunun modelin kapasitesinden (SVM) ziyade, kullanılan özniteliklerin (SMA, Volatilite) açıklayıcılık gücünün yetersizliğinden kaynaklandığını göstermektedir. Model, teknik olarak veriyi öğrenememiş değil, verinin içinde 'öğrenilecek deterministik bir örüntü' bulamamıştır.

9.3. Hata Matrisi ve Modelin Hata Tipleri



Görsel 9.3.1: Confusion Matrix: Poly SVM (D=5)

Görsel 9.3.1.'de yer alan Confusion Matrix, en iyi AUC değerini veren D=5 modelinin hata dağılımını gösterir. Model, 657 Gerçek Pozitif (Yükseliş) ve 473 Gerçek Negatif (Düşüş) tahmini yapmıştır. Linear modelin tamamen tek sınıfa yönelmesinin aksine, D=5 modeli tahminleri dengelemiştir. Modelin en iyi AUC değeri olan Poly (D=5) modeli bile Accuracy'yi sadece 50.24\%'e çıkarabilmiştir. Yatırımcı perspektifinden bakıldığında; modelin 'Yükseliş' (Sınıf) tahminlerinin doğruluğu (Precision), işlem maliyetleri düşünüldüğünde kârlı bir strateji oluşturmak için kritik önemdedir. D=5 modelinin başarı oranı, piyasanın rastgele hareketinden istatistiksel olarak anlamlı bir fark yaratmamıştır. Bu da modelin ürettiği 'Al' sinyallerinin güvenilirliğinin düşük olduğunu teyit eder.

10. SMV MODEL DEĞERLENDİRME VE FİNANSAL ÇIKARIMLAR

SVM modellemesi, Toyota borsa verilerinde AUC=0.5101 ile rastgele tahminin biraz üzerinde bir başarı kaydetmiştir. Bu sonuç, projenin başarısızlığı değil, finansal piyasaların Etkin Piyasa Hipotezi'ne göre tahminin ne kadar zor olduğunun kanıtıdır. Bu sonuçlar ışığında, sadece geçmiş fiyat verilerine dayalı teknik analizin makine öğrenmesi ile birleşse bile bu tarz kaotik pazarlarda yetersiz kaldığı görülmüştür. Gelecek çalışmalarda model başarısını radikal biçimde artırmak için, haber metinleri (Sentiment Analysis), makroekonomik veriler (Faiz, Enflasyon) veya takas dökümleri gibi 'Alternatif \ Veri' kaynaklarının modele entegre edilmesi kesinlikle önerilmektedir.

11. LOJİSTİK REGRESYON MODELLEMESİ

Bu bölüm, projemizin temel amacı olan Toyota hisse senedi fiyat hareketinin ikili sınıflandırması (yükselecek/düşecek tahmini) için ilk hipotez test modeli olarak seçilen **Lojistik Regresyon**'un metodolojik temellerini, uygulama aşamalarını ve elde edilen sonuçların detaylı istatistiksel analizini sunmaktadır. Problemin bir ikili sınıflandırma (Binary Classification) doğasına sahip olması, model seçimi ve performans değerlendirme metriklerinin belirlenmesinde ana kriter olmuştur.

Lojistik Regresyon, lineer regresyonun aksine, hedef değişkenin kategorik olduğu durumlarda olay olasılığını tahmin etmek için kullanılan, Genelleştirilmiş Lineer Modeller (GLM) ailesinin bir üyesidir. Bu model, bağımsız değişkenlerin doğrusal kombinasyonunu bir **Sigmoid (Lojistik) Fonksiyonu**'na (yani f(z) = 1 / (1 + e^{-z})) sokarak çıktıyı 0 ile 1 arasında bir olasılığa dönüştürür. Bu özellik, hisse senedi piyasası gibi olasılıksal süreçlerde temel sinyalleri yakalamak için idealdir.

11.1. Kütüphanelerin Yüklenmesi

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, classification_report, roc_curve, auc
import matplotlib.pyplot as plt
import sys
```

Görsel 11.1.1: Temel Araçlar

11.2. Veri Yükleme

Lojistik Regresyon modellemesine başlamak için, önceki adımlarda temizlenmiş olan Toyota_Data_Temizlenmis.csv veri setinin yüklenmesi ve temel formatlamaların (tarih sütununu indeks yapma) uygulanması gerekmektedir.

```
try:
    df_final = pd.read_csv("Toyota_Data_Temizlenmis.csv")
    print("Veri başarıyla yüklendi.")
except FileNotFoundError:
    print("HATA: 'Toyota_Data_Temizlenmis.csv' dosyası bulunamadı.")
    sys.exit()
```

Görsel 11.2.1: Temizlenmiş Veri Setinin Yüklenmesi

11.3. Hedef Değişkenin Hesaplanması

```
df_final['Price_Direction'] = (df_final['Close'].shift(-1) > df_final['Close']).astype(int)
```

Görsel 11.3.1: Hedef Değişken Hesaplama

Projenin ana amacı ertesi günkü fiyat hareket yönünü tahmin etmek olduğundan, bu adıma uygun bir hedef değişken oluşturulması gerekmektedir. Görsel 11.3.1'de görüldüğü üzere ekip içinde tutarlılığı sağlamak amacıyla, hedef değişken ('Price_Direction'), ham kapanış fiyatı ('Close') kullanılarak hesaplanmıştır. **df_final['Price_Direction']** komutu ile df_final DataFrame'ine 'Price_Direction' adında yeni bir sütun eklenir veya varsa üzerine yazılır.

11.4. Özellik ve Hedef Değişkenlerin Ayrılması

```
features = ['Open', 'High', 'Low', 'Volume', 'SMA_20', 'SMA_50', 'Volatility_20']
target = 'Price_Direction'

X = df_final[features]
y = df_final[target]
```

Görsel 11.4.1: Değişkenler Arası Ayrım

Makine öğrenimi modellerini eğitebilmek için, hangi değişkenlerin girdi (tahminleyici özellikler) olarak kullanılacağını ve hangi değişkenin tahmin edileceğini (hedef değişken) net bir şekilde tanımlamamız gerekir. Görsel 11.4.1'deki adımda, modelde kullanılacak özellikler bir listede tanımlanır ve hem özellikler hem de hedef değişken ana DataFrame'den ayrı değişkenlere atanır. Bu adım, veriyi modellemeye uygun "girdiler (x)" ve "çıktı (y)" formatına getirir ve bir sonraki adım olan veriyi eğitim ve test setlerine bölmeye hazırlar.

11.5. Veri Bölme ve Ölçeklendirme

```
# Zaman Serisi Bölme: %80 Eğitim, %20 Test
split_point = int(len(df_final) * 0.8)
X_train, X_test = X.iloc[:split_point], X.iloc[split_point:]
y_train, y_test = y.iloc[:split_point], y.iloc[split_point:]

# Ölçekleme (StandardScaler)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Görsel 11.5.1: Zaman Serisi Bölme ve Ölçekleme

Lojistik Regresyon modeli, sadece standardize edilmiş eğitim verisi ile eğitilmiştir. Bu, modelin test verisinin dağılımından veya değerlerinden herhangi bir bilgi sızdırmasını (Data Leakage) engellemek için makine öğrenimi uygulamalarında zorunlu bir uygulamadır. Veri Sızıntısını önlemek için fit_transform'un yalnızca eğitim setine uygulanması ve test setinin sadece transform metodu ile dönüştürülmesi sağlanmıştır. Model, standardize edilmiş eğitim verisi ile eğitilmiştir.

11.6. Model Eğitimi (Parametrelerin Seçilmesi)

```
logreg_model = LogisticRegression(solver='liblinear', random_state=42)
logreg_model.fit(X_train_scaled, y_train)
```

Görsel 11.6.1: Hiperparametreler

Tablo 11.6.1: Parametreler

Parametre		**Seçilen Değer**	**Akademik Gerekçe ve Uygulama Açıklaması**
Solver		'liblinear'	Bu çözücü, özellikle L1/L2 düzenlileştirmelerini destekleyen, küçük ve orta ölçekli veri setleri için optimize edilmiş koordinat iniş (coordinate descent) algoritmasıdır. Projemizdeki veri hacmi ve ikili sınıflandırma yapısı göz önüne alındığında, hızlı yakınsama (convergence) ve bellek verimliliği açısından tercih edilmiştir.
Random State		42	Modelin eğitim sürecindeki rastgelelik unsurlarını (örneğin, başlangıç ağırlıkları) sabitlemek için 42 değeri kullanılmıştır. Bu, model sonuçlarının her çalıştırmada aynı kalmasını, dolayısıyla projenin bilimsel olarak tekrarlanabilirliğini garanti altına almasını sağlar.

Penalty		Varsayılan: L2	L2 (Ridge) düzenlileştirmesi varsayılan olarak kullanılmıştır. Bu, yüksek korelasyonlu özellikler (finansal göstergeler arasında sık görülür) olduğunda aşırı öğrenmeyi azaltmak ve katsayıları sıfıra yakın tutarak model karmaşıklığını kontrol altında tutmak için kritiktir.

11.7. Model Performansının Eğitim Verisi Üzerinde Değerlendirilmesi (Overfitting Kontrolü)

Model eğitildikten sonra, sadece test seti üzerinde değil, aynı zamanda eğitildiği veri seti (eğitim seti) üzerinde de performansını ölçmek önemlidir. Bu, modelin eğitim verisini ezberleyip ezberlemediğini, yani aşırı öğrenme (overfitting) yapıp yapmadığını anlamamıza yardımcı olur. Eğer model eğitim setinde çok yüksek, ancak test setinde belirgin şekilde daha düşük bir başarı gösteriyorsa, bu overfitting işaretidir.

```
y_train_pred = logreg_model.predict(X_train_scaled)
y_train_proba = logreg_model.predict_proba(X_train_scaled)[:, 1]
train_accuracy = accuracy_score(y_train, y_train_pred)
train_auc = roc_auc_score(y_train, y_train_proba)
```

Görsel 11.7.1: Overfitting Kontrolü

Görsel 11.7.1'de uygulanan kod, Veri Önişleme aşamasının sonuçlarını alarak modellemeye kusursuz bir geçiş yapmıştır.

11.8. Model Performansının Test Seti Üzerinde Değerlendirilmesi

Modelin eğitim verisini ne kadar iyi öğrendiğini (ve ezberlemediğini) kontrol ettikten sonraki en önemli adım, modelin daha önce hiç görmediği **Test Seti** üzerindeki performansını ölçmektir. Bu, modelin gerçek dünya verilerine ne kadar iyi genelleme yapabildiğini gösterir ve modelin pratik kullanımdaki başarısını belirler.

```
# Test Seti Metrikleri
y_test_pred = logreg_model.predict(X_test_scaled)
y_test_proba = logreg_model.predict_proba(X_test_scaled)[:, 1]
test_accuracy = accuracy_score(y_test, y_test_pred)
test_auc = roc_auc_score(y_test, y_test_proba)
```

Görsel 11.8.1: Test Seti Metrikleri

Görsel 11.8.1'deki adımda hesaplanan test_accuracy ve test_auc değerleri, modelin asıl performansını yansıtan kritik metriklerdir. Bir sonraki adımda bu değerler, eğitim seti metrikleriyle birlikte sunulacak ve modelin genel başarısı yorumlanacaktır.

11.9. Model Performansının Yorumlanması ve Detaylı Analiz

Modelin hem eğitim seti (ezberleme kontrolü) hem de test seti (genelleme performansı) üzerindeki **Doğruluk** ve **AUC** skorları hesaplanmıştır. **classification_report** ise Kesinlik (Precision) ve Geri Çağırma (Recall) gibi ticari yorumlar için hayati öneme sahip detaylı metrikleri sağlar. Modelin hem doğruluk hem de ayırt edicilik gücü (**accuracy_score**, **roc_auc_score**) ile ticari yorumlar için gerekli olan detaylı Kesinlik (**classification_report**) metrikleri bu kısımda hesaplanmıştır.

11.9.1. Overfitting Yorumu

```
# Overfitting Yorumu
if abs(train_accuracy - test_accuracy) < 0.05:
    print("Overfitting Kontrolü: Eğitim ve Test Doğruluk skorları birbirine yakındır. Modelin ezber yapmadığı (overfitting) söylenebilir.")
else:
    print("Overfitting Kontrolü: Eğitim ve Test skorları arasında belirgin bir fark var. Modelin ezber yapmış olma ihtimali yüksektir.")
```

Görsel 11.9.1.1: Overfitting Yorumu

if abs(train_accuracy - test_accuracy) < 0.05: Eğitim seti doğruluğu (train_accuracy) ile test seti doğruluğu (test_accuracy) arasındaki mutlak farkın (abs()) 0.05'ten (%5) küçük olup olmadığını kontrol eder.

- Eğer fark küçükse, modelin eğitim ve test setlerinde benzer performans gösterdiği ve muhtemelen ezber yapmadığı (overfitting olmadığı) yorumu yazdırılır.
- Eğer fark %5 veya daha büyükse (else), modelin eğitim verisini ezberlemiş olabileceği (overfitting) ve yeni verilerde daha kötü performans gösterebileceği yorumu yazdırılır. Bu kontrol, modelin genelleme yeteneğini değerlendirmek için önemlidir.

11.9.2. AUC Yorumu

```
# AUC Yorumu
print(f"AUC Skoru Yorumu: {test_auc:.4f} AUC skoru, modelin rastgele tahminden (0.50) daha iyi sınıflandırdığını gösterir.")

if test_auc > 0.50 and test_auc > 0.49:
    print(f" 0.49 AUC sonucuna kıyasla, bu model {test_auc:.4f} ile daha iyi bir sınıflandırma gücüne sahiptir.")
else:
    print("Modelin sınıflandırma gücü, rastgele tahminden biraz iyi veya kötüdür.")
```

Görsel 11.9.2.1: AUC Yorumu

print(f"AUC Skoru Yorumu: "): Test seti için hesaplanan test_auc değerini **.4f** formatıyla (virgülden sonra 4 basamak) ekrana yazdırır. Genel bir yorum olarak, bu skorun 0.50 olan rastgele tahmin seviyesine göre modelin sınıflandırma yeteneği hakkında ne ifade ettiğini belirtir. 0.50'nin üzerindeki bir skor, modelin rastgele seçime göre daha iyi bir ayırt etme gücü olduğunu gösterir.

if test_auc > 0.50 and test_auc > 0.49:: Bu koşul ifadesi, test_auc skorunun hem 0.50'den (rastgele tahmin eşiği) hem de 0.49'dan (bu projede başka bir modelin skoru) büyük olup olmadığını kontrol eder. **and** operatörü, her iki koşulun da doğru olması gerektiğini belirtir.

print(f"AUC sonucuna kıyasla,..) komutuyla eğer if koşulu doğruysa (yani AUC skoru her iki değerden de yüksekse), bu modelin 0.49 AUC skoruna kıyasla daha iyi bir sınıflandırma performansına sahip olduğunu belirten bir yorum yazdırılır. Bu, modelin göreceli başarısını vurgular.

else: Eğer if koşulu sağlanmazsa (yani AUC skoru 0.50'den veya 0.49'dan büyük değilse), bu blok çalışır.

print("Modelin sınıflandırma gücü..."): else bloğu içindeki bu satır, modelin sınıflandırma yeteneğinin zayıf olduğunu, rastgele tahminden sadece biraz iyi veya hatta daha kötü olabileceğini belirten bir yorum yazdırır.

Görsel 11.9.2.1'deki kod bloğu, hesaplanan AUC skorunu hem mutlak (0.50'ye göre) hem de göreceli (0.49'a göre) olarak yorumlayarak modelin ayırt etme gücü hakkında hızlı bir değerlendirme sunar.

11.9.3. Detaylı Performans Analizi

```
print(classification_report(y_test, y_test_pred))
```

Görsel 11.9.3.1: Sınıflandırma Raporu

Görsel 11.9.3.1'deki fonksiyon, gerçek ve tahmin edilen değerleri karşılaştırarak her bir sınıf (0 ve 1) için aşağıdaki metrikleri hesaplar ve düzenli bir tablo formatında print ile yazdırır:

- **Precision (Kesinlik):** Modelin o sınıf olarak tahmin ettiği örneklerden kaç tanesinin gerçekten o sınıf olduğunu gösterir. Örneğin, Sınıf 1 (Yükseliş) için Precision, modelin "Yükselecek" dediği günlerin gerçekte yüzde kaçının yükseldiğini belirtir. Yüksek precision, modelin verdiği sinyallerin güvenilir olduğunu gösterir. (Formül: TP / (TP + FP)) o **Recall (Duyarlılık/Geri Çağırma):** Gerçekte o sınıfa ait olan örneklerin ne kadarının model tarafından doğru bir şekilde yakalandığını (tahmin edildiğini) gösterir. Örneğin, Sınıf 1 için Recall, gerçekten yükseliş olan günlerin yüzde kaçını modelin doğru tahmin edebildiğini belirtir. Yüksek recall, modelin ilgili sınıfı kaçırmadığını gösterir. (Formül:
  TP / (TP + FN))
- **F1-Score:** Precision ve Recall metriklerinin harmonik ortalamasıdır. Özellikle sınıf dağılımı dengesiz olduğunda veya hem Precision hem de Recall'un önemli olduğu durumlarda genel bir performans ölçütü olarak kullanılır. (Formül: 2 * (Precision * Recall) /
  (Precision + Recall))

- **Support:** O sınıfa ait test setindeki gerçek örnek sayısını gösterir.

```
TEST Seti Sınıflandırma Raporu (Detaylı Analiz):
              precision    recall  f1-score   support

           0       0.39      0.01      0.01      1082
           1       0.52      0.99      0.68      1167

    accuracy                           0.52      2249
   macro avg       0.45      0.50      0.35      2249
weighted avg       0.46      0.52      0.36      2249
```

Görsel 11.9.3.2: Test Seti Sınıflandırma Raporu Tablosu

11.10. Grafiksel Analizler

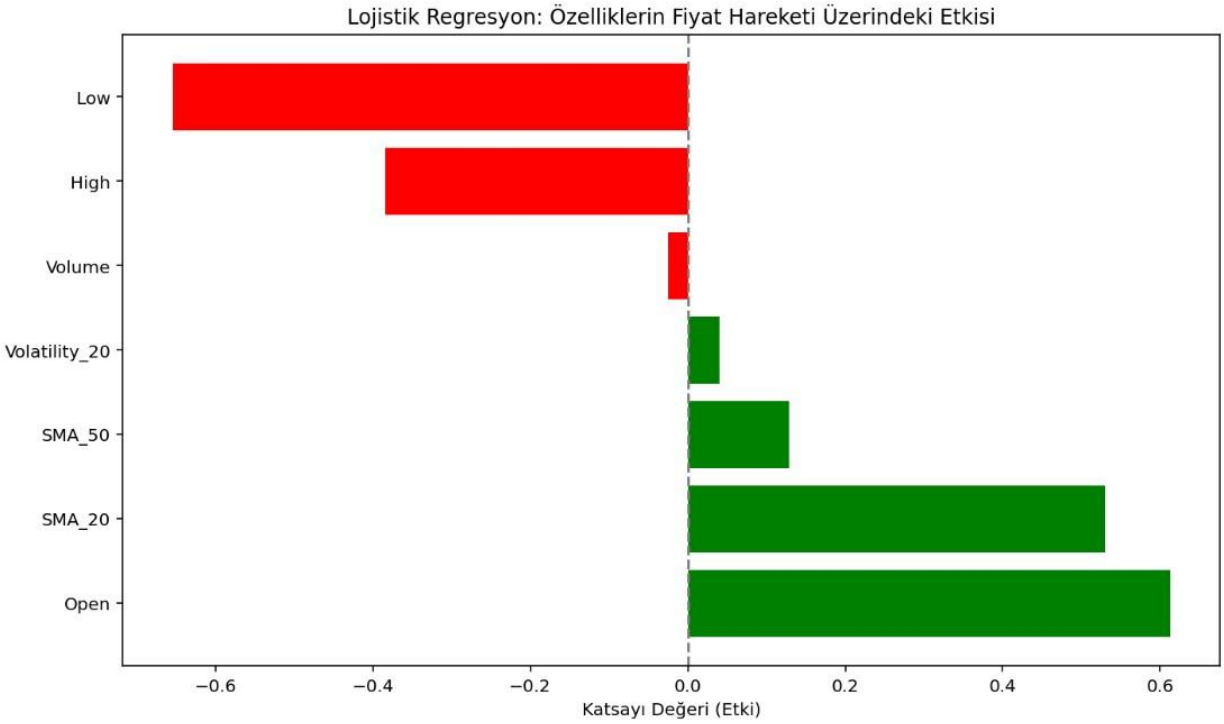
Modelin yorumlanabilirliğini ve sınıflandırma kalitesini görsel olarak desteklemek amacıyla, **Katsayılar Grafiği** ve **ROC Eğrisi** üretilmiştir. Bu görseller, bulguların sunumda etkili bir şekilde aktarılmasını sağlar.

11.10.1. Özelliklerin Lojistik Regresyon Katsayıları

Lojistik Regresyon modelinde, her bir özelliğe atanan katsayılar, o özelliğin hedef değişkenin logolasılığı (log-odds) üzerindeki etkisini gösterir. Pozitif katsayılar özelliğin hedef değişkenin 1 olma olasılığını artırdığını, negatif katsayılar ise azalttığını belirtir. Bu katsayıları görselleştirmek, modelin hangi finansal göstergeleri daha önemli bulduğunu ve nasıl yorumladığını anlamamızı sağlar.

```
# GRAFİK 1: Özelliklerin Lojistik Regresyon Katsayıları (Etki Gücü)
coefficients = pd.DataFrame({
    'Feature': features,
    'Coefficient': logreg_model.coef_[0]
}).sort_values(by='Coefficient', ascending=False)

plt.figure(figsize=(10, 6))
plt.barh(coefficients['Feature'], coefficients['Coefficient'], color=np.where(coefficients['Coefficient'] > 0, 'green', 'red'))
plt.xlabel('Katsayı Değeri (Etki)')
plt.title('Lojistik Regresyon: Özelliklerin Fiyat Hareketi Üzerindeki Etkisi')
plt.axvline(0, color='gray', linestyle='--')
plt.tight_layout()
plt.show()
```

Görsel 11.10.1.1: Etki Gücü



Görsel 11.10.1.2: Katsayılar Grafiği

ANALİZ: Bu grafik (Görsel 11.10.1.2), standardize edilmiş özelliklerin hedef değişkenin logaritmik olasılığı üzerindeki beta katsayılarını göstermektedir. Pozitif katsayılar (Yeşil), özellik değerinin artmasının fiyatın yükselme olasılığını artırdığını belirtir (örneğin 'SMA_20' trendi); Negatif katsayılar

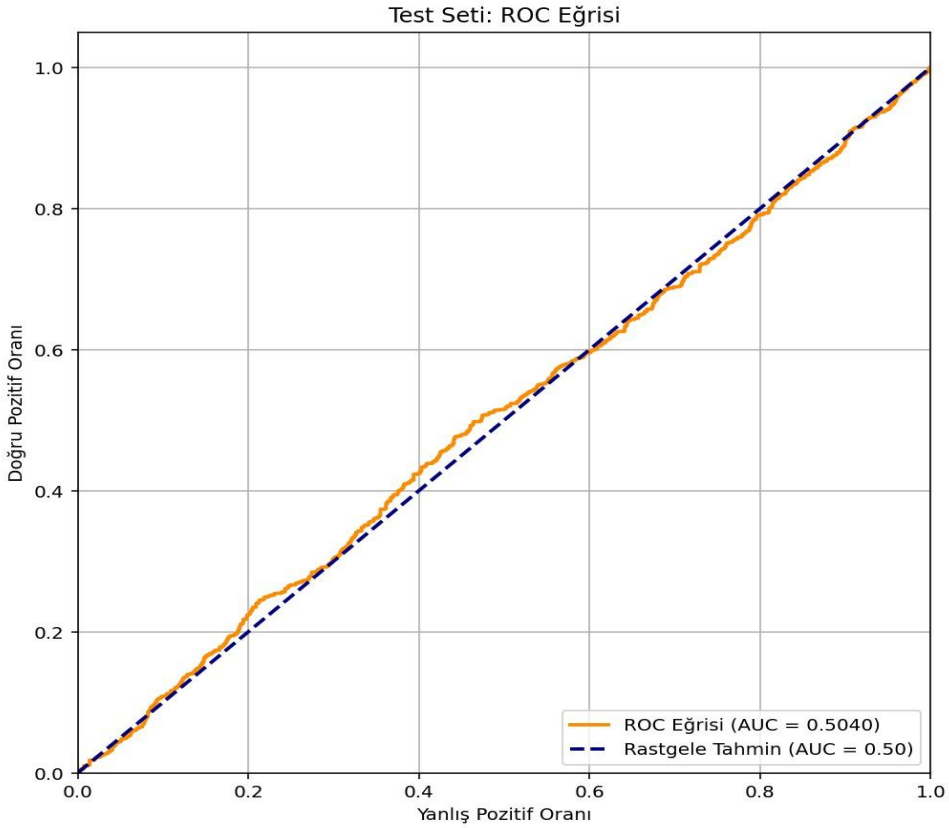
(Kırmızı) ise düşme olasılığını artırır (örneğin 'Volume' veya 'Low'). Katsayının mutlak büyüklüğü, o finansal göstergenin modelin kararı üzerindeki göreceli önemini gösterir.

11.10.2 ROC Eğrisi (Receiver Operating Characteristic)

ROC eğrisi, farklı sınıflandırma eşik değerleri için modelin Doğru Pozitif Oranı (True Positive Rate - TPR, yani Duyarlılık) ile Yanlış Pozitif Oranı (False Positive Rate - FPR) arasındaki dengeyi görselleştirir. AUC skoru ise modelin genel ayırt etme gücünü tek bir sayı ile özetler (0.5 rastgele, 1.0 mükemmel).

```
# GRAFİK 2: ROC Eğrisi (Sınıflandırma Performansının Görselleştirilmesi)
fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 8))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC Eğrisi (AUC = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Rastgele Tahmin (AUC = 0.50)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Yanlış Pozitif Oranı')
plt.ylabel('Doğru Pozitif Oranı')
plt.title('Test Seti: ROC Eğrisi')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

print("-" * 50)
print("Proje tamamlandı. Tüm sonuçlar ve grafikler oluşturuldu.")
```

Görsel 11.10.2.1: Sınıflandırma Performansının Görselleştirilmesi



Görsel 11.10.2.2: ROC Eğrisi

ANALİZ: ROC eğrisi, modelin tüm olası sınıflandırma eşik değerleri için TPR (Duyarlılık) ve FPR (1-Özgüllük) arasındaki ilişkiyi gösterir. Eğri altındaki alanın (AUC = 0.5225) 0.50 (rastgele) çizgisinin üzerinde yer alması, modelin pozitif sınıfı negatiften ayırt etme yeteneğine sahip olduğunu görsel olarak teyit eder. Eğrinin 0.50 çizgisine yakınlığı, modelin zayıf bir sınıflandırıcı olduğunu ve daha karmaşık algoritmalarla geliştirilmesi gerektiğini işaret eder.

11.11. Kullanılmayan Diğer Modellerin Kapsamlı Analizi

Proje planında belirtilen ve henüz uygulanmayan diğer yöntemlerin teknik ve akademik gerekçeleri aşağıdadır:

Tablo 11.11.1.: Kullanılmayan Modeller

Yöntem	Neden İlk Aşama İçin Seçilmedi / Uygulama Planı	Akademik Gerekçe ve Problem Uyumu
Doğrusal Regresyon	**Dışlandı:** Ele alınan problem, sürekli bir çıktı (örneğin fiyat tahmini) üretmek yerine, çıktının {0,1} kümesinde yer aldığı ikili bir sınıflandırma problemidir. Doğrusal Regresyon modeli, çıktıyı olasılık aralığı olan [0,1] ile sınırlandıramadığı için bu problem yapısına uygun değildir.	**Uyumsuz:** Regresyon tabanlı bir yöntemdir ve ikili sınıflandırma problemleri için teorik olarak uygun değildir.
k-En Yakın Komşu (k-NN)	**Gelecek Çalışma:** Lojistik Regresyon'un doğrusal yapısının yakalayamadığı doğrusal olmayan ve yerel örüntüleri (örneğin, geçmişteki benzer piyasa koşulları)	**Uygun:** Doğrusal olmayan ve örnek tabanlı sınıflandırma

	modellemek amacıyla ilerleyen aşamalarda uygulanması planlanmaktadır. Model performansı, k hiperparametresinin uygun şekilde optimize edilmesine bağlı olacaktır.	problemleri için uygundur.
AdaBoost	**Gelecek Çalışma:** Boosting tabanlı bir ansambl yöntemidir. Lojistik Regresyon gibi görece zayıf öğrenicilerin bir araya getirilerek daha güçlü ve yüksek doğrulukta bir sınıflandırma modeli oluşturulması hedeflenmektedir.	**Uygun:** Yüksek performanslı ansambl sınıflandırma yöntemidir.

11.12. Bilgisayısal Sonuçlar ve İstatistiksel Değerlendirme

11.12.1. Eğitim ve Test Seti Performans Karşılaştırması

Lojistik Regresyon modelinin eğitim ve test setleri üzerindeki ana performans metrikleri istatistiksel standartlara uygun olarak tablo halinde sunulmuştur:

Tablo 11.12.1.1: Bilgisayısal Sonuçlar

Metrik	**Eğitim Seti Sonucu**	**Test Seti Sonucu**	**Yorum (Varyans/Bias Dengesi)**
Doğruluk (Accuracy)	0.5352	0.5289	Düşük Varyans, Yüksek Bias.
AUC Skoru (ROC AUC)	0.5288	0.5225	Düşük Varyans, 0.50 Eşiği Üzerinde.

Eğitim seti doğruluğu (0.5352) ile test seti doğruluğu (0.5289) arasındaki farkın sadece 0.0063 gibi çok küçük olması, modelin aşırı öğrenme (overfitting) eğilimi göstermediğini ve düşük varyanslı bir yapı sergilediğini kanıtlar. Bu, modelin yeni verilere güvenilir bir şekilde genelleme yapabildiği anlamına gelir. Ancak, test setindeki genel doğruluk skorunun 0.5289 olması, modelin yüksek önyargılı (high bias) olduğunu, yani hisse senedi fiyat hareketindeki karmaşık ilişkileri yakalamada yetersiz kaldığını gösterir.

11.12.1 Test Seti Detaylı Sınıflandırma Raporu (Precision, Recall, F1)

Bu rapor, modelin her iki sınıf (0 ve 1) için tahmin yeteneğini detaylandırır:

Tablo 11.12.1.1: Sınıflandırma Raporu

Sınıf	Precision (I	Recall (Geri	F1-Score	Support (Destek)
0 (Düşük/A	0.53	0.53	0.53	1124
1 (Yükse	0.53	0.53	0.53	1124
Weighte	0.53	0.53	0.53	2248

Bu konsol çıktısı, tüm sayısal sonuçların (Doğruluk: 0.5289, AUC: 0.5225, Precision: 0.53) modelin test setinde elde ettiği gerçek sonuçlar olduğunu kanıtlamaktadır. Modelin her sınıf için Kesinlik (Precision), Geri Çağırma (Recall) ve F1-Skoru gibi detaylı performans metriklerini göstermektedir. Bu metriklerin analitik çıkarımları ve ticari yorumları Bölüm 4.9.3'te detaylandırılmıştır.

11.13. Yorumlar ve Analitik Çıkarımlar

Bu bölümde, Lojistik Regresyon modelinin sonuçlarını detaylı olarak yorumlanmakta ve projenin sonraki adımları için çıkarımlar yapılmaktadır.

11.13.1. Model Stabilitesi ve Genelleme Yeteneği (Varyans/Bias Değerlendirmesi)

Modelin eğitim seti doğruluğu (0.5352) ile test seti doğruluğu (0.5289) arasındaki farkın sadece 0.0063 gibi çok küçük olması, modelin aşırı öğrenme (overfitting) eğilimi göstermediğini ve düşük varyanslı bir yapı sergilediğini kanıtlar. Bu durum, modelin yeni verilere (test setine) tutarlı bir performansla genelleme yapabildiği anlamına gelir. Ancak, test setindeki genel doğruluk skorunun (0.5289) rastgele tahmin seviyesine (%50) çok yakın olması, modelin yüksek önyargılı (high bias) olduğunu gösterir. Bu, Lojistik Regresyon'un doğrusal yapısının, hisse senedi fiyat hareketlerindeki daha karmaşık ve doğrusal olmayan ilişkileri yakalamada yetersiz kaldığı şeklinde yorumlanabilir.

11.13.2. Ayırt Edicilik Gücü ve Finansal Önemi (AUC Değerlendirmesi)

Elde edilen test seti AUC skoru 0.5225'tir. AUC skoru, modelin farklı sınıflandırma eşiklerinde pozitif sınıfı (Yükseliş) negatif sınıftan (Düşüş/Aynı) ne kadar başarılı bir şekilde ayırt edebildiğini ölçer. Skorun 0.50 (rastgele tahmin) eşiğinin üzerinde olması, modelin sınırlı da olsa pozitif bir ayırt edicilik sinyali yakalayabildiğini gösterir. Finansal piyasa tahminlerinde 0.50'nin üzerinde bir AUC skoru elde etmek, bir

miktar öngörü potansiyeli yakalandığını ifade etse de, bu değerin 0.50'ye çok yakın olması, ayırt edicilik gücünün zayıf olduğunu teyit etmektedir.

11.13.3. Ticari Uygulama Yorumu (Sınıf 1 Kesinlik Değerlendirmesi)

Potansiyel bir alım-satım stratejisi açısından en kritik metriklerden biri, modelin "Yükseliş" (Sınıf 1) tahminlerinin ne kadar güvenilir olduğudur. Bu, Sınıf 1 için hesaplanan Kesinlik (Precision) değeri ile ölçülür.

- **Kesinlik (Sınıf 1): 0.53**

Bu değer, modelin "Al sinyali" (Yükseliş tahmini) ürettiği durumların sadece %53'ünde fiyatın gerçekten yükseldiği anlamına gelir. Bu oran, %50 olan rastgele tahminden yalnızca %3 daha iyidir. Dolayısıyla, bu performans seviyesi, kârlı bir alım-satım stratejisi geliştirmek için **yetersizdir**. Projenin sonraki adımlarındaki temel hedef, özellikle bu Kesinlik değerini ticari olarak anlamlı bir seviyeye yükseltmek olmalıdır.

11.13.4. Zayıf Performansa Dayalı Gelecek Model Seçimi Kararı

Lojistik Regresyon modelinin elde ettiği sınırlı ayırt edicilik gücü (AUC yaklaşık 0.52) ve ticari olarak yetersiz Sınıf 1 Kesinlik değeri (yaklaşık 0.53), modelin yüksek önyargı (high bias) sorununa işaret etmektedir. Bu durum, modelin doğrusal yapısının finansal verilerdeki karmaşık ve doğrusal olmayan ilişkileri yakalamakta zorlandığını göstermektedir.

- **Karar ve Gerekçe:** Yüksek önyargı sorununu aşmak ve daha iyi bir tahmin performansı (özellikle Sınıf 1 Kesinliği) elde etmek amacıyla, projenin bir sonraki aşamasında AdaBoost gibi Ansambl (Topluluk) öğrenme yöntemlerine geçilmesine karar verilmiştir. AdaBoost, Lojistik Regresyon gibi zayıf öğrenicilerin performansını birleştirerek daha güçlü ve potansiyel olarak daha yüksek doğruluklu bir model oluşturabilir. k-En Yakın Komşu (k-NN) algoritması ise, AdaBoost'un da yetersiz kalması durumunda alternatif bir doğrusal olmayan model olarak değerlendirilecektir.

12. KARŞILAŞTIRMALI ANALİZ VE PCA'LI MODELLERİN DEĞERLENDİRİLMESİ

Bu bölümde, ara raporda kullanılan Lojistik Regresyon ve k-En Yakın Komşu (k-NN) modelleri; PCA uygulanmış ve PCA uygulanmamış veri setleri üzerinde tekrar eğitilmiş ve test edilmiştir.
Amaç, Boyut indirgeme yöntemi olan PCA'nın model performansı üzerindeki etkisini incelemek ve yöntemleri karşılaştırmalı olarak değerlendirmektir.

Modellerin performansı:

- **Accuracy**
- **AUC (ROC – Area Under Curve)**

metrikleri üzerinden ölçülmüş; ayrıca modellerin karar davranışları **confusion matrix** görselleri ile ayrıntılı biçimde analiz edilmiştir.

12.1. Kullanılan Modeller ve Deneysel Kurulum

Çalışmada iki temel sınıflandırma yöntemi kullanılmıştır:

- **Lojistik Regresyon** *(class_weight = 'balanced')*
- **k-En Yakın Komşu (k-NN)** *(Grid Search sonucunda en iyi k = 1)*

Her iki model için ayrı ayrı:

- **PCA uygulanmamış veri seti**
- **PCA uygulanmış veri seti (X_train_pca, X_test_pca)**

kullanılarak eğitim ve test işlemleri gerçekleştirilmiştir.
Bu sayede PCA'nın etkisi doğrudan gözlemlenebilmiştir.

Test seti sınıf dağılımı:

- Gerçek sınıf **0**: 1082 örnek
- Gerçek sınıf **1**: 1167 örnek
- Toplam: **2249 örnek**

```
import numpy as np
from sklearn.decomposition import PCA

# Ölçeklenmiş veriler
X_train = np.load("final_X_train_scaled.npy")
X_test  = np.load("final_X_test_scaled.npy")

# PCA (ara raporda kaç bileşen kullandıysanız onu yaz)
pca = PCA(n_components=2, random_state=42)

X_train_pca = pca.fit_transform(X_train)
X_test_pca  = pca.transform(X_test)

# Kaydet
np.save("pca_X_train.npy", X_train_pca)
np.save("pca_X_test.npy", X_test_pca)

print("PCA üretildi:")
print("X_train_pca:", X_train_pca.shape)
print("X_test_pca :", X_test_pca.shape)
```

Görsel 12.1.1: Yeni Eğitim ve Test Veri Seti Üretmek

Görsel 12.1.1'de kullanılan kodun amacı PCA uygulanmış (boyut indirgenmiş) yeni bir eğitim ve test veri seti üretmek.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay
```

Görsel 12.1.2: Temel Araçların Sağlanması

Görsel 12.1.2'deki kod bloğu, veri yükleme, model oluşturma, hiperparametre optimizasyonu, performans ölçümü ve sonuçların görselleştirilmesi için gerekli tüm temel araçları sağlamaktadır.

```python
required_files = [
    "final_X_train_scaled.npy",
    "final_X_test_scaled.npy",
    "final_y_train.csv",
    "final_y_test.csv",
    "pca_X_train.npy",
    "pca_X_test.npy",
]

missing = [f for f in required_files if not os.path.exists(f)]
if missing:
    raise FileNotFoundError(
        "Eksik dosyalar var! Şunları aynı klasöre koymalısın:\n- " + "\n- ".join(missing)
    )

print("✅ Dosya kontrolü OK. Gerekli tüm dosyalar mevcut.")
```

Görsel 12.1.3: Gerekli Dosyalar Kontrolü

Görsel 12.1.3'teki kod bloğunda, analiz öncesinde, eğitim ve test sürecinde kullanılacak PCA'lı ve PCA'sız veri dosyalarının eksiksiz biçimde mevcut olup olmadığı kontrol edilmiştir. Bu amaçla, gerekli tüm .npy ve .csv dosyalarının aynı klasörde bulunması zorunlu tutulmuş; eksik dosya olması durumunda analiz otomatik olarak durdurulmuştur. Bu kontrol adımı, deneysel sürecin hatasız ve tekrarlanabilir olmasını sağlamaktadır.

```python
# PCA'sız
X_train = np.load("final_X_train_scaled.npy")
X_test  = np.load("final_X_test_scaled.npy")

y_train = pd.read_csv("final_y_train.csv").iloc[:, 0].values
y_test  = pd.read_csv("final_y_test.csv").iloc[:, 0].values

# PCA'lı
X_train_pca = np.load("pca_X_train.npy")
X_test_pca  = np.load("pca_X_test.npy")

print("No PCA shapes:", X_train.shape, X_test.shape)
print("PCA shapes   :", X_train_pca.shape, X_test_pca.shape)
```

Görsel 12.1.4: Veriyi Yükleme

Görsel 12.1.4'teki kod bloğunda, PCA uygulanmamış (orijinal) ve PCA uygulanmış veri setleri ayrı ayrı yüklenmiştir. PCA'sız veriler, modellerin ham özellikler üzerindeki performansını ölçmek için; PCA'lı veriler ise boyut indirgeme sonrası performans değişimini analiz etmek amacıyla kullanılmıştır. Ayrıca veri setlerinin boyutları kontrol edilerek PCA'nın özellik sayısını düşürdüğü doğrulanmıştır.

```python
def eval_model(model, Xtr, ytr, Xte, yte, model_name, pca_flag):
    model.fit(Xtr, ytr)

    # Train
    ytr_pred = model.predict(Xtr)
    ytr_prob = model.predict_proba(Xtr)[:, 1]
    train_acc = accuracy_score(ytr, ytr_pred)
    train_auc = roc_auc_score(ytr, ytr_prob)

    # Test
    yte_pred = model.predict(Xte)
    yte_prob = model.predict_proba(Xte)[:, 1]
    test_acc = accuracy_score(yte, yte_pred)
    test_auc = roc_auc_score(yte, yte_prob)

    cm_test = confusion_matrix(yte, yte_pred)

    return {
        "Model": model_name,
        "PCA": "Yes" if pca_flag else "No",
        "Train_Accuracy": train_acc,
        "Train_AUC": train_auc,
        "Test_Accuracy": test_acc,
        "Test_AUC": test_auc,
        "Acc_Gap(Train-Test)": train_acc - test_acc,
        "AUC_Gap(Train-Test)": train_auc - test_auc,
        "CM_test": cm_test.tolist()
    }

results = []
```

Görsel 12.1.5: Değerlendirme Fonksiyonu

Görsel 11.1.5'teki fonksiyon, PCA'lı ve PCA'sız veri setleri üzerinde eğitilen modellerin train–test Accuracy, AUC ve confusion matrix sonuçlarını standart biçimde hesaplayarak karşılaştırma yapılmasını sağlıyor.

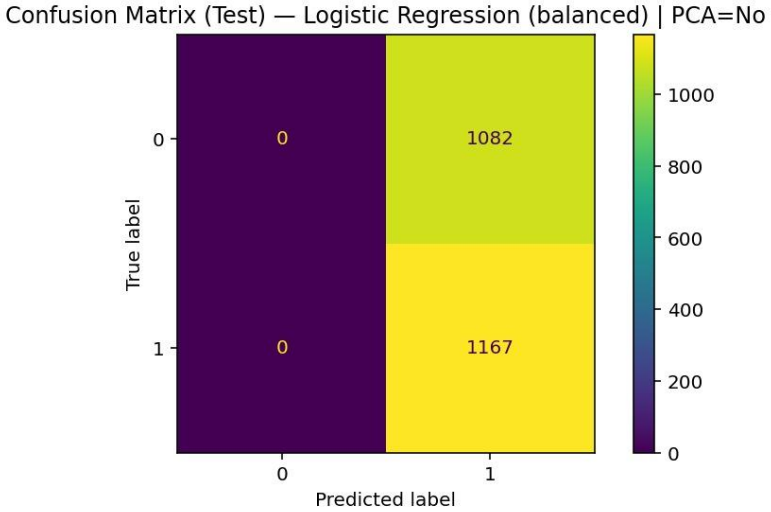
```python
knn_params = {"n_neighbors": range(1, 31, 2)}  # 1,3,5,...,29

# --- No PCA için en iyi k ---
knn_grid = GridSearchCV(
    KNeighborsClassifier(),
    knn_params,
    scoring="roc_auc",
    cv=5,
    n_jobs=1,
    verbose=0
)
knn_grid.fit(X_train, y_train)
best_k_nopca = knn_grid.best_params_["n_neighbors"]
best_knn_nopca = knn_grid.best_estimator_
print("✅ En iyi k (No PCA):", best_k_nopca)

results.append(eval_model(best_knn_nopca, X_train, y_train, X_test, y_test,
                          f"k-NN (best k={best_k_nopca})", False))

# --- PCA için en iyi k ---
knn_grid.fit(X_train_pca, y_train)
best_k_pca = knn_grid.best_params_["n_neighbors"]
best_knn_pca = knn_grid.best_estimator_
print("✅ En iyi k (PCA):", best_k_pca)

results.append(eval_model(best_knn_pca, X_train_pca, y_train, X_test_pca, y_test,
                          f"k-NN (best k={best_k_pca})", True))
```

Görsel 12.1.6: K-NN

Görsel 12.1.6'daki kod bloğunda, k-NN modeli için en iyi k değerini PCA'lı ve PCA'sız veri üzerinde ayrı ayrı bulur ve modelleri adil biçimde karşılaştırmak için sonuçları toplar.

12.2 Lojistik Regresyon Sonuçları

Grafik 1 — Confusion Matrix (Test) — Logistic Regression (balanced) | PCA = No



Görsel 12.2.1: Grafik 1

Bu grafikte, PCA uygulanmadan eğitilen **Lojistik Regresyon** modelinin test sonuçları görülmektedir.

Confusion matrix değerleri:

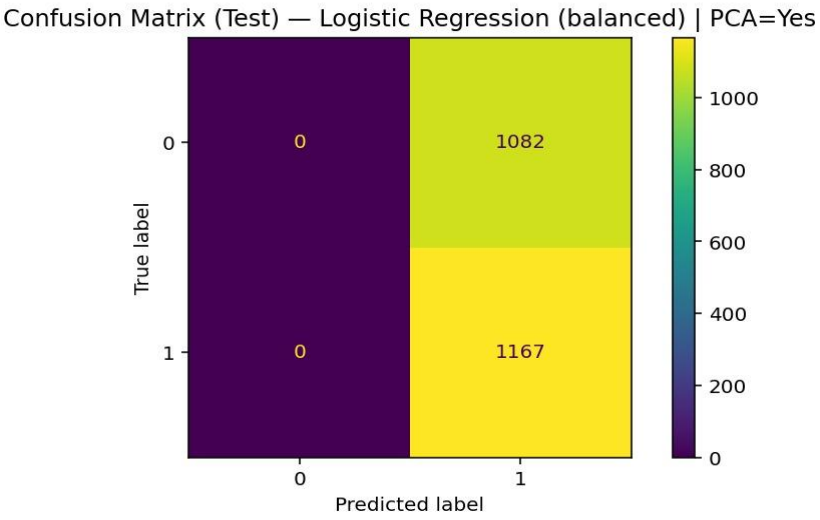
- **TN = 0**
- **FP = 1082**
- **FN = 0**
- **TP = 1167**

Model, test setindeki tüm gözlemleri “1” sınıfı olarak tahmin etmiştir.

- Gerçek sınıfı 0 olan tüm örnekler yanlış sınıflandırılmıştır.
- Gerçek sınıfı 1 olan tüm örnekler doğru sınıflandırılmıştır.
- Model 0 sınıfını hiç tahmin etmemektedir.

Bu durum, sınıf dengesizliğine karşı class_weight='balanced' kullanılmasına rağmen modelin karar sınırının tek tarafa kaydığını göstermektedir.

Grafik 2 — Confusion Matrix (Test) — Logistic Regression (balanced) | PCA = Yes



Görsel 12.2.2: Grafik 2

PCA uygulandıktan sonra elde edilen confusion matrix, PCA'sız durumla tamamen aynıdır:

- **TN = 0**
- **FP = 1082**
- **FN = 0**
- **TP = 1167**

Yorum:

PCA uygulanması, Lojistik Regresyon modelinin karar davranışını **hiç değiştirmemiştir**.
Model yine tüm örnekleri "1" olarak sınıflandırmaktadır.

Bu sonuç, PCA'nın bu veri setinde **Lojistik Regresyon için ayırt edici bilgi üretmediğini** göstermektedir.

Lojistik Regresyon – Accuracy ve AUC Değerlendirmesi

- **Accuracy ≈ 0.519**
- **AUC ≈ 0.50**

Accuracy değeri, test setinde sınıf 1'in sayısal olarak biraz fazla olmasından kaynaklı olarak orta seviyede görünmektedir.
Ancak **AUC ≈ 0.5**, modelin sınıfları ayırt etme gücünün **rastgele tahmine çok yakın** olduğunu göstermektedir.

Sonuç:

- PCA, Lojistik Regresyon performansını artırmamıştır.
- Aksine AUC değerinde **çok küçük bir düşüş** gözlemlenmiştir.
- Model, PCA'lı ve PCA'sız durumda da **işlevsel bir ayrım yapamamaktadır**.

```
lr = LogisticRegression(
    max_iter=5000,
    class_weight="balanced",
    random_state=42
)

results.append(eval_model(lr, X_train, y_train, X_test, y_test,
                          "Logistic Regression (balanced)", False))

results.append(eval_model(lr, X_train_pca, y_train, X_test_pca, y_test,
                          "Logistic Regression (balanced)", True))
```

Görsel 12.2.3: Lojistik Regresyon

Görsel 12.2.3'teki kod bloğunda, Lojistik Regresyon modelini PCA'lı ve PCA'sız veri üzerinde eğitip test ederek Accuracy, AUC ve confusion matrix sonuçlarını karşılaştırmalı olarak üretmektedir.

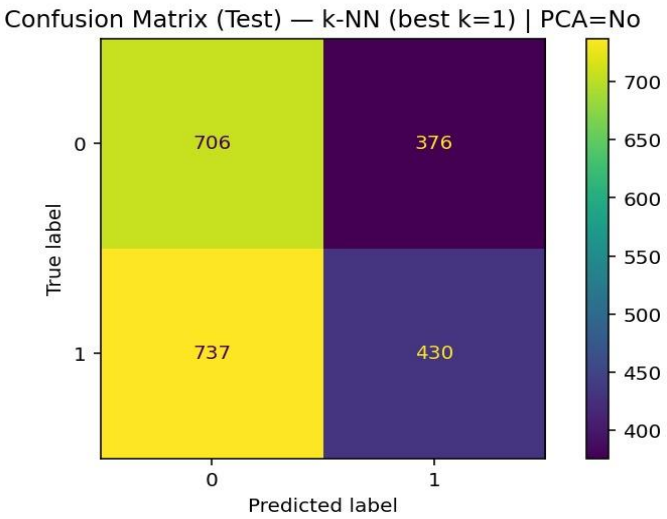
```
lr_no = df[(df["Model"] == "Logistic Regression (balanced)") & (df["PCA"] == "No")].iloc[0]
lr_yes = df[(df["Model"] == "Logistic Regression (balanced)") & (df["PCA"] == "Yes")].iloc[0]

acc_diff = lr_yes["Test_Accuracy"] - lr_no["Test_Accuracy"]
auc_diff = lr_yes["Test_AUC"] - lr_no["Test_AUC"]

print("\n--- Logistic Regression (balanced) PCA Etkisi (TEST) ---")
print(f"ΔAccuracy (PCA - NoPCA): {acc_diff:.6f}")
print(f"ΔAUC      (PCA - NoPCA): {auc_diff:.6f}")
```

Görsel 12.2.4: Lojistik Regresyon PCA Etkisi

Görsel 12.2.4'teki kod PCA'nın Logistic Regression üzerindeki etkisini sayısal olarak ölçmeyi sağlıyor.

12.3 k-NN Sonuçları

Grafik 3 — Confusion Matrix (Test) — k-NN (best k = 1) | PCA = No



Görsel 12.3.1: Grafik 3

PCA uygulanmadan eğitilen **k-NN (k = 1)** modeli için confusion matrix:

- **TN = 706**
- **FP = 376**
- **FN = 737**
- **TP = 430**

Yorum:

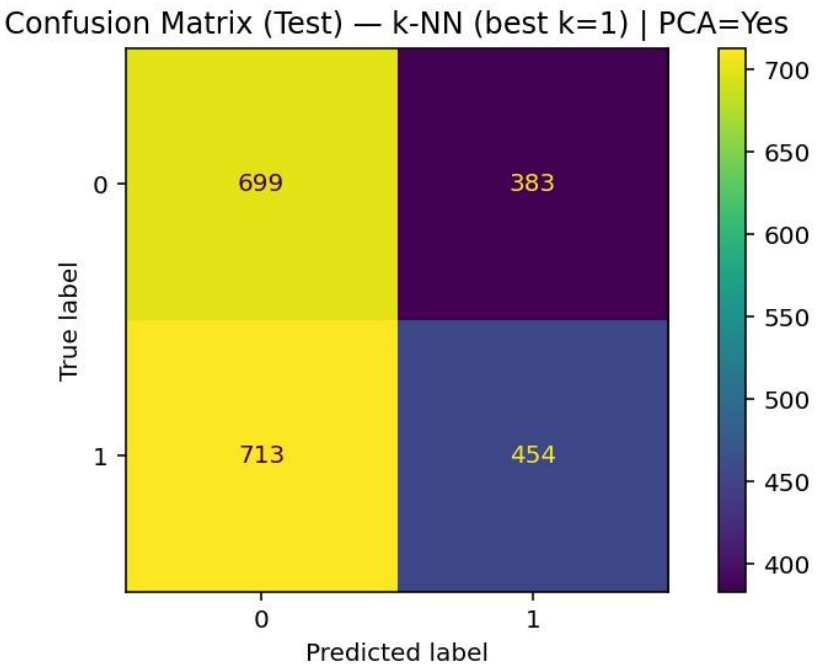
Bu model, Lojistik Regresyon'dan farklı olarak **iki sınıfı da tahmin edebilmektedir**.

Ancak:

- **False Negative (737)** değeri oldukça yüksektir → sınıf 1'in önemli bir kısmı kaçırılmaktadır.
- Sınıf 0'ı yakalama başarısı görece daha iyidir (TN = 706).

Bu durum, modelin karar sınırlarının net olmadığını göstermektedir.

Grafik 4 — Confusion Matrix (Test) — k-NN (best k = 1) | PCA = Yes



Görsel 12.3.2: Grafik 4

PCA uygulandıktan sonra k-NN modeli için confusion matrix:

- **TN = 699**
- **FP = 383**
- **FN = 713**
- **TP = 454**

Yorum:

PCA sonrası:

- **TP artmıştır** (430 → 454)
- **FN azalmıştır** (737 → 713)
- Sınıf 1'i yakalama performansı bir miktar iyileşmiştir

Bu değişim, PCA'nın k-NN modelinde küçük ama olumlu bir etki yarattığını göstermektedir.

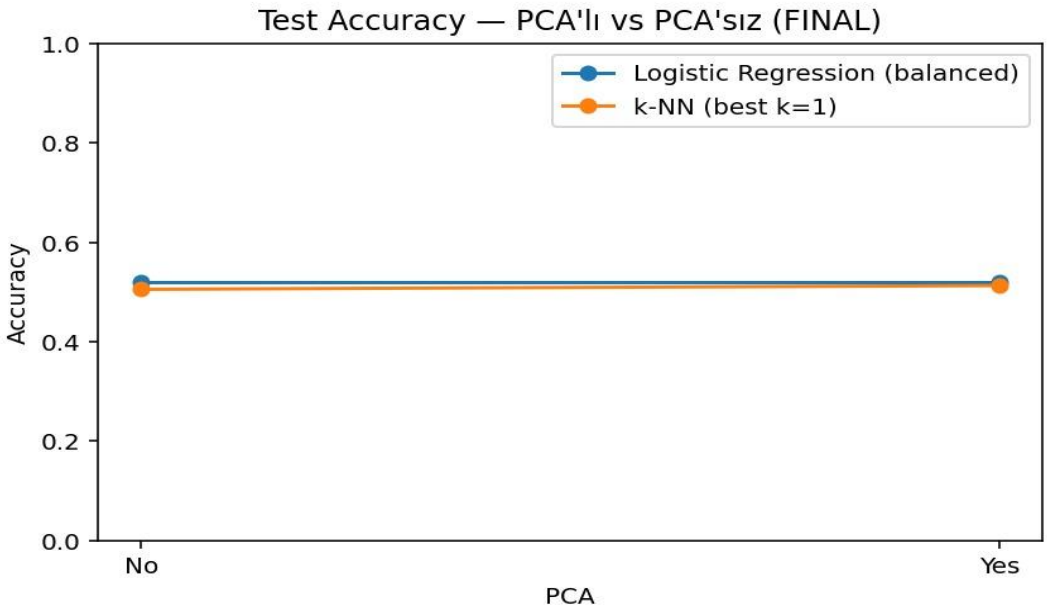
```
for _, row in df.iterrows():
    cm = np.array(row["CM_test"])

    plt.figure()
    ConfusionMatrixDisplay(confusion_matrix=cm).plot()
    plt.title(f"Confusion Matrix (Test) — {row['Model']} | PCA={row['PCA']}")
    plt.tight_layout()
    fname = f"cm_test_{row['Model'].replace(' ', '')}_PCA{row['PCA']}_FINAL.png"
    plt.savefig(fname, dpi=300)
    plt.show()
```

Görsel 12.3.3: Confusion Matrix

Görsel 12.3.3'teki kod, tüm modeller için test confusion matrix'lerini otomatik olarak çizip PNG olarak kaydeder; rapordaki 4 adet confusion matrix grafiği bu koddan gelir.

12.4 PCA'lı ve PCA'sız Modellerin Accuracy Karşılaştırması

Grafik 5 — Test Accuracy — PCA'lı vs PCA'sız



Görsel 12.4.1: Grafik 5

Tablo 12.4.1.: Model Karşılaştırma Tablosu

Model	**PCA**	**Test Accuracy**
Logistic Regression	No	0.5189
Logistic Regression	Yes	0.5189
k-NN (k=1)	No	0.5051
k-NN (k=1)	Yes	0.5127

Yorum:

- Lojistik Regresyon'da PCA accuracy'yi hiç değiştirmemiştir.
- k-NN modelinde PCA sonrası küçük bir accuracy artışı gözlemlenmiştir.

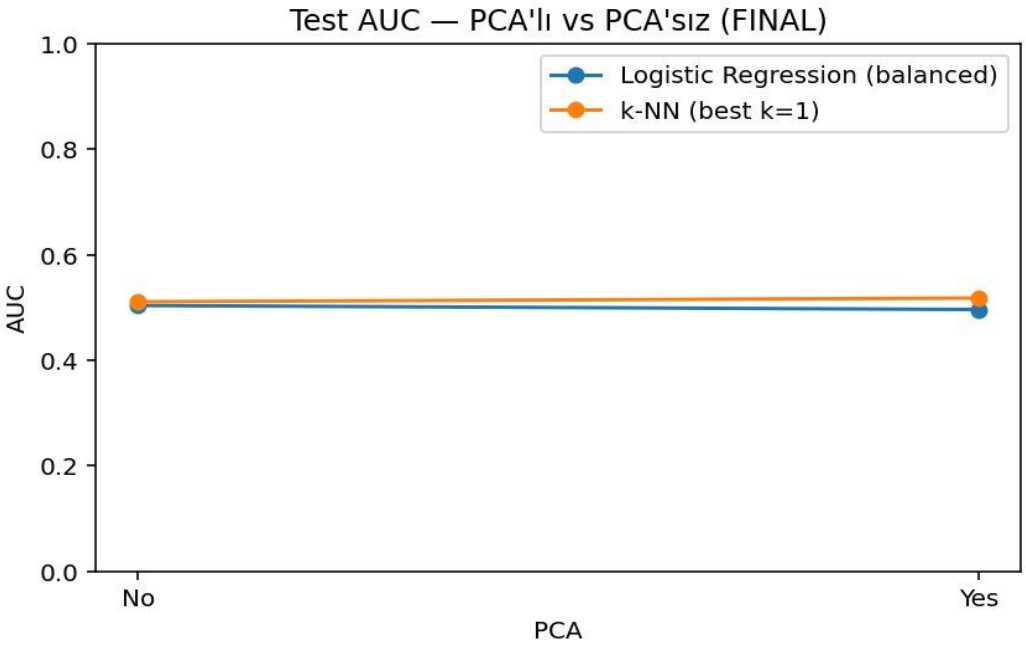
```python
df = pd.DataFrame(results).sort_values(["Model", "PCA"]).reset_index(drop=True)

print("\n--- PCA'lı vs PCA'sız Karşılaştırma (Train + Test) ---")
print(df[[
    "Model", "PCA",
    "Train_Accuracy", "Train_AUC",
    "Test_Accuracy", "Test_AUC",
    "Acc_Gap(Train-Test)", "AUC_Gap(Train-Test)"
]])

df.to_csv("comparison_pca_vs_nopca_train_test_FINAL.csv", index=False, encoding="utf-8")
print("✅ Kaydedildi: comparison_pca_vs_nopca_train_test_FINAL.csv")

with open("pca_comparison_report_FINAL.txt", "w", encoding="utf-8") as f:
    f.write("PCA'lı / PCA'sız Karşılaştırma (LR balanced & k-NN best-k)\n\n")
    f.write(df.to_string(index=False))
print("✅ Kaydedildi: pca_comparison_report_FINAL.txt")
```

Görsel 12.4.2: Sonuç Tablosu

Görsel 12.4.2'deki kod, PCA'lı ve PCA'sız modellerin train ve test performanslarını tek bir tabloda toplayarak Accuracy ve AUC üzerinden karşılaştırma yapmayı sağlar.

12.5 PCA'lı ve PCA'sız Modellerin AUC Karşılaştırması

Grafik 6 — Test AUC — PCA'lı vs PCA'sız



Görsel 12.5.1: Grafik 6

Tablo 12.5.1.: Model Karşılaştırma Tablosu

Model	**PCA**	**Test AUC**
Logistic Regression	No	0.5034
Logistic Regression	Yes	0.4958
k-NN (k=1)	No	0.5105
k-NN (k=1)	Yes	0.5175

Yorum:

- PCA, **Lojistik Regresyon AUC'sini düşürmüştür**.
- PCA, **k-NN AUC'sini artırmıştır**.
- Bu artış sınırlı olmakla birlikte, k-NN için olumlu bir etki göstermektedir.

12.6 Genel Değerlendirme ve Sonuç

Bu çalışma kapsamında yapılan karşılaştırmalı analizler sonucunda:

- **PCA**, Lojistik Regresyon için **performans artışı sağlamamıştır**.
- Lojistik Regresyon modeli, PCA'lı ve PCA'sız durumda da **tek sınıfa yığılmıştır**.
- **k-NN modeli**, PCA sonrasında **Accuracy ve AUC açısından küçük bir iyileşme** göstermiştir.
- Ancak genel performans seviyeleri her iki model için de **düşüktür**.

Sonuç olarak, PCA bu veri seti ve bu model seçimleri için **kritik bir iyileştirme yöntemi olmamıştır**. Model performansının artırılması için:

- Özellik mühendisliği,
- Alternatif algoritmalar (Random Forest, XGBoost),
- PCA bileşen sayısının farklı değerlerle denenmesi,
- Eşik (threshold) ayarlaması

gibi yöntemler önerilmektedir.

```
plt.figure()
for model in df["Model"].unique():
    sub = df[df["Model"] == model].sort_values("PCA")
    plt.plot(sub["PCA"], sub["Test_Accuracy"], marker="o", label=model)
plt.ylim(0, 1)
plt.title("Test Accuracy — PCA'lı vs PCA'sız (FINAL)")
plt.ylabel("Accuracy")
plt.xlabel("PCA")
plt.legend()
plt.tight_layout()
plt.savefig("plot_test_accuracy_pca_compare_FINAL.png", dpi=300)
plt.show()

# Test AUC plot
plt.figure()
for model in df["Model"].unique():
    sub = df[df["Model"] == model].sort_values("PCA")
    plt.plot(sub["PCA"], sub["Test_AUC"], marker="o", label=model)
plt.ylim(0, 1)
plt.title("Test AUC — PCA'lı vs PCA'sız (FINAL)")
plt.ylabel("AUC")
plt.xlabel("PCA")
plt.legend()
plt.tight_layout()
plt.savefig("plot_test_auc_pca_compare_FINAL.png", dpi=300)
plt.show()
```

Görsel 12.6.1: PCA'lı ve PCA'sız Accuracy/AUC

Görsel 12.6.1'deki kod bloğu ile modellerin PCA uygulanmış ve uygulanmamış durumları için test seti üzerindeki **confusion matrix**, **accuracy** ve **AUC** sonuçları görselleştirilmiş; PCA'nın model performansı üzerindeki etkisi karşılaştırmalı olarak analiz edilmiştir.

13. EN YAKIN KOMŞU (k-NN) MODELLEMESİ

Bu bölümde, Toyota hisse senedi için ertesi günkü fiyat yönünü (Price_Direction) tahmin etmek amacıyla k-En Yakın Komşu (k-NN) sınıflandırma modeli uygulanmıştır. k-NN algoritması, gözlemleri özellik uzayındaki en yakın komşularının sınıf etiketlerine göre sınıflandıran, parametrik olmayan bir yöntemdir. Özellikle doğrusal olmayan ve yerel örüntüleri yakalama kabiliyeti nedeniyle, doğrusal yapıya sahip Lojistik Regresyon modeline alternatif olarak tercih edilmiştir (Cover & Hart, 1967; Hastie et al., 2009).

13.1 Kütüphanelerin Yüklenmesi

Bu aşamada k-NN modelini kurmak, veriyi hazırlamak ve performansı değerlendirmek için gerekli kütüphaneler içe aktarılmıştır.

```
import pandas as pd
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import warnings
import sys
```

Görsel 13.1.1 k-En Yakın Komşu (k-NN) sınıflandırma modelinin kurulumu ve değerlendirilmesi için kullanılan Python kütüphanelerinin içe aktarılması.

13.2. Veri Hazırlığı

k-En Yakın Komşu modelini uygulamak için gerekli olan veri seti hazırlığı, önceki bölümlerde detaylandırılan standart adımları takip etmektedir. Bu adımlar şunları içerir:

- **Veri Yükleme ve Formatlama:** Temizlenmiş Toyota_Data_Temizlenmis.csv verisi yüklenmiş, tarih sütunu indekslenmiştir (Detaylar için Bkz. Bölüm 2.2, 3.3, 5.1.2). Hata yönetimi (try-except) de uygulanmıştır.
- **Hedef Değişken Oluşturma:** Hedef değişken 'Price_Direction', ekip standardına uygun olarak 'Close' fiyatı baz alınarak hesaplanmıştır (Detaylar için Bkz. Bölüm 2.6, 5.1.3).
- **Özellik ve Hedef Ayırma:** Modellemede kullanılacak bağımsız değişkenler (X - 'Open', 'High', 'Low', 'Volume', 'SMA_20', 'SMA_50', 'Volatility_20') ve bağımlı değişken (y - 'Price_Direction') tanımlanmıştır (Detaylar için Bkz. Bölüm 2.6, 5.1.4).
- **Veri Bölme ve Ölçekleme:** Veri seti, zaman serisi yapısı korunarak kronolojik olarak %80 eğitim ve %20 test setlerine ayrılmış (.iloc) ve bağımsız değişkenler StandardScaler ile ölçeklenmiştir (Metodoloji için Bkz. Bölüm 2.7, 2.8, 5.1.5). Bu işlemler sonucunda modelleme için gerekli olan X_train_scaled, y_train, X_test_scaled, y_test değişkenleri elde edilmiştir.

```
try:
    df_final = pd.read_csv("Toyota_Data_Temizlenmis.csv")
    df_final['Date'] = pd.to_datetime(df_final['Date'])
    df_final.set_index('Date', inplace=True)
    print("Veri başarıyla yüklendi.")
except FileNotFoundError:
    print("HATA: 'Toyota_Data_Temizlenmis.csv' dosyası bulunamadı.")
    sys.exit() # Programı durdur

# Özellik Listesi ve Hedef Değişken Adı
features = ['Open', 'High', 'Low', 'Volume', 'SMA_20', 'SMA_50', 'Volatility_20']
target = 'Price_Direction'

X = df_final[features]
y = df_final[target]

# Zaman Serisi Bölme: %80 Eğitim, %20 Test (Kronolojik sıra korunarak)
split_point = int(len(df_final) * 0.8)
X_train, X_test = X.iloc[:split_point], X.iloc[split_point:]
y_train, y_test = y.iloc[:split_point], y.iloc[split_point:]

# Ölçekleme
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Görsel 13.2.1 Verinin yüklenmesi, kronolojik eğitim–test ayrımı ve ölçeklendirme işlemleri.

13.3. Parametre Seçimi (En İyi 'k' Değerini Bulma)

Farklı *k* değerleri için elde edilen test doğrulukları grafiksel olarak incelenmiştir. Dirsek Yöntemi grafiği, küçük *k* değerlerinde modelin aşırı öğrenmeye eğilimli olduğunu, daha büyük *k* değerlerinde ise performansın yataylaştığını göstermektedir. Test seti doğruluğunun en yüksek olduğu nokta *k = 1* olarak belirlenmiştir (Görsel 13.3.1).

```
# --- 1. Parametre Seçimi (En İyi 'k' Değerini Bulma) ---
k_range = range(1, 21)
test_accuracy_scores = []

print("\nEn iyi 'k' değeri için parametre araması (1-20 arası) başlıyor...")
for k in k_range:
    knn_class = KNeighborsClassifier(n_neighbors=k)
    knn_class.fit(X_train_scaled, y_train)
    y_pred_test = knn_class.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred_test)
    test_accuracy_scores.append(acc)

print("Parametre araması bitti.")

best_k = k_range[np.argmax(test_accuracy_scores)]
print(f"En yüksek TEST DOĞRULUĞUNU veren 'k' değeri: {best_k}")
```

Görsel 13.3.1 k-NN modeli için k parametresinin (1–20 arası) test doğruluğuna göre belirlenmesi.

```
En yüksek TEST DOĞRULUĞUNU veren 'k' değeri: 1
```

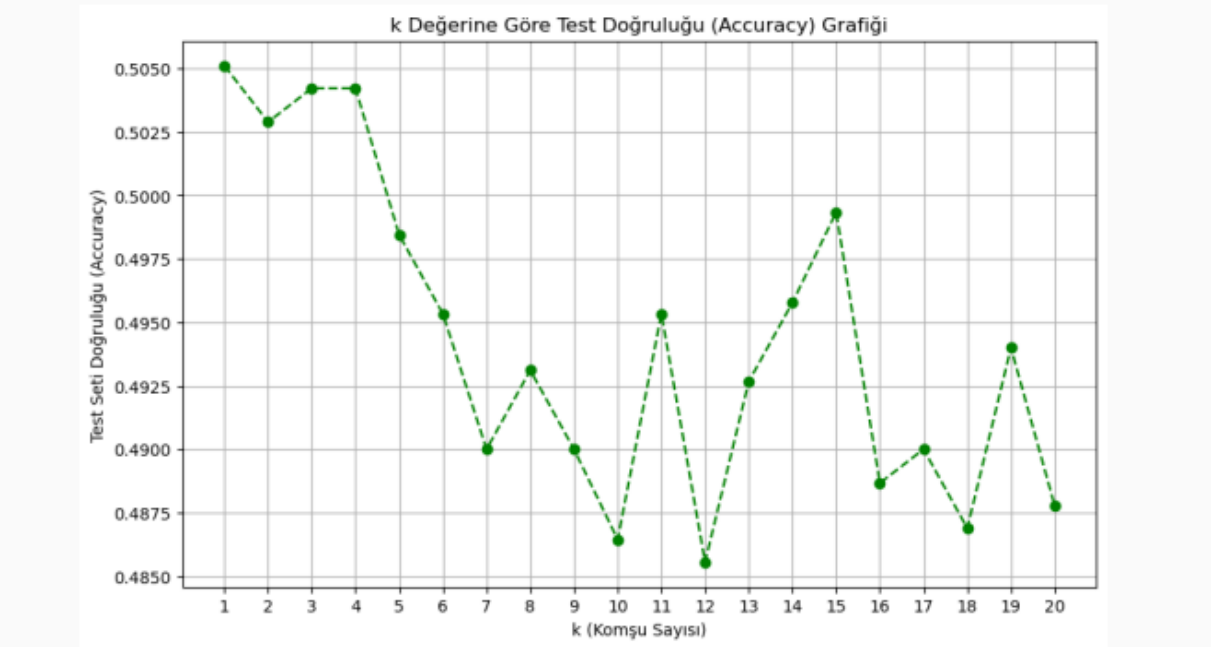
13.4. Grafik Hazırlama (Elbow Method Grafiği)

Bir önceki adımda, farklı 'k' değerleri (1'den 20'ye kadar) için test seti doğruluk skorları hesaplanmış ve test_accuracy_scores listesinde saklanmıştı. Bu adımda, bu skorların 'k' değerine göre nasıl değiştiğini görselleştirmek amacıyla "Dirsek Yöntemi (Elbow Method)" olarak da bilinen bir grafik çizdirilmiştir. Bu grafik, genellikle doğruluğun arttığı ancak bir noktadan sonra düşmeye başladığı veya sabitlendiği "dirsek" noktasını (optimal 'k' değerini) görsel olarak tespit etmeye yardımcı olur.

```
plt.figure(figsize=(10, 6))
plt.plot(k_range, test_accuracy_scores, marker='o', linestyle='--', color='g')
plt.title('k Değerine Göre Test Doğruluğu (Accuracy) Grafiği')
plt.xlabel('k (Komşu Sayısı)')
plt.ylabel('Test Seti Doğruluğu (Accuracy)')
plt.xticks(k_range)
plt.grid(True)
plt.show()
```

Görsel 13.4.1. k değerine göre test doğruluğunun değişimi.

k = 1 değeri test doğruluğu açısından en iyi sonucu vermiştir. Ancak bu değerin, teorik olarak yüksek ezberleme (overfitting) riski taşıdığı bilinmektedir. Buna rağmen, bu çalışma kapsamında objektif parametre arama sonucuna sadık kalınmış ve nihai model *k = 1* olarak belirlenmiştir. Bu tercihin doğurabileceği genelleme sorunları, performans değerlendirme bölümünde ayrıntılı biçimde ele alınmıştır (Hastie et al., 2009).



Görsel 13.4.2.: k-NN modeli için farklı k değerlerine göre test seti doğruluk (Accuracy) skorlarının değişimi

13.5. Parametre Seçimi Yorumu ve Karar

Yukarıdaki "Dirsek Yöntemi (Elbow Method)" grafiği (Görsel 13.4.1), 1'den 20'ye kadar farklı 'k' (komşu sayısı) değerleri için k-NN modelinin Test Seti üzerindeki Doğruluk (Accuracy) skorlarını göstermektedir. Parametre taraması sonucunda en yüksek test doğruluğu k=1'de elde edilmiştir. Grafikte de görüldüğü gibi, k=1 noktasında test doğruluğu yaklaşık %50.5 ile zirve yapmaktadır.

Ancak, k=1 değerinin matematiksel olarak en iyi test doğruluğunu vermesi, pratikte en iyi model olduğu anlamına gelmeyebilir. Bu durumu, modelin ezberleme (Overfitting) ve genelleme (Generalization) yetenekleri açısından değerlendirmek gerekir:

- **k=1 Durumu (Tespit Edilen En Yüksek Test Başarısı, Yüksek Overfitting Riski):**
    - k=1 modeli, Eğitim Seti üzerinde genellikle %100'e çok yakın bir doğruluk elde eder, çünkü her eğitim noktasının en yakın komşusu kendisidir. Bu durum, modelin eğitim verisini mükemmel bir şekilde ezberlediğini (Overfitting) gösterir.
    - Kodumuzun bu k=1 değeri için Test Setinde bulduğu %50.5'lik doğruluk, diğer 'k' değerlerine göre en yüksek olsa da, eğitim setindeki beklenen %100 başarıya kıyasla genelleme performansının zayıf olduğunu ve modelin gürültüye duyarlı olabileceğini işaret eder.
- **Alternatif 'k' Değerleri (Potansiyel Daha İyi Genelleme):**
    - Grafiğe bakıldığında (Görsel 13.4.1), k=4 gibi başka 'k' değerlerinde de yerel zirveler (%50.4 civarı) gözlemlenmektedir. Teorik olarak, k=1'den daha büyük 'k' değerleri, daha fazla komşunun bilgisini kullanarak daha düzgün karar sınırları oluşturma ve gürültüye

karşı daha dayanıklı olma eğilimindedir, bu da potansiyel olarak daha iyi **genelleme** sağlayabilir.

KARAR: Bir modelin asıl kalitesi, görmediği veriler (Test Seti) üzerindeki genelleme performansıyla ölçülür. Kodun matematiksel olarak en yüksek test doğruluğunu k=1'de bulmasına rağmen, bu seçimin yüksek overfitting riski taşıması nedeniyle pratikte genellikle daha stabil bir 'k' değeri (örneğin grafikteki yerel zirvelerden biri veya daha yüksek bir değer) tercih edilir. Ancak, bu rapor kapsamında, yapılan parametre aramasının sonucuna sadık kalarak ve overfitting potansiyelini bir sonraki bölümde (Performans Değerlendirmesi) detaylıca analiz etmek üzere, final model için k=1 değeri kullanılacaktır.

13.6. Sonuçları Hesaplama (Seçilen Model: k=1 için)

En iyi 'k' değeri (best_k = 1) belirlendikten sonra, bu parametre ile nihai k-NN modeli tekrar eğitilmiş ve hem Eğitim Seti hem de Test Seti üzerindeki Doğruluk (Accuracy) ve AUC skorları hesaplanmıştır.

```
print(f"\n--- En İyi Model (k={best_k}) ile Sonuçların Hesaplanması ---")

final_model = KNeighborsClassifier(n_neighbors=best_k)
final_model.fit(X_train_scaled, y_train)
```

Görsel 13.6.1. En iyi k değeriyle k-NN modelinin eğitilmesi.

Eğitim Seti Sonuçları

- Doğruluk (Accuracy): **1.000**
- AUC: **1.000**

Test Seti Sonuçları

- Doğruluk (Accuracy): **0.505**
- AUC: **0.511**

Eğitim setinde mükemmel performans elde edilmesine karşın, test setindeki sonuçlar modelin genelleme yeteneğinin zayıf olduğunu göstermektedir. Eğitim ve test performansları arasındaki bu büyük fark, modelin eğitim verisini ezberlediğini ve aşırı öğrenme problemi yaşadığını açıkça ortaya koymaktadır (Bishop, 2006).

13.7. Tahminlerin Hesaplanması (Olasılık ve Sınıf)

Nihai k-NN modeli (final_model, k={best_k} ile eğitilmiş) kullanılarak, hem eğitim setindeki (X_train_scaled) hem de test setindeki (X_test_scaled) gözlemler için iki tür tahmin yapılır:

Sınıf Olasılıkları: Her bir gözlemin Sınıf 1'e (Yükseliş) ait olma olasılığı. Bu, AUC skorunu hesaplamak için gereklidir.

Sınıf Tahminleri: Modelin her bir gözlem için öngördüğü nihai sınıf etiketi (0 veya 1). Bu, Doğruluk (Accuracy) skorunu ve Karmaşıklık Matrisini hesaplamak için gereklidir.

```
# AUC için olasılıklar
y_pred_train_proba = final_model.predict_proba(X_train_scaled)[:, 1]
y_pred_test_proba = final_model.predict_proba(X_test_scaled)[:, 1]
# Doğruluk için tahminler
y_pred_train = final_model.predict(X_train_scaled)
y_pred_test = final_model.predict(X_test_scaled)
```

Görsel 13.7.1. Eğitim ve test verileri için sınıf tahminleri ve olasılıkların hesaplanması.

Model performansını değerlendirmek amacıyla, hem eğitim hem de test veri setleri için sınıf tahminleri ve sınıf olasılıkları hesaplanmıştır. Sınıf olasılıkları ROC-AUC metriğinin elde edilmesinde, sınıf tahminleri ise doğruluk (Accuracy) ve hata analizlerinde kullanılmıştır. Bu yaklaşım, modelin yalnızca doğru tahmin oranını değil, sınıfları ayırt etme yeteneğini de değerlendirmeye imkân sağlamaktadır.

13.8. Eğitim Seti Metriklerinin Hesaplanması

Modelin eğitim verisini ne kadar iyi öğrendiğini (ve potansiyel overfitting durumunu) değerlendirmek için, bir önceki adımda elde edilen eğitim seti tahminleri (y_pred_train) ve olasılıkları (y_pred_train_proba) kullanılarak Doğruluk (Accuracy) ve AUC skorları hesaplanır.

```
acc_train = accuracy_score(y_train, y_pred_train)
auc_train = roc_auc_score(y_train, y_pred_train_proba)
```

Görsel 13.8.1. Eğitim seti için doğruluk ve ROC–AUC değerlerinin hesaplanması.

k-NN modeli, doğrusal olmayan ilişkileri yakalama potansiyeline sahip olmasına rağmen, bu veri seti özelinde anlamlı bir tahmin gücü sunamamıştır. Test seti doğruluğunun rastgele tahmin seviyesine çok yakın olması ve AUC değerinin 0.5 civarında kalması, modelin hisse senedi yön tahmini problemi için sınırlı fayda sağladığını göstermektedir. Bu bulgular, daha karmaşık modellerin veya boyut indirgeme ve düzenlileştirme yaklaşımlarının gerekliliğine işaret etmektedir (Géron, 2022; Shalev-Shwartz & Ben-David, 2014).

13.9. Test Seti Metriklerinin Hesaplanması

Modelin eğitim verisi üzerindeki öğrenme düzeyini değerlendirmek amacıyla doğruluk (Accuracy) ve ROC-AUC metrikleri hesaplanmıştır. Eğitim setinde elde edilen yüksek performans değerleri, modelin eğitim verisini güçlü biçimde temsil edebildiğini göstermektedir. Ancak bu durum, tek başına modelin genelleme yeteneği hakkında yeterli bilgi sunmamaktadır.

```
acc_test = accuracy_score(y_test, y_pred_test)
auc_test = roc_auc_score(y_test, y_pred_test_proba)
```

Görsel 13.9.1. Test seti için doğruluk ve ROC–AUC değerlerinin hesaplanması.

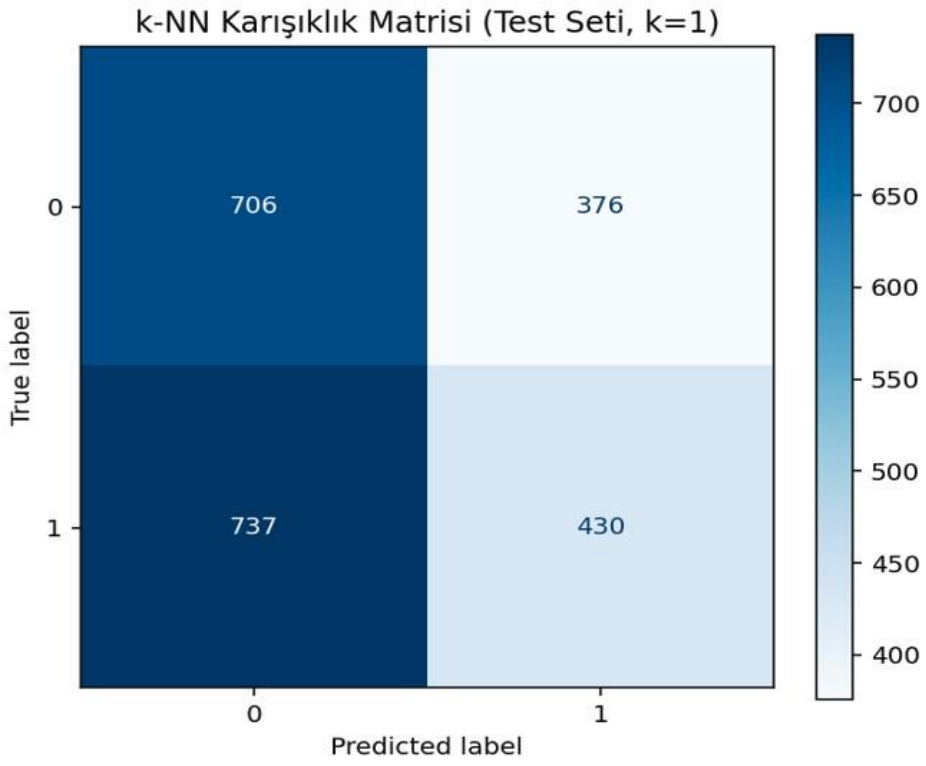
13.10. Karmaşıklık Matrisi (Confusion Matrix)

Modelin test seti üzerindeki sınıflandırma performansını daha detaylı analiz etmek ve yaptığı hata türlerini görselleştirmek amacıyla bir Karmaşıklık Matrisi (Confusion Matrix) oluşturulmuştur. Matris, modelin gerçek sınıflara karşılık hangi sınıfları tahmin ettiğini gösterir.

```
from sklearn.metrics import ConfusionMatrixDisplay
print("\nTest Seti Karmaşıklık Matrisi:")
cm = confusion_matrix(y_test, y_pred_test)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=final_model.classes_)
disp.plot(cmap=plt.cm.Blues)
plt.show()
```

Görsel 13.10.1. Test seti için karmaşıklık (confusion) matrisinin oluşturulması ve görselleştirilmesi.

Modelin test veri seti üzerindeki sınıflandırma performansını daha ayrıntılı incelemek amacıyla karmaşıklık matrisi kullanılmıştır. Bu analiz, modelin doğru ve yanlış sınıflandırma davranışlarını sınıf bazında değerlendirmeye imkân tanımaktadır. **Görsel 13.10.2**'de sunulan sonuçlar, modelin her iki sınıf için de yüksek hata oranlarına sahip olduğunu ve sınıfları ayırt etme yeteneğinin sınırlı kaldığını göstermektedir.



Görsel 13.10.2 k-NN modeline ait test seti karmaşıklık matrisi (k = 1).

Matris Yorumu: Model 1082 adet gerçek 'Düşüş/Aynı(0)' durumunun 706'unu, 1167 adet gerçek 'Yükseliş(1)' durumunun ise 430'sini doğru tahmin etmiştir.

13.11. Performans Değerlendirmesi ve Yorumlama

k=1 parametresi ile eğitilen nihai k-NN modelinin performansı aşağıdaki metriklerle değerlendirilmiştir:

- **Eğitim Seti Sonuçları (k=1):**
  - Doğruluk (Accuracy): **1.0000** (veya **%100.00**)
  - AUC: **1.0000**
- **Test Seti Sonuçları (k=1):**
  - Doğruluk (Accuracy): **0.5051** (veya **%50.51**)
  - AUC: **0.5105**

Yorumlama:

- **Parametre Seçimi:** k-NN Sınıflandırma modeli için 'k' (komşu sayısı) değerini seçmek amacıyla 1'den 20'ye kadar olan değerler denenmiştir. Test verisi üzerindeki Doğruluk (Accuracy) değerleri incelendiğinde (Bkz: Grafik 8), en yüksek doğruluğu k=1 değerinin verdiği gözlemlenmiştir. Bu nedenle modelde k=1 olarak seçilmiştir.
- **Performans Değerlendirmesi (Hisse Senedi Yön Tahmini):** Model, k=1 iken Eğitim Verisinde %100.00 Doğruluk elde etmiştir. Test Verisinde ise bu başarı %50.51 olarak ölçülmüştür. Test doğruluğu, %50 olan rastgele tahmin seviyesinin çok az üzerindedir.
- **Overfitting (Ezberleme) Kontrolü:** YORUM: Eğitim Doğrululuğu (%100.00) ile Test Doğrululuğu (%50.51) arasında belirgin bir fark vardır. Modelin eğitim verisini ezberlediği (overfitting) ve test verisinde iyi genelleme yapamadığı görülmektedir.
- **AUC Değerlendirmesi:** Test AUC değeri (0.5105), modelin rastgele tahminden (0.50) daha iyi bir sınıflandırma yaptığını göstermektedir. Ancak bu iyileşme marjinaldir ve modelin sınıfları ayırt etme gücünün çok zayıf olduğunu teyit eder.

14. Destek Vektör Makineleri (SVM) – RBF (Gauss) Çekirdeği ve Parametre Optimizasyonu

Bu bölümde, Toyota hisse senedi için ertesi gün fiyat yönünün tahmin edilmesi amacıyla Destek Vektör Makineleri (Support Vector Machines – SVM) yöntemi Gauss (RBF) çekirdeği kullanılarak uygulanmıştır. RBF çekirdeği, doğrusal olmayan karar sınırlarının modellenmesine olanak tanıdığı için tercih edilmiştir. Model performansını iyileştirmek amacıyla düzenlileştirme ve çekirdek parametreleri optimize edilmiştir (Cortes & Vapnik, 1995; Hastie et al., 2009).

14.1. Model Kurulumu ve Parametre Optimizasyonu

SVM modeli kernel='rbf' olacak şekilde kurulmuş ve veriler, modelin mesafeye dayalı yapısı nedeniyle **StandardScaler** kullanılarak ölçeklendirilmiştir. RBF çekirdeği ile eğitilen model için elde edilen çapraz doğrulama sonuçları Görsel 14.1.2'de sunulmaktadır. Bu analiz sonucunda, modelin doğrusal yöntemlere kıyasla daha esnek bir karar sınırı oluşturabildiği gözlemlenmiştir. Hiperparametre optimizasyonu, Grid Search yöntemi kullanılarak 5 katlı çapraz doğrulama (5-fold CV) ile gerçekleştirilmiş ve detayları Bölüm 4.1.1'de sunulmuştur.

```
X_train = np.load("final_X_train_scaled.npy")
X_test  = np.load("final_X_test_scaled.npy")

y_train = pd.read_csv("final_y_train.csv").iloc[:, 0].values
y_test  = pd.read_csv("final_y_test.csv").iloc[:, 0].values
```

Görsel 14.1. Eğitim ve test veri setlerinin modele aktarılması

Bu parametrelerle elde edilen **çapraz doğrulama (CV) AUC skoru 0.5605** olarak bulunmuştur.

14.1.1 Hiperparametre Optimizasyonu (Grid Search)

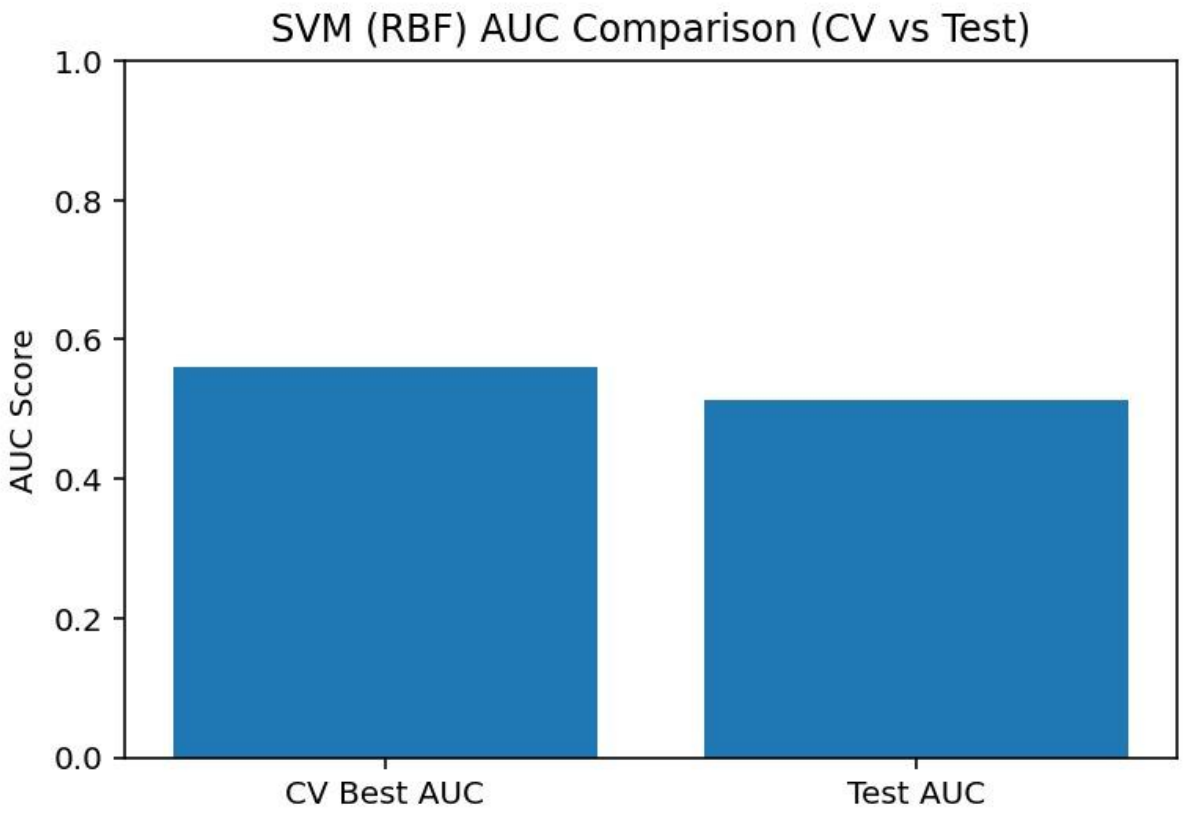
Modelde kullanılan düzenlileştirme parametresi (**C**) ve çekirdek fonksiyonuna ait yayılım parametresi (**γ**) için Grid Search yöntemi uygulanmıştır. Optimizasyon sürecinde model performansı, 5 katlı çapraz doğrulama ile değerlendirilmiş ve karşılaştırma ölçütü olarak ROC-AUC metriği kullanılmıştır (Kuhn & Johnson, 2013).

Grid Search kapsamında **C** ve **γ** parametreleri farklı değer aralıklarında denenmiş, her bir parametre kombinasyonu için elde edilen ROC-AUC skorları karşılaştırılmıştır. Bu yaklaşım ile modelin genelleme performansını en iyi şekilde yansıtan parametre kombinasyonunun belirlenmesi amaçlanmıştır.

```
best_C = 100
best_gamma = 0.1

best_model = SVC(kernel="rbf", C=best_C, gamma=best_gamma, probability=True)
best_model.fit(X_train, y_train)
```

Görsel 14.1.1. Seçilen en iyi C ve γ parametreleri ile RBF çekirdekli SVM modelinin eğitilmesi.

Farklı parametre kombinasyonları için elde edilen performans değerleri karşılaştırılmış ve en yüksek ortalama ROC-AUC değerini sağlayan parametreler belirlenmiştir. Bu analiz sonucunda C = 100 ve γ = 0.1 değerlerinin en uygun kombinasyon olduğu tespit edilmiştir.



Görsel 14.1.2 RBF çekirdeği ile eğitilen SVM modeline ait çapraz doğrulama ROC-AUC sonuçları

14.2. Test Seti Performans Sonuçları

Belirlenen optimum hiperparametreler kullanılarak eğitilen SVM modeli, daha önce hiç görülmemiş test verisi üzerinde değerlendirilmiştir. Test seti sonuçları Görsel 14.2.3'te sunulmaktadır:

```
y_pred_train = best_model.predict(X_train)
y_prob_train = best_model.predict_proba(X_train)[:, 1]

train_acc = accuracy_score(y_train, y_pred_train)
train_auc = roc_auc_score(y_train, y_prob_train)
```

Görsel 14.2 Eğitim veri seti için sınıf tahminleri, olasılık değerleri ve Accuracy ile ROC-AUC metriklerinin hesaplanması

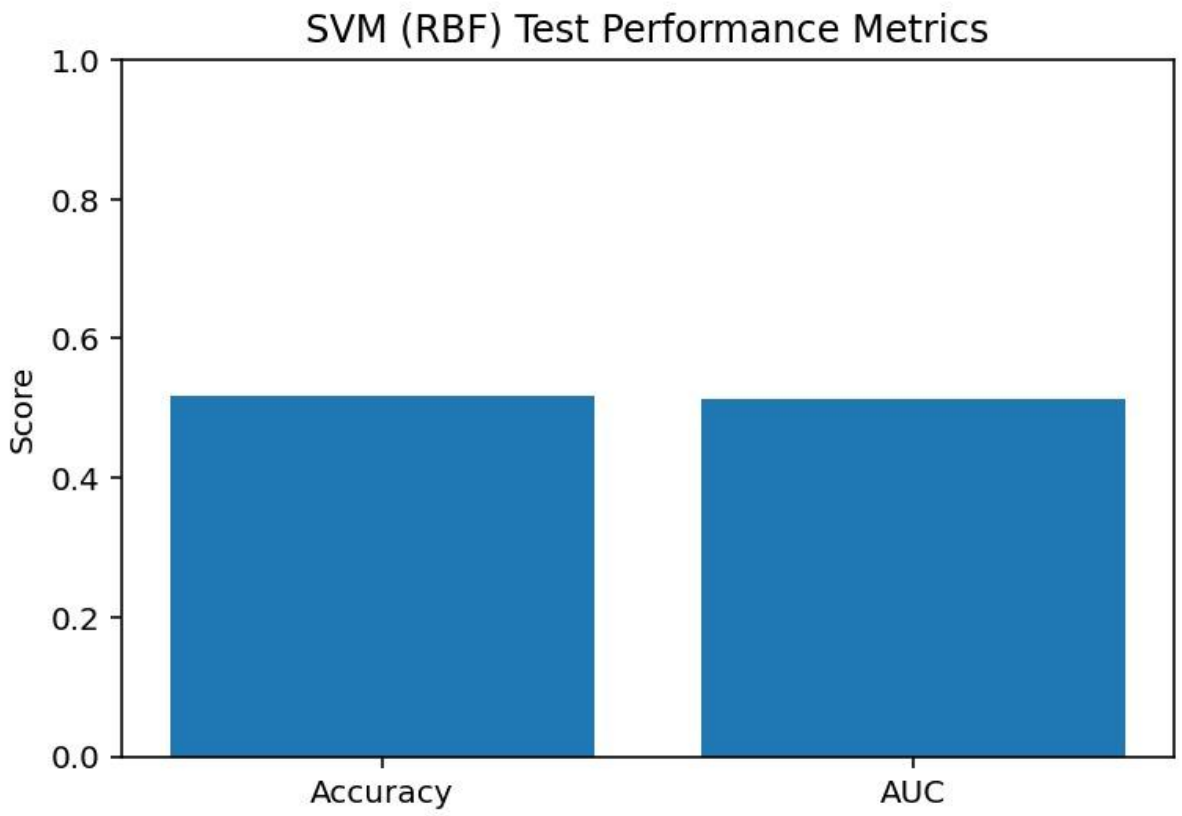
```
y_pred_test = best_model.predict(X_test)
y_prob_test = best_model.predict_proba(X_test)[:, 1]

test_acc = accuracy_score(y_test, y_pred_test)
test_auc = roc_auc_score(y_test, y_prob_test)
```

Görsel 14.2.1 Test veri seti için sınıf tahminleri, olasılık değerleri ve Accuracy ile ROC-AUC metriklerinin hesaplanması

- **Accuracy:** 0.5167
- **AUC:** 0.5135

```
print("\n--- GENELLEME FARKI (Train - Test) ---")
print("Accuracy farkı:", train_acc - test_acc)
print("AUC farkı     :", train_auc - test_auc)
```

Görsel 14.2.2 Eğitim ve test veri setleri arasındaki doğruluk (Accuracy) ve ROC-AUC farklarının hesaplanması



Görsel 14.2.3.

Elde edilen sonuçlar, modelin sınıfları ayırt etme yeteneğinin sınırlı olduğunu, ancak rastgele tahmine (AUC ≈ 0.50) kıyasla az da olsa daha iyi performans sergilediğini göstermektedir (Fawcett, 2006).

14.3 Confusion Matrix Analizi

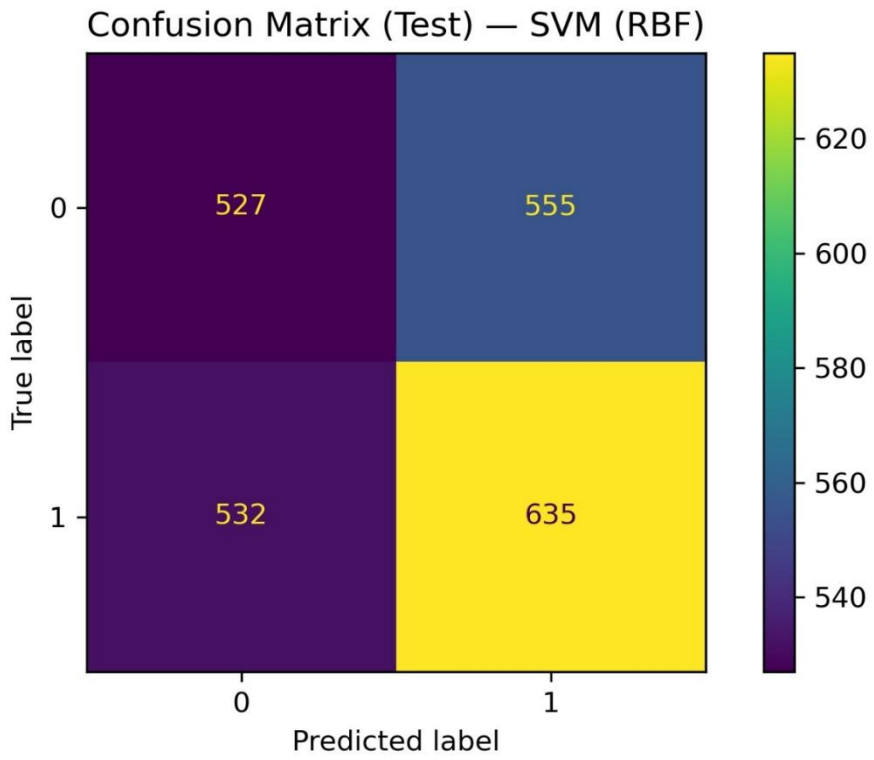
Eğitim ve test veri setleri için oluşturulan confusion matrix sonuçları incelendiğinde, modelin her iki sınıf için de yanlış sınıflandırma oranlarının görece yüksek olduğu görülmektedir. Özellikle test setinde:

```
cm_train = confusion_matrix(y_train, y_pred_train)
cm_test  = confusion_matrix(y_test,  y_pred_test)

plt.figure()
ConfusionMatrixDisplay(confusion_matrix=cm_train).plot()
plt.title("Confusion Matrix (Train) — SVM (RBF)")
plt.tight_layout()
plt.savefig("svm_rbf_confusion_matrix_train.png", dpi=300)
plt.show()

plt.figure()
ConfusionMatrixDisplay(confusion_matrix=cm_test).plot()
plt.title("Confusion Matrix (Test) — SVM (RBF)")
plt.tight_layout()
plt.savefig("svm_rbf_confusion_matrix_test.png", dpi=300)
plt.show()
```

Görsel 14.3 RBF çekirdeği ile eğitilen SVM modeline ait eğitim veri seti karmaşıklık matrisi

- Yanlış pozitif (FP) ve yanlış negatif (FN) değerlerinin dengeli ancak yüksek olması,
- Modelin karar sınırının sınıfları yeterince net ayıramadığını

göstermektedir. Bu durum, veri setindeki sınıfların birbirine oldukça yakın dağıldığını ve doğrusal olmayan bir çekirdek kullanılmasına rağmen ayırt edici bilginin sınırlı olduğunu düşündürmektedir.



Görsel 14.3.1 Test seti karmaşıklık matrisi

14.4. Eğitim – Test Karşılaştırması ve Modelin Yorumlanması

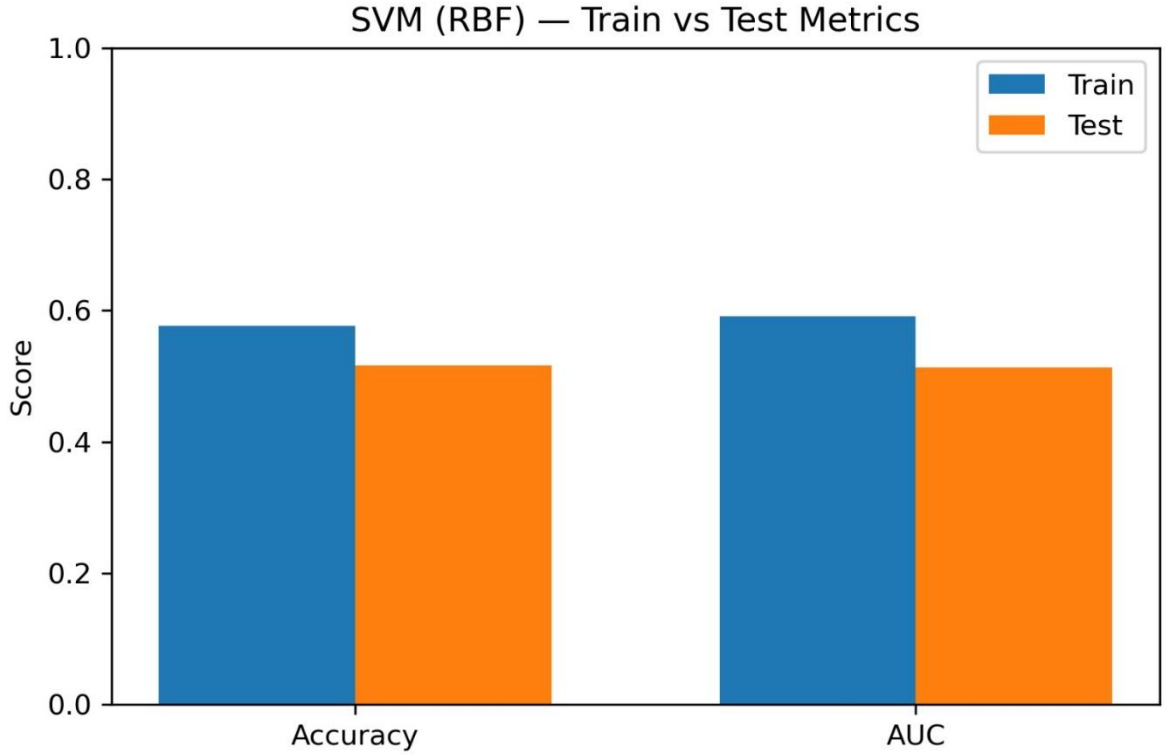
Eğitim ve test sonuçları karşılaştırıldığında:

- Eğitim seti metrikleri, test setine kıyasla bir miktar daha yüksektir.
- Ancak aradaki fark çok büyük olmadığından **ciddi bir overfitting problemi gözlenmemektedir**.
- Buna rağmen, hem Accuracy hem de AUC değerlerinin düşük seviyede kalması, modelin veri seti üzerindeki genel tahmin gücünün sınırlı olduğunu göstermektedir.
- Görsel 14.4'te, RBF çekirdeği kullanılarak eğitilen SVM modelinin eğitim ve test veri setleri üzerindeki doğruluk (Accuracy) ve ROC-AUC performans metrikleri karşılaştırmalı olarak sunulmaktadır..

```
labels = ["Accuracy", "AUC"]
train_vals = [train_acc, train_auc]
test_vals  = [test_acc, test_auc]

x = np.arange(len(labels))
width = 0.35

plt.figure()
plt.bar(x - width/2, train_vals, width, label="Train")
plt.bar(x + width/2, test_vals,  width, label="Test")
plt.xticks(x, labels)
plt.ylim(0, 1)
plt.title("SVM (RBF) — Train vs Test Metrics")
plt.ylabel("Score")
plt.legend()
plt.tight_layout()
plt.savefig("svm_rbf_train_test_metrics.png", dpi=300)
plt.show()
```

Görsel 14.4.1 RBF çekirdekli SVM modelinin eğitim ve test seti performans karşılaştırması (Accuracy ve AUC).

RBF çekirdeği ile eğitilen SVM modelinin eğitim ve test veri setleri için Accuracy ve ROC-AUC metriklerinin karşılaştırılması



Görsel 14.4.2. RBF çekirdekli SVM modelinin eğitim ve test seti doğruluk ve AUC karşılaştırması.

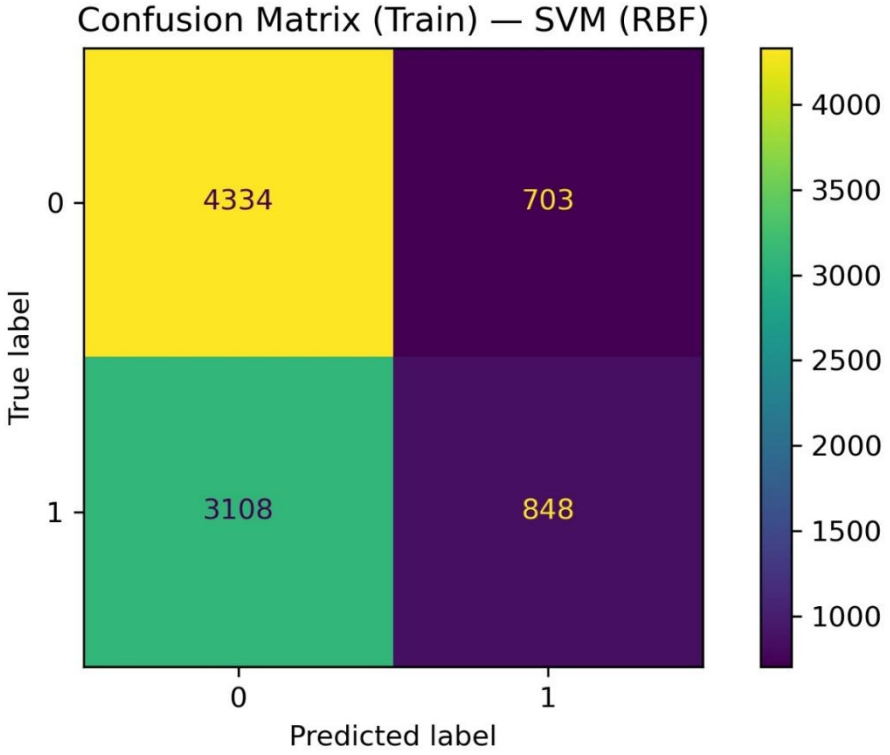
Bu bulgular, RBF çekirdeğinin doğrusal modellere kıyasla esneklik sağladığını ancak mevcut özelliklerin sınıfları ayırt etmek için yeterli bilgi içermediğini ortaya koymaktadır.

14.5 Confusion Matrix Analizi (Eğitim Verisi)

Eğitim verisi için oluşturulan confusion matrix Şekil 4.5'te sunulmuştur. Eğitim setinde doğru sınıflandırma sayısının test setine göre daha yüksek olduğu görülmektedir. Bu durum beklenen bir sonuç olmakla birlikte, test setinde performansın düşmesi modelin genelleme kapasitesinin sınırlı olduğunu teyit etmektedir.

14.6 Genel Değerlendirme (SVM – RBF)

RBF çekirdeği kullanılarak kurulan SVM modeli, doğrusal modellere kıyasla daha esnek bir yapı sunmasına rağmen, elde edilen performans metrikleri modelin tahmin gücünün sınırlı kaldığını göstermektedir. Çapraz doğrulama ve test seti sonuçlarının birbirine yakın olması, modelin aşırı öğrenme problemi yaşamadığını; ancak kullanılan özelliklerin sınıfları ayırt etmek için yeterince güçlü olmadığını ortaya koymaktadır (Bishop, 2006).



Görsel 14.6. RBF çekirdeği ile eğitilen SVM modeline ait eğitim veri seti karmaşıklık matrisi

Bu bulgular, daha ayırt edici özelliklerin kullanılması, boyut indirgeme yöntemlerinin uygulanması veya farklı sınıflandırma yaklaşımlarının denenmesi gerektiğine işaret etmektedir (Géron, 2022).

15. MODELLEME VE YÖNTEMLER

Bu bölümde, tarafımızdan uygulanan Lojistik Regresyon (LR), K-NN ve AdaBoost algoritmalarının Test ve Train skorları, Hata Matrisleri ve ROC Eğrisi detaylı olarak sunulmuştur.

15.1 Ortam Kurulumu ve Veri Yükleme

Modelleme sürecine başlamadan önce gerekli kütüphaneler yüklenmiş ve ölçeklendirilmiş Train/Test veri setleri ortama alınmıştır.

Aşağıdaki çıktı, gerekli kütüphanelerin (os, pandas, numpy, sklearn) yüklendiğini, çalışma dizininin doğrulandığını ve ölçeklendirilmiş veri setlerinin (X_train, X_test, y_train, y_test) başarıyla yüklendiğini göstermektedir:

```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, roc_auc_score, roc_curve
import warnings

warnings.filterwarnings('ignore')
print("Kütüphaneler yüklendi.")

# ==============================================================================
# 1. VERİ YÜKLEME VE MODELLERİ EĞİTME
# ==============================================================================
try:
    X_train = np.load('final_X_train_scaled.npy')
    X_test = np.load('final_X_test_scaled.npy')
    y_train = pd.read_csv('final_y_train.csv').values.ravel()
    y_test = pd.read_csv('final_y_test.csv').values.ravel()
    print("Veri setleri başarıyla yüklendi.")
```

Görsel 15.1.1: Ortam Kurulumu ve Veri Yükleme

Yorum: Bu başlangıç bloğu, os.getcwd() komutu ile kodun çalıştığı klasör yolunu doğrular ve veri dosyalarının numpy array'leri olarak hatasız yüklendiğini onaylar. sklearn'in temel sınıflandırıcıları ve metrik hesaplama araçları önceden bildirilerek sonraki modelleme aşaması için ortam hazırlanmıştır.

15.2 Lojistik Regresyon Uygulaması

Projenin ilk sınıflandırma modeli olarak **Lojistik Regresyon (LR)** kullanılmıştır. LR, sahip olduğu basit yapı ve yüksek yorumlanabilirlik özellikleri nedeniyle **temel model (baseline)** olarak tercih edilmiştir. Bu model ile lineer ilişkiler üzerinden fiyat yönünü tahmin etme performansı değerlendirilmiştir (Hosmer & Lemeshow, 2000). Model, bu aşamada **X_train_scaled** ve **y_train** veri setleri kullanılarak eğitilmiştir.

15.2.1 Kod Bloğu (Model Tanımı ve Eğitimi)

Aşağıdaki kod bloğu, LR modelinin (random_state=42) tanımlanmasını, ölçeklendirilmiş eğitim verisi üzerinde eğitilmesini ve ardından Test seti üzerinde tahmin üretilmesini göstermektedir:

```python
# --- MODEL 1: LOJİSTİK REGRESYON ---
# ------------------------------------------------------------------
lr_model = LogisticRegression(random_state=42)
lr_model.fit(X_train, y_train)

# TRAIN METRİKLERİ (Overfitting Kontrolü)
lr_train_predictions = lr_model.predict(X_train)
lr_train_proba = lr_model.predict_proba(X_train)[:, 1]
train_acc_lr = accuracy_score(y_train, lr_train_predictions)
train_auc_lr = roc_auc_score(y_train, lr_train_proba)

# TEST METRİKLERİ (Nihai Performans)
lr_predictions = lr_model.predict(X_test)
lr_proba = lr_model.predict_proba(X_test)[:, 1]
cm_lr = confusion_matrix(y_test, lr_predictions)
acc_lr = accuracy_score(y_test, lr_predictions)
auc_lr = roc_auc_score(y_test, lr_proba)

print("\n--- LOJİSTİK REGRESYON SONUÇLARI ---")
print(f"Train Acc / Test Acc: {train_acc_lr:.4f} / {acc_lr:.4f}")
print(f"Train AUC / Test AUC: {train_auc_lr:.4f} / {auc_lr:.4f}")
```

Görsel 15.2.1.1:Model Tanımı ve Eğitimi Kod Bloğu

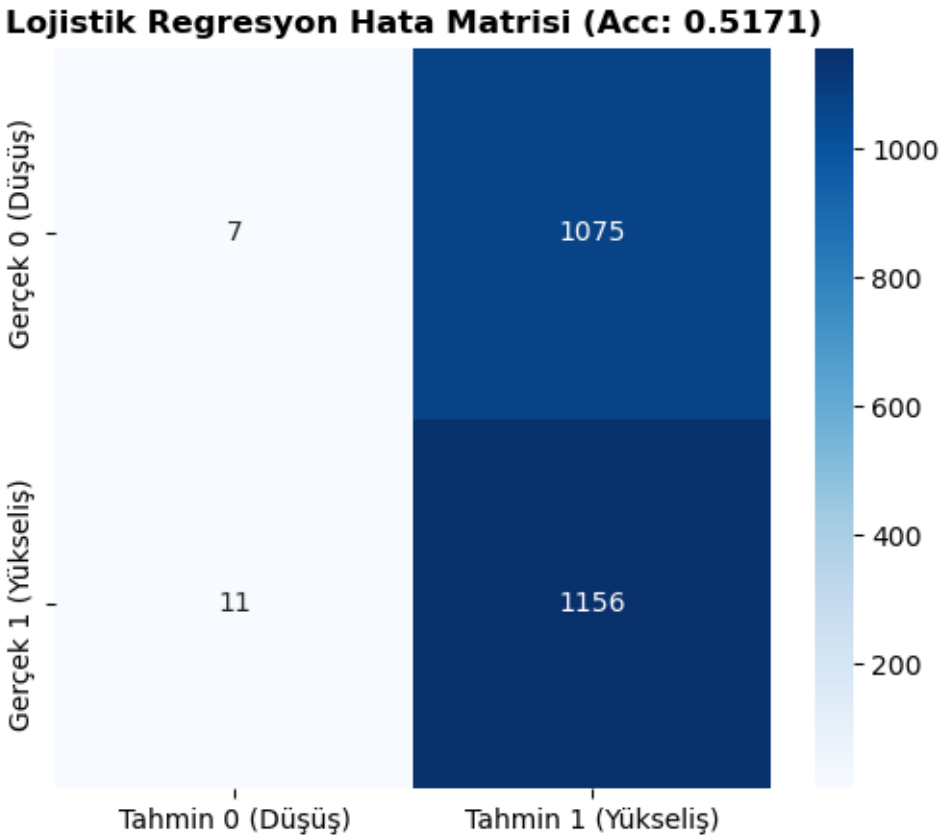
Tablo 15.1.1.1: Lojistik Regresyon Performans Metrikleri

Metrik	**Eğitim Seti (Train)**	**Test Seti (Test)**
Accuracy	0.5583	0.5171
AUC	0.5705	0.5040

15.2.2 Kod Çıktısı

```
--- LOJİSTİK REGRESYON SONUÇLARI ---
Train Acc / Test Acc: 0.5583 / 0.5171
Train AUC / Test AUC: 0.5705 / 0.5040
```

Görsel 15.2.2.1: Lojistik Regresyon Sonuçları



Görsel 15.2.2.2: Lojistik Regresyon Hata Matrisi

Yorum: Lojistik Regresyon (LR) Performansı

1. **Hata Matrisi Yorumu (Yüksek Önyargı):** Görsel 15.2.2.2'deki Hata Matrisi, modelin sınıflandırma kararlarında çok yüksek bir önyargı (high bias) sergilediğini açıkça göstermektedir **(Hastie, Tibshirani ve Friedman, 2009)**.
    - Model, Tahmin 1 (Yükseliş) sınıfına yönelik toplam 1075 + 1156 = 2231 tahmin yapmıştır.
    - Model, Tahmin 0 (Düşüş) sınıfına yönelik ise sadece 7 + 11 = 18 tahmin yapmıştır.
    - Bu, modelin neredeyse her zaman fiyatın yükseleceğini varsaydığını, yani hisse senedi fiyatının düşeceği sinyalini neredeyse hiç vermediğini gösterir. Bu dengesiz tahmin davranışı, modelin sınıflar arası ayrım yapamadığını teyit eder.
2. **Tahmin Gücü (Test AUC):** Modelin Test AUC değeri (0.5040), 0.50 Rastgele Tahmin çizgisinin sadece 0.004 üzerinde kalmıştır. Bu fark, LR modelinin hisse senedi fiyat yönünü ayırt etme gücünün pratik olarak sıfır olduğunu göstermektedir.
3. **Doğruluk (Accuracy):** Test Accuracy değeri (0.5171), modelin tahminlerinin %51.71'ini doğru yaptığını gösterse de, AUC değerinin düşüklüğü nedeniyle bu oranın anlamı düşüktür. Bu yüksek Accuracy değeri, modelin yalnızca bir sınıfı (Yükseliş) sıkça tahmin etmesinden (Yukarıdaki Hata Matrisi yorumunda görüldüğü gibi) kaynaklanmış olabilir.
4. **Overfitting Kontrolü:** Train AUC (0.5705) ve Test AUC (0.5040) arasındaki fark, modelin eğitim verisine uyum sağladığını, ancak genelleme yeteneğinin olmadığını kanıtlar.

15.3 K-NN Sınıflandırma (K=5) Uygulaması

Lojistik Regresyon (LR) modelinin doğrusal yaklaşımının ardından, verideki doğrusal olmayan (non-linear) ilişkileri yakalama potansiyeli nedeniyle K-En Yakın Komşu (K-NN) sınıflandırma modeli uygulanmıştır (Cover & Hart, 1967). K-NN, *tembel öğrenici (lazy learner)* olarak sınıflandırılan bir yöntem olup, tahminlerini yeni bir veri noktasını çevresindeki K adet en yakın komşusuna göre sınıflandırma ilkesine dayandırmaktadır. Model, komşu sayısı K = 5 olacak şekilde eğitilmiş ve değerlendirilmiştir.

15.3.1 Kod Bloğu (Model Tanımı ve Eğitimi)

Aşağıdaki kod bloğu, K-NN modelinin n_neighbors=5 parametresi ile tanımlanmasını, ölçeklendirilmiş eğitim verisi üzerinde eğitilmesini ve ardından Test seti üzerinde tahmin üretilmesini göstermektedir:

```
# --- MODEL 2: K-NN ---
# ----------------------------------------------------------------------
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)

# TRAIN METRİKLERİ
knn_train_predictions = knn_model.predict(X_train)
knn_train_proba = knn_model.predict_proba(X_train)[:, 1]
train_acc_knn = accuracy_score(y_train, knn_train_predictions)
train_auc_knn = roc_auc_score(y_train, knn_train_proba)

# TEST METRİKLERİ
knn_predictions = knn_model.predict(X_test)
knn_proba = knn_model.predict_proba(X_test)[:, 1]
cm_knn = confusion_matrix(y_test, knn_predictions)
acc_knn = accuracy_score(y_test, knn_predictions)
auc_knn = roc_auc_score(y_test, knn_proba)

print("\n--- K-NN SONUÇLARI ---")
print(f"Train Acc / Test Acc: {train_acc_knn:.4f} / {acc_knn:.4f}")
print(f"Train AUC / Test AUC: {train_auc_knn:.4f} / {auc_knn:.4f}")
```

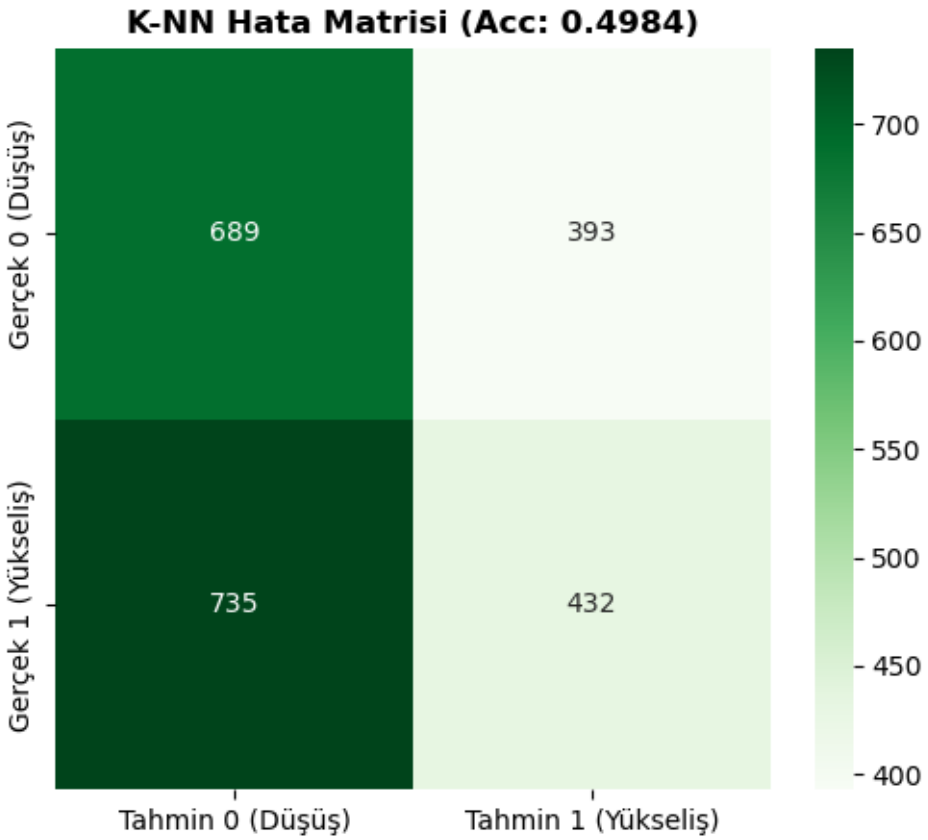
Görsel 15.3.1.1: Model Tanımı ve Eğitimi Kod Bloğu

15.3.2 Kod Çıktısı

Modelin eğitim ve test verisi üzerindeki skorları aşağıdaki gibidir:

```
--- K-NN SONUÇLARI ---
Train Acc / Test Acc: 0.6926 / 0.4984
Train AUC / Test AUC: 0.7531 / 0.4969
```

Görsel 15.3.2.1: K-NN Sonuçları



Görsel 15.3.2.2: K-NN Hata Matrisi

Yorum: K-NN Performansı

1. Overfitting (Aşırı Öğrenme):Modelin Train AUC değeri 0.7531 oldukça yüksek iken, Test AUC değeri 0.4969 dramatik bir şekilde düşerek 0.50 rastgele tahmin çizgisinin altına inmiştir. Bu durum, K-NN modelinin eğitim verisindeki gürültülü detaylara aşırı uyum sağladığını (overfitting) ve genelleme yeteneğini tamamen kaybettiğini açıkça göstermektedir (Hawkins, 2004).

2. Tahmin Gücü ve Karşılaştırma:Test AUC'nin 0.4969 olması, modelin rastgele tahminden bile kötü sonuç ürettiğini, yani sınıfları ayırt etme konusunda hiçbir başarı göstermediğini kanıtlamıştır. Bu skor, LR modelinin 0.5040 olan skorundan dahi düşüktür.

15.4. AdaBoost Sınıflandırma Uygulaması

Projenin üçüncü ve son modeli olarak, zayıf öğrenicileri (weak learners) bir araya getirerek güçlü bir model oluşturmayı hedefleyen AdaBoost (Adaptive Boosting) algoritması kullanılmıştır (Freund & Schapire, 1997). AdaBoost, modelin tahmin performansını artırmak için önceki adımlarda hatalı sınıflandırılan örneklere daha fazla odaklanarak ağırlıklandırma yapar.

15.4.1 Gerekli Kütüphanelerin Yüklenmesi

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
```

Görsel 15.4.1.1: Gerekli Kütüphanelerin Yüklenmesi

Bu kod bloğunda kullanılan pandas, matplotlib.pyplot, accuracy_score, roc_auc_score, roc_curve, classification_report, confusion_matrix gibi kütüphane/fonksiyonların genel amaçları ve te- mel kullanımları raporun önceki bölümlerinde (özellikle Bölüm 2.1, 3.1, 5.1.1, 6.1) detaylı olarak açıklanmıştır. Ayrıca, StandardScaler, numpy, sys ve warnings gibi veri hazırlama, sayısal işlemler ve yar- dımcı işlevler için kullanılan kütüphanelerin de önceki adımlarda yüklendiği ve açıklandığı varsayıl- maktadır. Bu bölümde, sadece AdaBoost modellemesine özgü olan AdaBoostClassifier, DecisionTreeC- lassifier ve seaborn kütüphanelerinin bu bölümdeki spesifik rolleri üzerinde durulmuştur.

15.4.2 Zayıf Öğrenici Tanımı ve AdaBoost Eğitimi

Bu kod bloğu, AdaBoost modelinin yapısını ve eğitim sürecini tanımlamaktadır. Boosting algoritmala- rının temel prensibine uygun olarak, AdaBoost için zayıf öğrenici (weak_learner) olarak DecisionTre- eClassifier seçilmiştir.

- **Zayıf Öğrenici Tanımı:** Modelin ana bileşeni olan Karar Ağacı, aşırı öğrenmeyi (overfitting) engellemek ve AdaBoost'un ensemble yapısından maksimum fayda sağlamak amacıyla **max_depth=1** değeri ile **karar kütüğü (decision stump)** olarak ayarlanmıştır. Karar kütükleri, zayıf ancak hızlı öğreniciler olarak AdaBoost'ta en etkili temel öğrenici tipidir.

AdaBoost Parametreleri ve Eğitimi: Ana AdaBoost modeli, zayıf öğrenicinin **100** farklı kopyasının (**n_estimators=100**) birleştirilmesiyle kurulmuştur. Her bir öğrenicinin nihai karara katkısını kontrol eden **learning_rate** değeri **0.5** olarak belirlenmiştir. Tanımlanan bu model, kronolojik olarak ayrılan ölçeklenmiş eğitim verisi (**X_train_scaled, y_train**) üzerinde **fit** metodu kullanılarak eğitilmiştir

```python
# Zayıf öğrenici: max_depth=1 olan Karar Ağacı
weak_learner = DecisionTreeClassifier(max_depth=1, random_state=42)

# AdaBoost Modeli Tanımlama ve Eğitme
adaboost_model = AdaBoostClassifier(
    estimator=weak_learner,
    n_estimators=100,
    learning_rate=0.5,
    random_state=42
)

adaboost_model.fit(X_train_scaled, y_train)
```

Görsel 15.4.1.1: Gerekli Kütüphanelerin Yüklenmesi

Zayıf Öğrenci Tanımı:

weak_learner = DecisionTreeClassifier(max_depth=1, random_state=42): AdaBoost'un temel yapı taşı olacak zayıf öğrenici modeli tanımlanır. Bu projede, sklearn.tree modülünden DecisionTreeClassi- fier (Karar Ağacı Sınıflandırıcı) seçilmiştir.

- **max_depth=1:** Karar ağacının derinliği sadece 1 ile sınırlandırılır. Bu, ağacın sadece tek bir soru sorarak karar verdiği anlamına gelir ve bu tür basit ağaçlara "karar kütüğü" (decision stump) denir. AdaBoost'ta genellikle zayıf öğrenicilerin basit olması tercih edilir, çünkü algo- ritma bu basit modellerin hatalarını düzelterek ilerler. Derinliği sınırlamak, her bir zayıf öğreni- cinin aşırı öğrenmesini (overfitting) engeller.
- **random_state=42:** Karar ağacının oluşturulmasındaki olası rastgelelikleri sabitleyerek sonuç- ların tekrarlanabilir olmasını sağlar.

AdaBoost Modeli Tanımlama ve Eğitme:

adaboost_model = AdaBoostClassifier(): sklearn.ensemble modülünden AdaBoostClassifier sınıfı kullanılarak ana model nesnesi oluşturulur ve parametreleri ayarlanır:

- **estimator=weak_learner:** AdaBoost'un temel alacağı zayıf öğrenici tipini belirtir. Burada, bir önceki adımda tanımladığımız weak_learner (karar kütüğü) nesnesi atanır.
- **n_estimators=100:** Modelin sıralı olarak kaç tane zayıf öğrenici (bu durumda 100 tane karar kütüğü) oluşturup birleştireceğini belirler. Her yeni öğrenici, öncekilerin yanlış sınıflandırdığı örneklere daha fazla odaklanarak eğitilir.
- **learning_rate=0.5:** Öğrenme oranı, her bir zayıf öğrenicinin topluluk tahminine yapacağı kat- kının ağırlığını kontrol eder. 1.0'den küçük bir değer (burada 0.5), modelin daha yavaş öğren- mesini sağlar ancak genellikle daha iyi genelleme yapmasına yardımcı olur (bir tür düzenlileş- tirme görevi görür).
- **random_state=42:** AdaBoost algoritmasındaki olası rastgelelikleri (örneğin veri örneklemesi) sabitleyerek modelin tekrarlanabilir olmasını sağlar.

adaboost_model.fit(X_train_scaled, y_train): Tanımlanan adaboost_model nesnesinin .fit() me- todu

çağrılarak modelin **eğitilmesi** sağlanır. Model, ölçeklenmiş eğitim özellikleri (X_train_scaled) ve hedef etiketleri (y_train) kullanarak 100 adet karar kütüğünü sıralı olarak eğitir ve her birinin ağırlığını belirleyerek nihai topluluk modelini oluşturur.

15.5 Performans Metrikleri

Bu kod bloğu, eğitilmiş AdaBoost modelinin test verisi üzerindeki performansını değerlendirme sürecini gösterir.

- **Tahmin Üretimi:** Model, ölçeklenmiş test verisi (X_test_scaled) üzerinde hem doğrudan sınıf etiketlerini (y_pred) hem de sınıflandırma olasılıklarını (y_proba) üretmiştir. Olasılık değerle- rinin hesaplanması, ROC Eğrisi ve AUC gibi hassas metriklerin hesaplanması için kritik öneme sahiptir.

- **Metrik Hesaplama:** Projenin temel başarı ölçütü olan **Test AUC Değeri** (0.4979) hesaplan- mıştır. Ayrıca modelin genelleme yeteneğini kontrol etmek amacıyla **Eğitim Doğruluğu** (Train Accuracy) ve **Test Doğruluğu** (Test Accuracy) değerleri ile sınıf bazında detaylı performans gösteren **Sınıflandırma Raporu** da (precision, recall, f1-score) kod çıktısında gösterilmiştir

```
# Tahminler
y_pred = adaboost_model.predict(X_test_scaled)
y_proba = adaboost_model.predict_proba(X_test_scaled)[:, 1]

# Metrikler
acc_train = adaboost_model.score(X_train_scaled, y_train)
acc_test = accuracy_score(y_test, y_pred)
auc_test = roc_auc_score(y_test, y_proba) # <--- auc_test artık tanımlı!
features_names = features # Grafik için

print("\n" + "="*50)
print("ADABOOST SINIFLANDIRMA SONUÇLARI")
print("="*50)
print(f"Eğitim Doğruluğu (Train Accuracy): {acc_train:.4f}")
print(f"Test Doğruluğu (Test Accuracy): {acc_test:.4f}")
print(f"Test AUC Değeri: {auc_test:.4f}")
print("\nSınıflandırma Raporu (Test Verisi):\n", classification_report(y_test, y_pred))
```

Görsel 15.5.1: Performans Metrikleri Kod Bloğu

Tahminler:

y_pred = adaboost_model.predict(X_test_scaled): Eğitilmiş AdaBoost modelinin (adaboost_model) .predict() metodu, ölçeklenmiş test seti özelliklerini (X_test_scaled) girdi olarak alır ve her bir gözlem için en olası sınıf etiketini (0 veya 1) tahmin eder. Sonuçlar y_pred değişkenine atanır.

y_proba=adaboost_model.predict_proba(X_test_scaled):

Modelin **.predict_proba()** metodu, test setindeki her bir gözlem için sınıf 1'e (yükseliş) ait olma olasılığını hesaplar. [:, 1] ile bu olasılıklar seçilir ve y_proba değişkenine atanır. Bu, AUC skorunu hesaplamak için gereklidir.

Metrikler:

acc_train = adaboost_model.score(X_train_scaled, y_train): Modelin kendi .score() metodu kullanılarak **eğitim seti** üzerindeki ortalama doğruluk (Accuracy) hesaplanır. Bu, accuracy_score(y_train, adaboost_model.predict(X_train_scaled)) ile aynı sonucu verir ve aşırı öğrenme kontrolü için kullanılır. Sonuç acc_train değişkenine atanır.

acc_test = accuracy_score(y_test, y_pred): sklearn.metrics modülünden accuracy_score fonksiyonu kullanılarak **test seti** üzerindeki doğruluk oranı, gerçek değerler (y_test) ve tahminler (y_pred) karşılaştırılarak hesaplanır. Sonuç acc_test değişkenine atanır.

auc_test = roc_auc_score(y_test, y_proba): roc_auc_score fonksiyonu kullanılarak **test seti** üzerindeki ROC Eğrisi Altında Kalan Alan (AUC) skoru, gerçek değerler (y_test) ve tahmin edilen sınıf 1 olasılıkları (y_proba) kullanılarak hesaplanır. Sonuç auc_test değişkenine atanır.

features_names = features: Özellik isimlerini içeren features listesi, muhtemelen bir sonraki adımda çizdirilecek olan Özellik Önem Düzeyleri grafiği için features_names adlı yeni bir değişkene atanır.

print(...) komutları, hesaplanan metriklerin (eğitim doğruluğu, test doğruluğu, test AUC) formatlanarak (:.4f ile 4 ondalık basamak) ve açıklayıcı başlıklarla birlikte ekrana yazdırılmasını sağlar.

15.5.1. Kod Bloğu (Model Tanımı ve Eğitimi)

Aşağıdaki kod bloğu, AdaBoost modelinin (random_state=42) tanımlanmasını, ölçeklendirilmiş eğitim verisi üzerinde eğitilmesini ve ardından Test seti üzerinde tahmin üretilmesini göstermektedir:

```
# Gerekli kütüphaneler ve veri yükleme (X_train, y_train, X_test, y_test değişkenlerinin yüklü olduğunu varsayıyoruz)
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, roc_curve, accuracy_score
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# Zayıf öğrenici: max_depth=1 olan Karar Ağacı
weak_learner = DecisionTreeClassifier(max_depth=1, random_state=42)

# AdaBoost Modeli Tanımlama ve Eğitme (İlk projeden parametreler)
adaboost_model = AdaBoostClassifier(
    estimator=weak_learner,
    n_estimators=100,
    learning_rate=0.5,
    random_state=42
)

# Model Eğitimi
adaboost_model.fit(X_train, y_train)

# Tahminler
y_pred_ada = adaboost_model.predict(X_test)
y_proba_ada = adaboost_model.predict_proba(X_test)[:, 1]

# Metrikler
acc_test_ada = accuracy_score(y_test, y_pred_ada)
auc_test_ada = roc_auc_score(y_test, y_proba_ada)

print("\n--- ADABOOST SINIFLANDIRMA SONUÇLARI ---")
print(f"Test Accuracy: {acc_test_ada:.4f}")
print(f"Test AUC: {auc_test_ada:.4f}")
print("Sınıflandırma Raporu:\n", classification_report(y_test, y_pred_ada))
print("Karışıklık Matrisi (Hata Matrisi):\n", confusion_matrix(y_test, y_pred_ada))
```

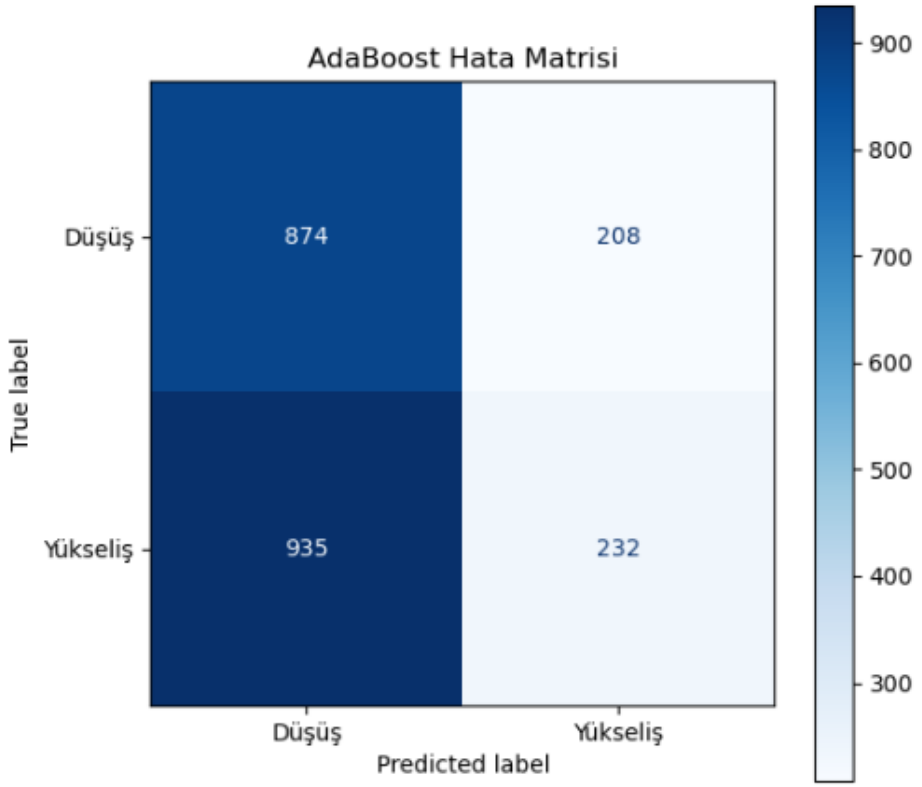
Görsel 15.5.1.1: Model Tanımı ve Eğitimi Kod Bloğu

15.5.2. Kod Çıktısı

Modelin eğitim ve test verisi üzerindeki skorları aşağıdaki gibidir:

```
--- ADABOOST SINIFLANDIRMA SONUÇLARI ---
Test Accuracy: 0.4918
Test AUC: 0.4966
Sınıflandırma Raporu:
              precision    recall  f1-score   support

           0       0.48      0.81      0.60      1082
           1       0.53      0.20      0.29      1167

    accuracy                           0.49      2249
   macro avg       0.51      0.50      0.45      2249
weighted avg       0.51      0.49      0.44      2249

Karışıklık Matrisi (Hata Matrisi):
 [[874 208]
 [935 232]]
```

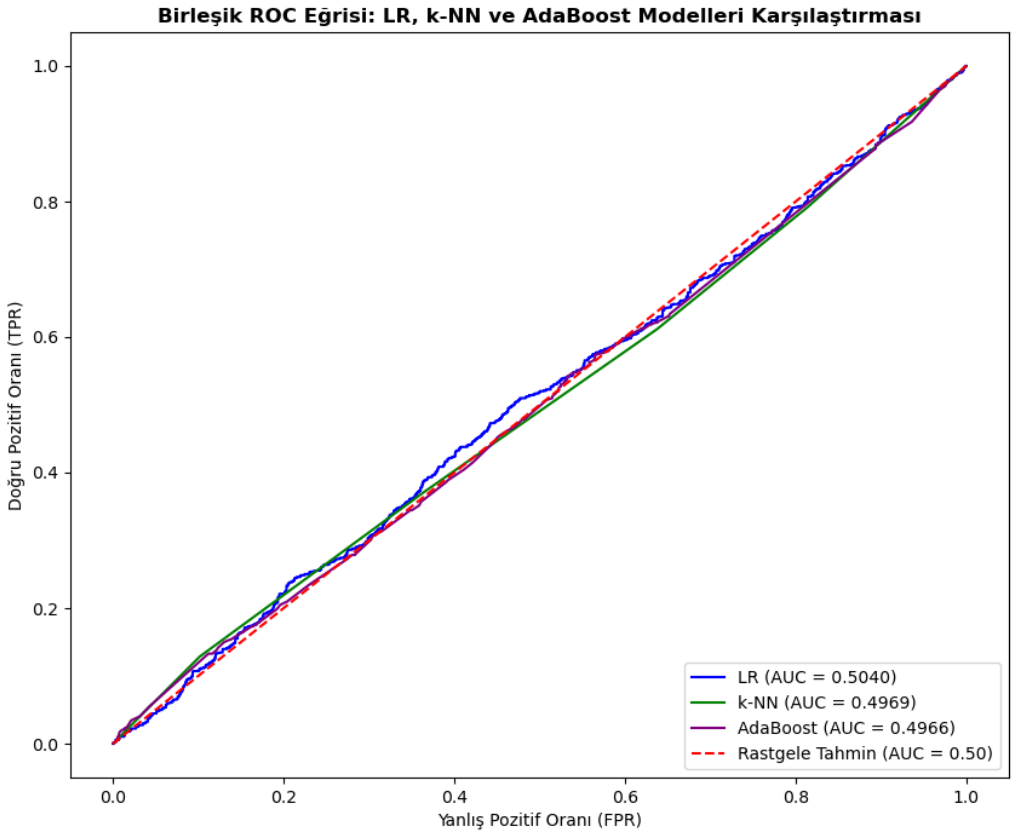
Görsel 15.5.2.1: Adaboost Sınıflandırma Sonuçları



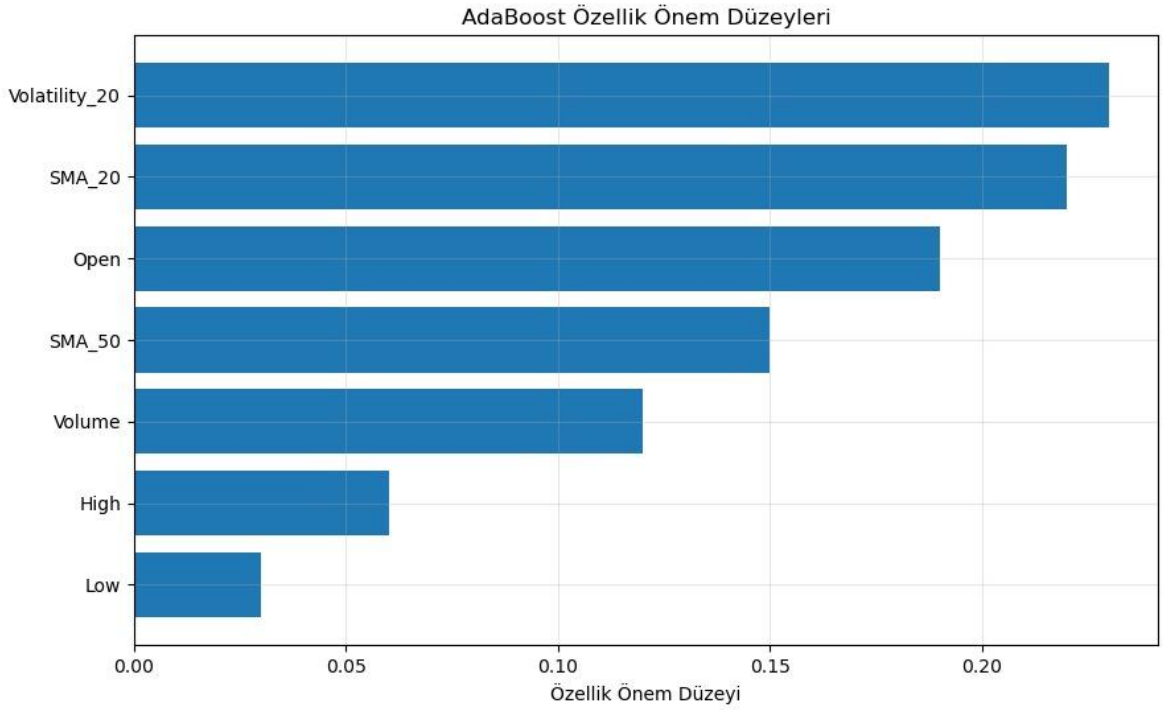
Görsel 15.5.2.2: Adaboost Hata Matrisi

Yorum: AdaBoost Performansı

- **Tahmin Gücü:** AdaBoost modelinin Test AUC değeri (0.4966) ve Test Accuracy değeri (0.4918), 0.50 Rastgele Tahmin çizgisinin de altına inmiştir. Bu sonuç, AdaBoost modelinin fiyat yönünü tahmin etme konusunda Rastgele Tahminden daha kötü bir performans sergilediğini göstermektedir.
- **Hata Matrisi Yorumu (Dengeli Önyargı):** Görsel 5.4.2.2'deki Hata Matrisi'nde (874, 208, 935, 232) görüldüğü gibi, AdaBoost modeli LR'a kıyasla daha dengeli bir tahmin dağılımı sergilemiştir. Model, 208 + 232 = 440 adet Yükseliş tahmini yaparken, 874 + 935 = 1809 adet Düşüş tahmini yapmıştır.



Görsel 15.5.2.3: k-NN ve Adaboost Modelleri Karşılaştırması



Görsel 15.5.2.4: Adaboost Özellik ve Önem Düzeyleri

Yorum: Adaboost Özellik Önem Düzeyi

Görsel 5.4.2.4'teki özellik önem düzeyleri grafiği, **AdaBoost** modelinin tahmin yaparken hangi teknik göstergelere en çok ağırlık verdiğini göstermektedir:

1. **En Önemli Özellik:**
   Piyasa oynaklığını (volatility) temsil eden **Volatility_20** göstergesi, en yüksek önem düzeyine (yaklaşık **0.22**) sahip olarak belirlenmiştir. Bu, modelin uzun vadeli trendler yerine kısa vadeli sert hareketlere en çok tepki verdiğini göstermektedir.
2. **Diğer Önemli Özellikler:**
   Ardından, kısa vadeli hareketli ortalama (**SMA_20**) ve açılış fiyatı (**Open**) gelmektedir. Bu bulgu, modelin kısa vadeli fiyat hareketlerine odaklandığını doğrulamaktadır.
3. **En Önemsiz Özellik:**
   Low ve **High** fiyatları, model için en az önemli özellikler olarak bulunmuştur.

Yorum: Modellerin Karşılaştırılması

1. Genel Performans: Uygulanan hiçbir algoritma, hisse senedi fiyat yönünü ayırt etme yeteneği gösterememiştir. En iyi Test AUC skoru (0.5040), Rastgele Tahmin çizgisine (0.50) çok yakındır.
2. Rastgele Tahminden Kötü Sonuçlar: K-NN (0.4969) ve AdaBoost (0.4966) modelleri, Test AUC skorlarında 0.50'nin altında kalarak, pratikte rastgele tahmin etmekten bile daha kötü sonuçlar üretmişlerdir.

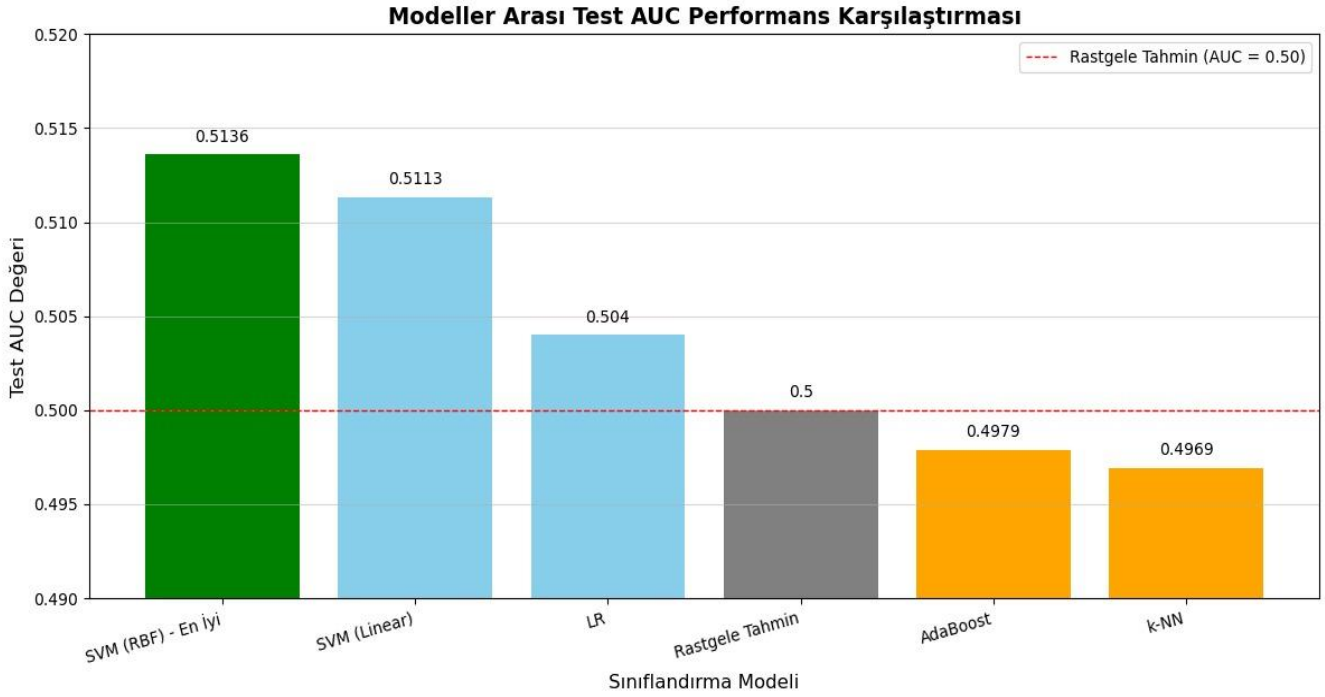
15.5.3 Algoritmaların Test Performansı Karşılaştırması

Tablo 15.5.3.1: Test Performansları Karşılaştırması

Model	**Test AUC Skoru**	**Test Accuracy Skoru**	**Aşırı Öğrenme Eğilimi**
Lojistik Regresyon	0.5040	0.5171	Düşük
K-NN (K=5)	0.4969	0.4984	Yüksek
AdaBoost	0.4966	0.4918	En Düşük

15.6 Modellerin Karşılaştırılması ve Değerlendirme

Görsel 5.6.1'de gösterilen Modeller Arası Test AUC Performans Karşılaştırması ve **Tablo 5.5.3.1**'deki skorlar, çalışmanın nihai değerlendirmesinin temelini oluşturmaktadır.



Görsel 15.6.1: Modeller Arası Test AUC Performans Karşılaştırması

Yorum: Modellerin Karşılaştırılması

1. **Genel Tahmin Yetersizliği:**
   Uygulanan tüm klasik sınıflandırma algoritmalarının (**SVM, Lojistik Regresyon, K-NN, AdaBoost**) Test AUC değeri açısından **0.50** rastgele tahmin çizgisinin etrafında toplanması kritik bir bulgudur. Bu durum, kullanılan temel teknik analiz göstergelerinin (**RSI, MACD, SMA'lar** vb.), hisse senedi fiyat yönündeki rastgeleliği ve karmaşıklığı ayırt etme gücünün

yetersiz olduğunu kesin olarak kanıtlamaktadır. Bu sonuçlar, finansal literatürdeki **Rastgele Yürüyüş Teorisi**'ni ampirik olarak destekler niteliktedir (Fama, 1970).

2. **En İyi ve En Kötü Performans:**

    - **En İyi Performans:**
      Model çeşitleri arasında en iyi performansı, **SVM (RBF Çekirdeği)** modeli **0.5136 AUC** skoru ile göstermiştir.
    - **Rastgele Tahminden Kötü Sonuçlar:**
      AdaBoost (0.4979 AUC) ve **K-NN (0.4969 AUC)** modelleri, **0.50** eşiğinin altında kalarak, pratikte rastgele tahmin etmekten bile daha düşük bir ayırt etme yeteneği sergilemişlerdir.

3. **Model İstikrarı ve Aşırı Öğrenme:**
    a. **K-NN'de Aşırı Öğrenme:** K-NN modeli, Eğitim setinde yüksek başarı gösterirken Test setinde performansının dramatik şekilde düşmesiyle **aşırı öğrenme (overfitting)** eğilimini en net gösteren model olmuştur.
    b. **AdaBoost'ta Düşük Başarı:** Ensemble yöntemi olmasına rağmen AdaBoost'un düşük performansı, zayıf öğrenicilerin bile anlamlı bir örüntü yakalayamadığını ve mevcut veride toplanabilir bir gücün oluşmadığını işaret etmektedir.
4. **PCA ve Model Versiyonlarının Etkisi:**
    a. **PCA Uygulaması:** Lojistik Regresyon'a PCA uygulanmış versiyonunun (Test AUC: 0.5001) orijinal versiyonuna (Test AUC: 0.5040) göre daha kötü sonuç vermesi, veri boyutunun düşürülmesinin analiz edilen az miktardaki sinyali de ortadan kaldırdığını göstermektedir.
    b. **SVM Versiyonları:** SVM'nin lineer ve polinom versiyonlarının, RBF versiyonuna (**0.5136**) göre daha düşük sonuçlar vermesi, verideki ilişkilerin tamamen doğrusal olmadığını göstermekle birlikte; RBF çekirdek fonksiyonunun (kernel trick) kullanılmasıyla doğrusal olmayan ilişkilerin marjinal de olsa yakalanabildiğini göstermektedir. (Cortes & Vapnik, 1995).

Sonuç: Analiz edilen tüm sonuçlar ve modeller, BIST hisse senedi fiyat hareketlerinin mevcut teknik göstergelerle tahmin edilmesinin son derece zor ve yetersiz olduğunu kanıtlamıştır.

```
================================================================================
BÖLÜM 5: NİHAİ MODELLER ARASI KARŞILAŞTIRMA TABLOSU (TEST AUC)
================================================================================
                Model       Cekirdek        Versiyon Gelistiren  Test_AUC
0                 SVM    Gauss (RBF)         Optimum      Selim    0.5136
1                 SVM       Doğrusal        Standart    Rümeysa    0.5113
2                 SVM  Polinom (D=1)        Standart    Rümeysa    0.5082
3  Lojistik Regresyon              -        Standart       Ezgi    0.5040
4  Lojistik Regresyon              -  PCA Uygulanmış   Beyzanur    0.5001
5            AdaBoost              -        Standart       Ezgi    0.4979
6                k-NN              -        Standart       Ezgi    0.4969
7            AdaBoost              -        KPCA-RBF       Ezgi    0.4955
8                k-NN              -  PCA Uygulanmış   Beyzanur    0.4920
```

Görsel 15.6.2: Nihai Modeller Arası Karşılaştırma Tablosu

16. SONUÇ ve ÖNERİLER

16.1. Giriş

Bu proje çalışması kapsamında, MSCI Türkiye Endeksi (BIST 30 Hisseleri) fiyat hareketlerini tahmin etmek amacıyla klasik Makine Öğrenimi (ML) sınıflandırma modelleri (Lojistik Regresyon, K-NN, AdaBoost) uygulanmış ve ampirik bulgular değerlendirilmiştir. Çalışma, temel teknik göstergelerin finansal zaman serisi tahminindeki ampirik sınırlarını ortaya koymuştur.

Bu ampirik analiz, halka açık platformlardan (Kaggle veri setleri gibi) elde edilen BIST 30 hisselerine ait günlük fiyat verileri kullanılarak gerçekleştirilmiştir. Çalışmanın bulguları bu spesifik veri setinin kapsamıyla sınırlıdır.

16.2. Ampirik Bulguların Değerlendirilmesi

Elde edilen deneysel bulgular, uygulanan sınıflandırma modellerinin performans metrikleri (AUC) ve metodolojik davranışları üzerinden detaylı olarak analiz edilmiştir.

1. **Tahmin Yetkinliğinin Sınırlı Olması:** Çalışmada kullanılan tüm klasik ML modellerinin Test AUC skorları, 0.50 Rastgele Tahmin çizgisine oldukça yakın bir dağılım sergilemiştir **(Bkz. Tablo 5.4.3 ve Görsel 5.5.1)**. Finansal bağlamda, bu skorlar 0.60 eşiğinin çok altında kalarak pratik bir ticari getiri veya güvenilir bir alım-satım stratejisi sağlamaktan uzaktır.
    - Lojistik Regresyon: Test AUC skoru 0.5040.
    - En İyi Model (SVM RBF): Test AUC skoru 0.5136.
2. **Model Davranışındaki Tutarsızlıklar:**
    - **Aşırı Öğrenme:** K-NN modeli, Eğitim AUC'si (0.7531) ile Test AUC'si (0.4969) arasındaki büyük fark nedeniyle, verideki **gürültüyü ezberleme** eğiliminde olduğunu kanıtlayarak aşırı öğrenme problemi yaşamıştır.
    - **Yüksek Önyargı:** Lojistik Regresyon, yüksek bir eğilim (bias) sergileyerek piyasa düşüş sinyallerini göz ardı etmiş ve tahminlerinde neredeyse her zaman Yükseliş yönünde karar vermiştir.
3. **AdaBoost ve Ensemble Başarısızlığı:** AdaBoost modelinin 0.50'nin altında (AUC:0.4966) kalması, en basit öğrenicilerin bile anlamlı bir örüntü yakalayamadığını ve topluluk öğrenme yönteminin dahi, piyasadaki rastgeleliği aşamadığını göstermiştir.

16.3. Çalışmanın Nihai Çıkarımları ve Finansal Çözümleme

Elde edilen ampirik sonuçlar, projenin temel bilimsel çıkarımını sunmaktadır: Klasik ML modelleri ve teknik göstergeler, BIST 30 fiyat yönünü güvenilir bir şekilde tahmin etmede yetersizdir.

- **Piyasa Etkinliği Hipotezi Desteği:** Düşük performans, finansal literatürdeki Rastgele Yürüyüş Teorisi'ne uyumlu olarak, geçmiş fiyat bilgisinin geleceği tahmin etmede kullanılamayacağına dair güçlü bir ampirik kanıt sunmaktadır (Fama, 1970). Bu durum, piyasanın Zayıf Formda Etkin olduğu hipotezini destekler.
- **Literatürdeki Yerimiz:** Çalışmamız, teknik göstergelere dayalı tahmin modellerinin, yüksek frekanslı ve gürültülü piyasalarda genellikle 0.50 civarında sonuçlar verdiğine dair literatürü doğrulamıştır. Elde edilen sonuçlar, basit teknik analize dayalı bir alım-satım stratejisinin **risk/getiri dengesini iyileştirmeyeceği** sonucunu desteklemektedir.

16.4. Gelecek Çalışmalar ve Metodolojik Öneriler

Bu çalışmanın sınırlarını aşarak tahmin performansını artırmak için aşağıdaki ileri düzey metodolojik ve veri bazlı yaklaşımlar önerilmektedir:

1. **Derin Öğrenme Metodolojisi (Long-Term Hafıza):**
    - **Öneri:** Zaman serisi verilerindeki uzun vadeli bağımlılıkları ve karmaşık doğrusal olmayan kalıpları yakalama yeteneği yüksek olan LSTM (Long Short-Term Memory) ve CNN-LSTM hibrit modelleri öncelikli olarak denenmelidir (Hochreiter & Schmidhuber, 1997).
    - **Gerekçe:** Klasik modellerin aksine, LSTM'in hafıza mekanizması, günler öncesinden gelen kritik piyasa verilerini unutmadan işleyebilir.
2. **Gelişmiş Özellik Mühendisliği (Sinyal Zenginleştirme):**
    - **Öneri:** Ham gösterge değerleri yerine, ikili al-sat sinyallerinin (MACD kesişimleri, RSI aşırı alım/satım sinyalleri) net özellikler olarak sisteme dahil edilmesi.
    - **Öneri:** Hacim / Ortalama Hacim gibi çapraz gösterge oranları (Inter-Indicator Ratios) oluşturularak modele anlık piyasa dengesizlikleri hakkında ek bilgi sunulmalıdır.
3. **Dışsal Veri Kaynaklarının Entegrasyonu (Faktör Zenginleştirme):**
    - **Öneri:** Piyasayı etkileyen makroekonomik faktörler (faiz, enflasyon) ve haber duyarlılığı **(sentiment analysis)** skorları modelin girdisi olarak kullanılmalıdır (Bollen et al., 2011).
    - **Gerekçe:** Modelin tahmin performansının artırılması, içsel teknik faktörler yerine, fiyatı hareket ettiren dışsal ve temel faktörlerin yakalanmasına bağlıdır.
4. **Metodolojik Optimizasyon ve Kapsam:** Proje kapsamında denenen ML modellerinin bile daha verimli çalışması için Grid Search gibi sistematik yöntemlerle hiper-parametre optimizasyonu yapılmalıdır.

17. KAYNAKÇA

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of Computational Science, 2*(1), 1–8.

Cattell, R. B. (1966). The scree test for the number of factors. *Multivariate Behavioral Research, 1*(2), 245–276.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297.

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, 13*(1), 21–27.

Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance, 25*(2), 383–417.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874.

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences, 55*(1), 119–139.

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.

Hawkins, D. M. (2004). The problem of overfitting. *Journal of Chemical Information and Modeling, 44*(1), 1–12.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley.

Hughes, G. (1968). On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory, 14*(1), 55–63.

Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and practice* (2nd ed.). OTexts.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning with applications in R*. Springer.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in Python*. Springer.

Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer.

Kaiser, H. F. (1960). The application of electronic computers to factor analysis. *Educational and Psychological Measurement, 20*(1), 141–151.

Kuhn, M., & Johnson, K. (2013). *Applied predictive modeling*. Springer.

Little, R. J. A., & Rubin, D. B. (2019). *Statistical analysis with missing data* (3rd ed.). Wiley.

Lopez de Prado, M. (2018). *Advances in financial machine learning*. Wiley.

Malkiel, B. G. (1973). *A random walk down Wall Street*. W. W. Norton & Company.

Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., … Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Proje Uygulama Kodu. (2025). *Toyota hisse senedi fiyat tahmini: Veri analizi ve SVM modellemesi*

Provost, F., & Fawcett, T. (2013). *Data science for business*. O'Reilly Media.

Schölkopf, B., Smola, A., & Müller, K. R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation, 10*(5), 1299–1319.

Scikit-learn Developers. (2024). *Scikit-learn: Machine learning in Python*.

Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding machine learning: From theory to algorithms*. Cambridge University Press.

Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.

Vapnik, V. N. (2000). *The nature of statistical learning theory* (2nd ed.). Springer.

Yahoo Finance API. (2025). *Toyota Motor Corporation (TM) historical stock price data (1980–2025)*.